MODULE-2

Symmetric Encryption Algorithms

Classical Cryptography

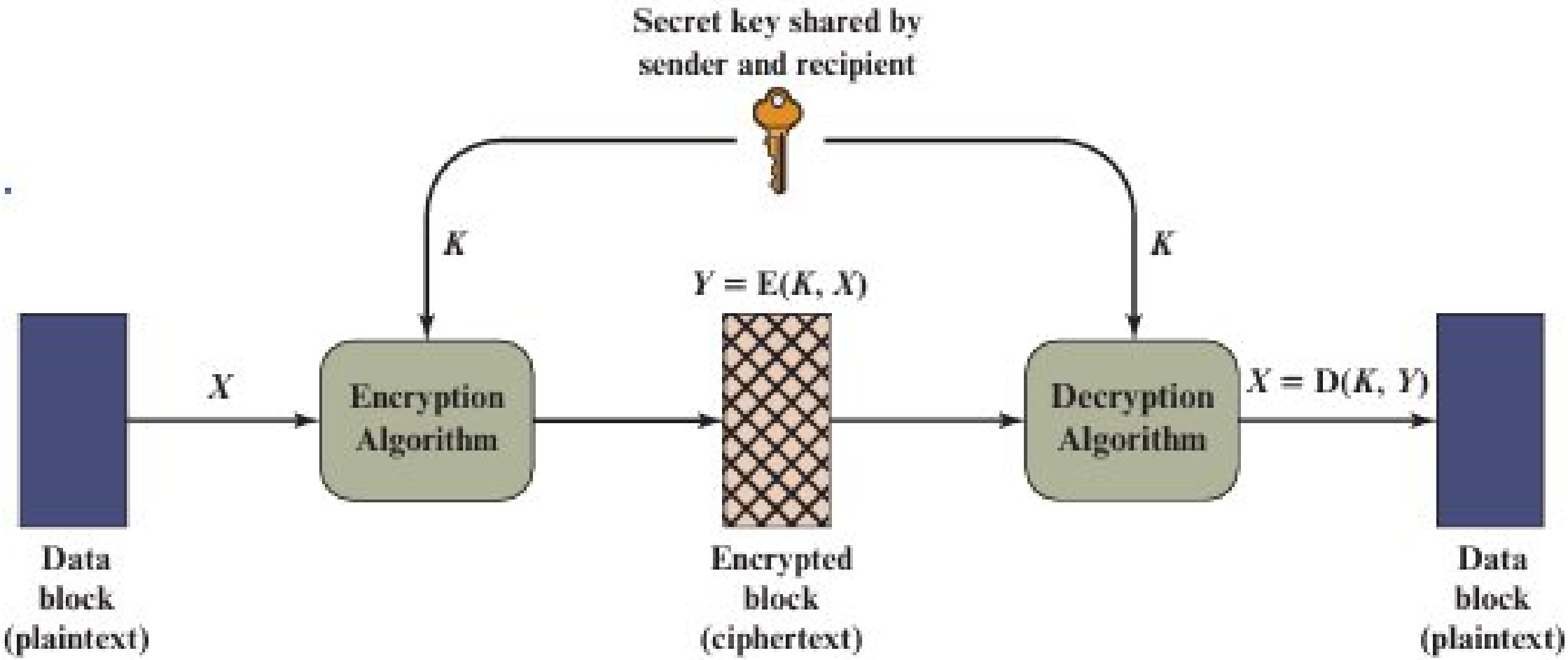
Definitions

- **Plain text**: original text which is fed into the algorithm
- **Encryption Algorithm (cipher)**: The algorithm used to encrypt the plain text using substitution and repositioning
- **Secret key**: It is an additional input to the encrytion algorithm along with the plain text

Definitions

- Ciphertext
  - It is the encrypted text, a random stream of data produced as output.
  - In a single key cryptography the ciphertext varies for two different keys for the same input for the same algorithm
- Decryption Algorithm
  - It is the encryption algorithm run in reverse
  - It takes the key and the encrypted text as input and produces the plain text as output

Single Key Encryption



Properties of Single Key Encryption

- **A strong algorithm** : An opponent who knows the algorithm and has the possession of one or more ciphertext along with the corresponding plain text should be unable to crack the key
- **A safe key sharing protocol:** Sender and receiver must have obtained the key secretly and must store it secretly

Model of a Symmetric Crypto System

- The plain text is usually a sequence of alphabets X=[X1,X2…Xm] from a finite set of alphabets. (Traditionally the 26 alphabets of english used in caps_
- The cipher text is a sequence of alphabets in classical cryptograpy Y=[Y1,Y2….Yn]
- In modern cryptography the plain text is binary
- The Key is a sequence K=[K1,K2,,Kj]

Cryptanalysis

- It is the method used by a hacker to estimate the plain text X’ given that
  - He knows the encryption and decryption algorithm
  - He knows one or more cipher text Y
  - He knows the corresponding plain text X
  - He does not know Key K
- It also includes the methods to decipher the Key K and produce estimates K’

Types of Cryptography

- Types of operation used for encryption
  - Substitution
  - Transposition
- Based on number of Keys Used
  - Key less
  - Single Key
  - Double Key
- Based on the size of the input plain text
  - Block text encryption
  - Stream Encryption

Crytopgraphic Operations

- **Two types of operations used for encryption are Substitution** and Transposition
- Fundamental requirement : No information should be lost. Given algorithm and the key the plain text should be completely recoverable from the ciphered text.
- Usually there are multiple stages of substitution and transposition called production system is used for encryption

Cryptographics Operations

- **Substitution**: Each element
  - A bit
  - An alphabet
  - A sequence of bits
  - A sequence of alphabets

  Is mapped in to a different element in the ciphered text.

Cryptographic Operations

- **Transposition**: The elements of a plain text are rearranged

Keys used

- Key less
- Single Key cryptography also called:
  - Classical (Conventional)
  - Symmetric
  - Secret Key (private key)
- Two Key Cryptography also called
  - Modern
  - Asymmetric
  - Public key

Block Vs Stream Cipher

- Block Cipher
  - The plain text is encrypted, one block of elements at a time by a block cipher producing a single block of cipher text at a time
- Stream Cipher
  - A stream of elements is continuously processed encrypting one element at a time in to a single element of cipher text.

Attacks

- Cryptanalytic attacks
- Brute-force attack

Cryptanalytic Attacks

- Cryptanalytic attacks :
  - rely on the nature of the algorithm
  - knowledge of the general characteristics of the plaintext
  - sample plaintext–ciphertext pairs.

  This type of attack exploits the charac teristics of the algorithm to attempt to deduce a specific plaintext or to deduce the key being used.

Brute-force Attacks

- Brute-force attack:
  - The attacker tries every possible key on a piece of cipher text until an intelligible translation into plaintext is obtained.
  - On average, half of all possible keys must be tried to achieve success

Type of Attack	Known to Cryptanalyst
Ciphertext Only	■ Encryption algorithm ■ Ciphertext
Known Plaintext	■ Encryption algorithm ■ Ciphertext ■ One or more plaintext–ciphertext pairs formed with the secret key
Chosen Plaintext	■ Encryption algorithm ■ Ciphertext ■ Plaintext message chosen by cryptanalyst, together with its corresponding ciphertext generated with the secret key
Chosen Ciphertext	■ Encryption algorithm ■ Ciphertext ■ Ciphertext chosen by cryptanalyst, together with its corresponding decrypted plaintext generated with the secret key
Chosen Text	■ Encryption algorithm ■ Ciphertext ■ Plaintext message chosen by cryptanalyst, together with its corresponding ciphertext generated with the secret key ■ Ciphertext chosen by cryptanalyst, together with its corresponding decrypted plaintext generated with the secret key

- Usually weak encryptions can be attacked with possession of only cipher text
- Generally encryptions are considered strong when they are resilient against known plain text methods

Unconditionally Secure Encryption

- An encryption is unconditionally secure if
  - Possession of any amount of cipher text
  - Possession of any amount of time for deciphering

  does not enable recovering the plain text because the cipher text does not have enough information to uniquely reproduce the output text
  - one time pad approach is the only algorithm known so far

Computationally Secure Encryption Algorithms

- For every algorithm which is not unconditionally secure
- It is considered computationally secure if it satisfies one of the following requirements
  - The cost of breaking the cipher exceeds the value of the encrypted information.
  - The time required to break the cipher exceeds the useful lifetime of the information.

NAS18 list of requirements for Strong Cipher

- Appropriate choice of cryptographic algorithm
- Use of sufficiently long key lengths
- Appropriate choice of protocols
- A well-engineered implementation
- The absence of deliberately introduced hidden flaws

Types of *Symmetric Key Cryptography:*

- Stream ciphers
- Block ciphers

Stream vs Block Ciphers

- stream ciphers process messages a bit or byte at a time when en/decrypting
- block ciphers process messages in blocks, each of which is then en/decrypted
- like a substitution on very big characters
  - 64-bits or more
- many current ciphers are block ciphers
- broader range of applications

Classical Cryptography Techniques

- **Substitution and Transposition are types of**
- **Substitution cipher** → Replaces letters with other letters/symbols.
  Example: Caesar cipher (A → D, B → E).
- **Transposition cipher** → Rearranges the positions of the letters without changing the letters themselves.
  Example: Rail Fence cipher.

Substitution Technique

- Monoalphabetic **cipher**
- Polyphabetic or **polygraphic cipher**

Monoalphabetic Cipher

- Permutation
  - Of a finite set of elements *S* is an ordered sequence of all the elements of *S* , with each element appearing exactly once
- If the “cipher” line can be any permutation of the 26 alphabetic characters, then there are 26! or greater than 4×10^{26} possible keys
  - This is 10 orders of magnitude greater than the key space for DES
  - Approach is referred to as a *monoalphabetic substitution cipher* because a single cipher alphabet is used per message

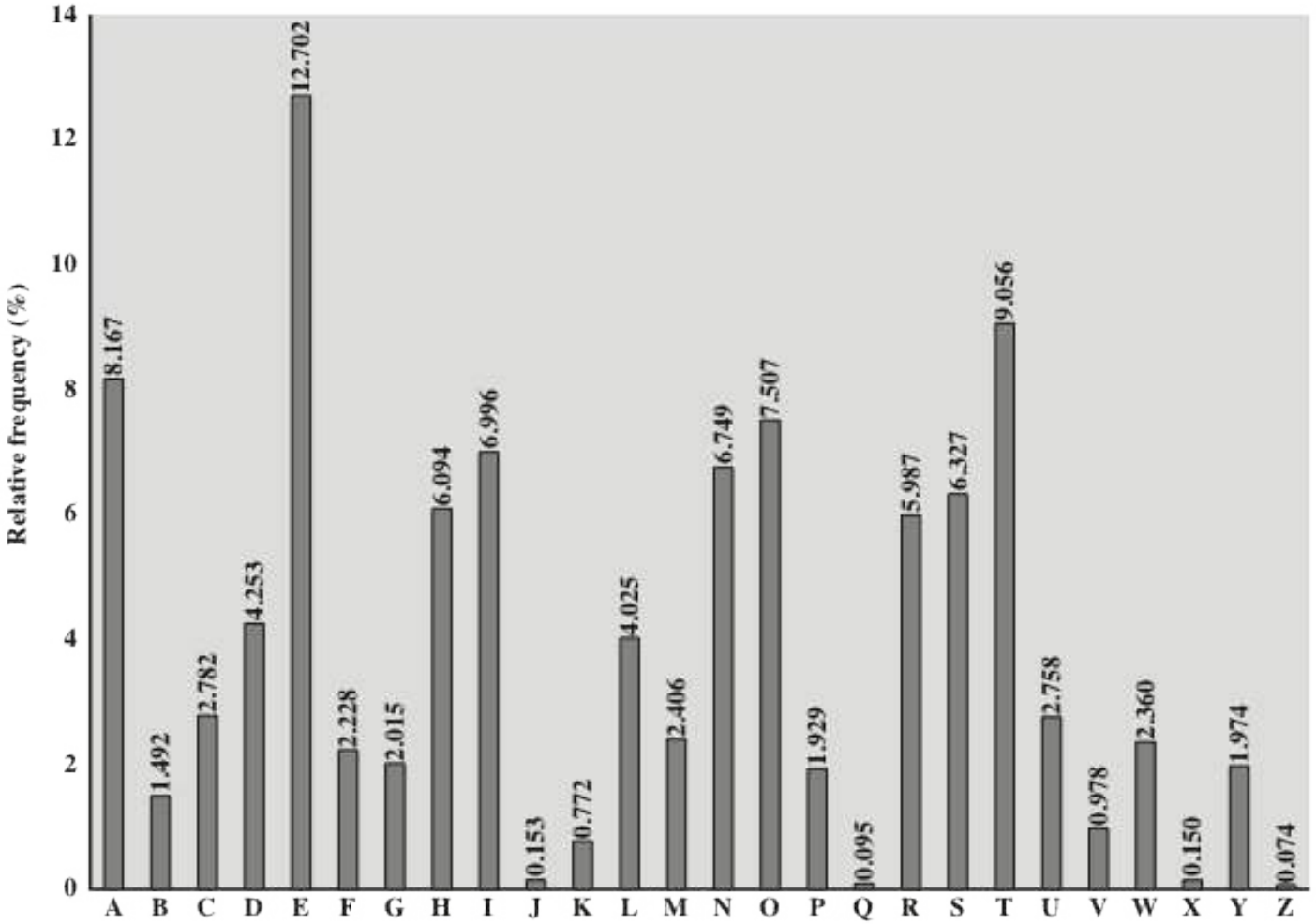
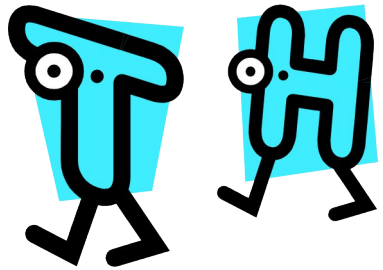
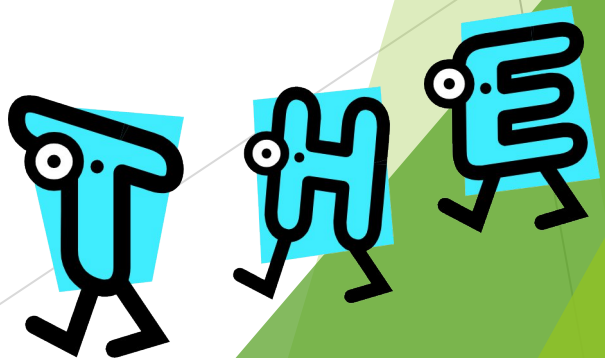


Figure 2.5 Relative Frequency of Letters in English Text

Monoalphabetic Ciphers

- Easy to break because they reflect the frequency data of the original alphabet
- Countermeasure is to provide multiple substitutes (homophones) for a single letter
- Digram
  - Two-letter combination
  - Most common is *th*
- Trigram
  - Three-letter combination
  - Most frequent is *the*





Caesar Cipher

- Simplest and earliest known use of a substitution cipher
- Used by Julius Caesar
- Caesar /additive/ Shift Cipher
- Involves replacing each letter of the alphabet with the letter standing **three** places further down the alphabet
- Alphabet is wrapped around so that the letter following Z is A

  plain: meet me after the toga party

  cipher: PHHW PH DIWHU WKH WRJD SDUWB

Caesar Cipher Algorithm (Key=3)

- Can define transformation as:

  a b c d e f g h i j k l m n o p q r s t u v w x y z

  D E F G H I J K L M N O P Q R S T U V W X Y Z A B C

- Mathematically give each letter a number

  a b c d e f g h i j k l m n o p q r s t u v w x y z

  0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25

- Algorithm can be expressed as:

  $c = E(3, p) = (p + 3) \bmod (26)$

- A shift may be of any amount, so that the general Caesar algorithm is:

  $$c = E(k , p) = (p + k) \bmod 26$$

- Where k takes on a value in the range 1 to 25; the decryption algorithm is simply:

  $p = D(k , c) = (c - k) \bmod 26$

Exercises

1. Encrypt the plain text “cryptography” using shift cipher with the key=4
2. break ciphertext "GCUA VQ DTGCM"

Polyalphabetic or Polygraphic Ciphers

- Polyalphabetic substitution cipher
  - Improves on the simple monoalphabetic technique by using different monoalphabetic substitutions as one proceeds through the plaintext message

All these techniques have the following features in common:

- A set of related monoalphabetic substitution rules is used
- A key determines which particular rule is chosen for a given transformation

Playfair Cipher

- Best-known multiple-letter encryption cipher
- Treats digrams in the plaintext as single units and translates these units into ciphertext digrams
- Based on the use of a 5 x 5 matrix of letters constructed using a keyword

Playfair Key Matrix

- a 5X5 matrix of letters based on a keyword
- Fill in letters of keyword (minus duplicates) from left to right and from top to bottom, then fill in the remainder of the matrix with the remaining letters in alphabetic order

- eg. using the keyword MONARCHY

M	O	N	A	R
C	H	Y	B	D
E	F	G	I/J	K
L	P	Q	S	T
U	V	W	X	Z

Substitution techniques –Summary

Playfair Cipher

M	O	N	A	R
C	H	Y	B	D
E	F	G	I/J	K
L	P	Q	S	T
U	V	W	X	Z

M	O	N	A	R
C	H	Y	B	D
E	F	G	I/J	K
L	P	Q	S	T
U	V	W	X	Z

- In this case, the keyword is *monarchy*.
- Plaint Text : **Instruments**

- The matrix is constructed by filling in the letters of the keyword (minus duplicates) from left to right and from top to bottom, and then filling in the remainder of the matrix with the remaining letters in alphabetic order. The letters I and J count as one letter.

- **The Playfair Cipher Encryption Algorithm:**
  The Algorithm consists of 2 steps:

- **Generate the key Square(5×5):**
  - The key square is a 5×5 grid of alphabets that acts as the key for encrypting the plaintext. Each of the 25 alphabets must be unique and one letter of the alphabet (usually J) is omitted from the table (as the table can hold only 25 alphabets). If the plaintext contains J, then it is replaced by I.
  - The initial alphabets in the key square are the unique alphabets of the key in the order in which they appear followed by the remaining letters of the alphabet in order.

- **Algorithm to encrypt the plain text:**
  - The plaintext is split into pairs of two letters (digraphs). If there is an odd number of letters, a Z is added to the last letter.

- **PlainText: "instruments"**
- **After Split: 'in' 'st' 'ru' 'me' 'nt' 'sz'**

1. Pair cannot be made with same letter. Break the letter in single and add a bogus letter to the previous letter.

- **Plain Text:** “hello”
- **After Split:** ‘he’ ‘lx’ ‘lo’
- Here ‘**x**’ is the bogus letter.

2. If the letter is standing alone in the process of pairing, then add an extra bogus letter with the alone letter

- **Plain Text:** “helloe”
- **AfterSplit:** ‘he’ ‘lx’ ‘lo’ ‘ez’
- Here ‘**z**’ is the bogus letter.

- **Rules for Encryption: After Split: 'in' 'st' 'ru' 'me' 'nt' 'sz'**

  Rule No:1

- **If both the letters are in the same column**: Take the letter below each one (going back to the top if at the bottom).

Example:

- Diagraph: "me"
- Encrypted Text: cl
- Encryption:
  - m -> c
  - e -> l

M	O	N	A	R
C	H	Y	B	D
E	F	G	I	K
L	P	Q	S	T
U	V	W	X	Z

- Rule 2: **After Split: 'in' 'st' 'ru' 'me' 'nt' 'sz'**
- **If both the letters are in the same row**: Take the letter to the right of each one (going back to the leftmost if at the rightmost position).

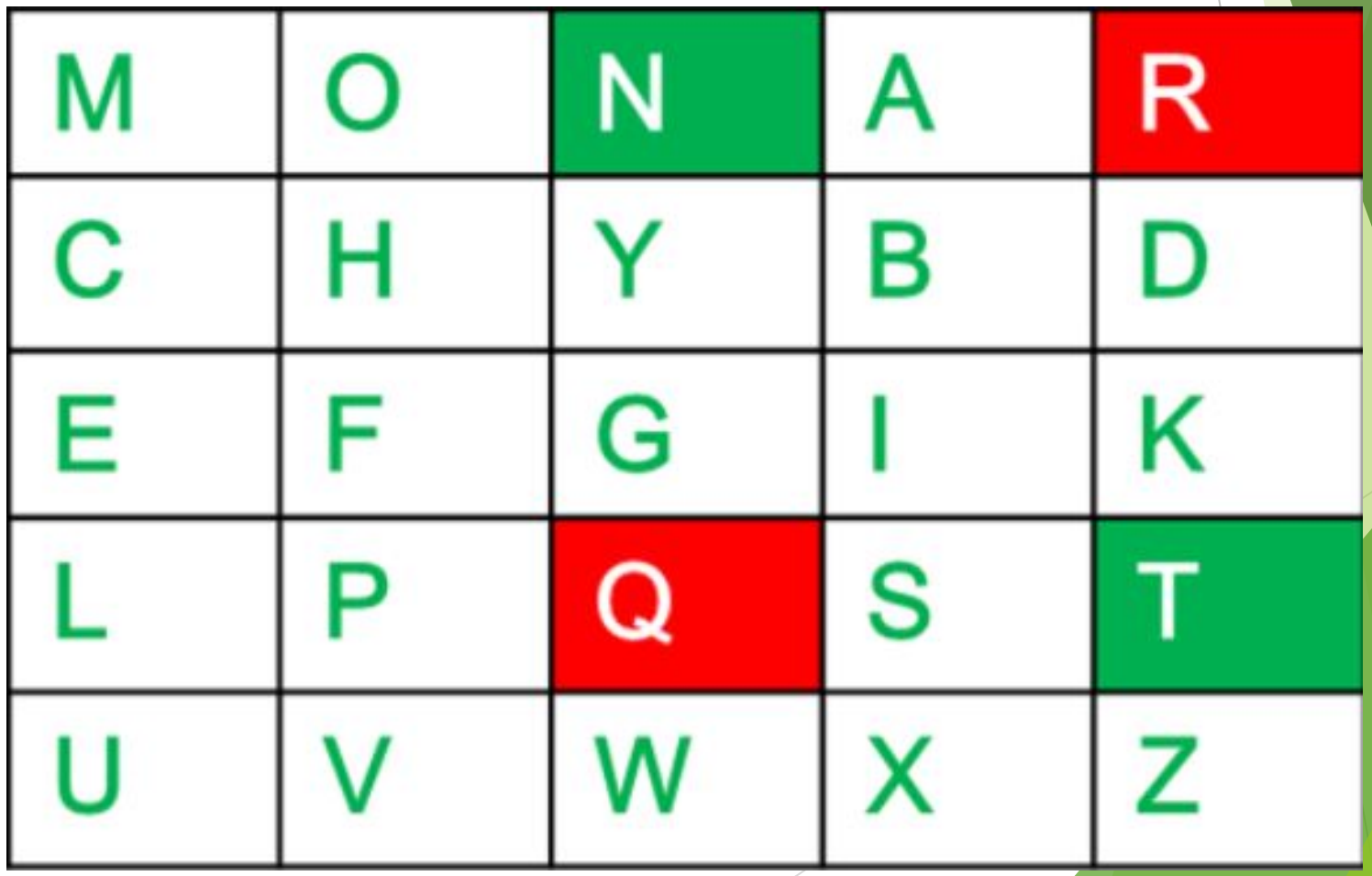
- Diagraph: "st"
- Encrypted Text: tl
- Encryption:
  - s -> t
  - t -> l

M	O	N	A	R
C	H	Y	B	D
E	F	G	I	K
L	P	Q	S	T
U	V	W	X	Z

- **Rule 3:** **After Split: 'in' 'st' 'ru' 'me' 'nt' 'sz'**

If neither of the above rules is true: Form a rectangle with the two letters and take the letters on the horizontal opposite corner of the rectangle

- Diagraph: "nt"
- Encrypted Text: rq
- Encryption:
- n -> r
- t -> q.



- Plain Text: "instrumentsz"
- Encrypted Text: gatlmzclrqtx
- Encryption:
  - i -> g
  - n -> a
  - s -> t
  - t -> l
  - r -> m
  - u -> z
  - m -> c
  - e -> l
  - n -> r
  - t -> q
  - s -> t
  - z -> x

in:

M	O	N	A	R
C	H	Y	B	D
E	F	G	I	K
L	P	Q	S	T
U	V	W	X	Z

st:

M	O	N	A	R
C	H	Y	B	D
E	F	G	I	K
L	P	Q	S	T
U	V	W	X	Z

ru:

M	O	N	A	R
C	H	Y	B	D
E	F	G	I	K
L	P	Q	S	T
U	V	W	X	Z

me:

M	O	N	A	R
C	H	Y	B	D
E	F	G	I	K
L	P	Q	S	T
U	V	W	X	Z

nt:

M	O	N	A	R
C	H	Y	B	D
E	F	G	I	K
L	P	Q	S	T
U	V	W	X	Z

sz:

M	O	N	A	R
C	H	Y	B	D
E	F	G	I	K
L	P	Q	S	T
U	V	W	X	Z

Security of the Playfair Cipher

- security much improved over monoalphabetic
- since have 26 x 26 = 676 diagrams
- would need a 676 entry frequency table to analyse (verses 26 for a monoalphabetic)
- and correspondingly more ciphertext
- was widely used for many years (eg. US & British military in WW1)
- it **can** be broken, given a few hundred letters since it still has much of plaintext structure

Encrypting and Decrypting

- plaintext is encrypted two letters at a time
  1. if a pair is a repeated letter, insert filler like 'X'
  2. if both letters fall in the same row, replace each with letter to right (wrapping back to start from end)
  3. if both letters fall in the same column, replace each with the letter below it (again wrapping to top from bottom)
  4. otherwise each letter is replaced by the letter in the same row and in the column of the other letter of the pair

Key-MONARCHY : P- BALLOON

- Input : BA | LL| OO | N
- Add filler to remove replication

BA | LX| LO |ON

W.r.t Key : Encryption

BA = IB

LX = SU

LO= PM

ON = NA

CT = IB SU PM NA

M	O	N	A	R
C	H	Y	B	D
E	F	G	I/J	K
L	P	Q	S	T
U	V	W	X	Z

Cipher : IB SU PM NA

- No replication or no odd pair
- W.r.t Key : Decryption

IB = BA

SU = LX

PM = LO

NA = ON

- BA | LX| LO |ON
- Remove the Filler
- **PT = BALLOON**

Exercise

- Use playfair cipher to encrypt the text “informations” with the key “network”

Security of Playfair Cipher

- security much improved over monoalphabetic
- since have 26 x 26 = 676 diagrams
- would need a 676 entry frequency table to analyse (verses 26 for a monoalphabetic)
- and correspondingly more ciphertext
- was widely used for many years
- it **can** be broken, given a few hundred letters
- since still has much of plaintext structure

Hill Ciphers

- Lester Hill, 1929. first time linear algebra used in crypto.
- multiletter cipher
- Block cipher
- Uses Matrix arithmetic modulo 26
- Use an *n* x *n* matrix M.
- Encrypt by breaking plaintext into blocks of length *n* (padding with x's if needed) and multiplying each by M (mod 26).
- Encryption $C = P * K \bmod 26$
- Decryption $P = C * K^{-1} \bmod 26$
- The key matrix chosen must have an inverse for it ($K.\ K^{-1}$ = **Identity matrix). Only then decryption is possible**

Hill example 1

a	b	c	d	e	f	g	h	i	j	k	l	m
0	1	2	3	4	5	6	7	8	9	10	11	12

n	o	p	q	r	s	t	u	v	w	x	y	z
13	14	15	16	17	18	19	20	21	22	23	24	25

- For example, consider the plaintext "pay mor emo ney" and use the encryption key. K = $\begin{bmatrix} 17 & 17 & 5 \\ 21 & 18 & 21 \\ 2 & 2 & 19 \end{bmatrix}$
- The first three letters of the plaintext "pay" are represented by the vector
- E = P.K mod 26
- P=(15 0 24) for "pay"
- Then $(15\ 0\ 24)\ X \begin{bmatrix} 17 & 17 & 5 \\ 21 & 18 & 21 \\ 2 & 2 & 19 \end{bmatrix}$ = (303 303 531) mod 26 = (17 17 11) = RRL.
- Continuing in this fashion, the ciphertext for the entire plaintext is RRL MWB KAS PDH.

Hill Cipher

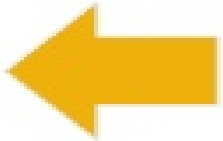
- Strength is that it completely hides single-letter frequencies
  - The use of a larger matrix hides more frequency information
  - A 3 x 3 Hill cipher hides not only single-letter but also two-letter frequency information
- Strong against a ciphertext-only attack but easily broken with a known plaintext attack

Vigenère Cipher

- Best known and one of the simplest polyalphabetic substitution ciphers
- effectively multiple caesar ciphers
- key is multiple letters long $K = k_1\ k_2\ \ldots\ k_d$
- i^{th} letter specifies i^{th} alphabet to use
- use each alphabet in turn
- repeat from start after d letters in message
- decryption simply works in reverse
- In this scheme, the set of related monoalphabetic substitution rules consists of the 26 Caesar ciphers with shifts of 0 through 25
- Each cipher is denoted by a key letter which is the ciphertext letter that substitutes for the plaintext letter a

Example of Vigenère Cipher

- **Plain Text:** ATTACKATDAWN
- **Key:** LEMON
- **Plain text:** ATTACKATDAWN
- **Key:** LEMONLEMONLE
- **Cipher text:**
- LXFOPVEFRNHR



	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
A	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
B	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A
C	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B
D	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C
E	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D
F	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E
G	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F
H	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G
I	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H
J	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I
K	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J
L	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K
M	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L
N	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M
O	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N
P	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
Q	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
R	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q
S	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
T	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S
U	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T
V	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U
W	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V
X	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W
Y	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X
Z	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y

Example of Vigenère Cipher

- To encrypt a message, a key is needed that is as long as the message
- Usually, the key is a repeating keyword
- For example, if the keyword is *deceptive*, the message "we are discovered save yourself" is encrypted as:

key: deceptivedeceptivedeceptive

plaintext: wearediscoveredsaveyourself

ciphertext: ZICVTWQNGRZGVTWAVZHCQYGLMGJ

Vigenère Autokey System

- A keyword is concatenated with the plaintext itself to provide a running key
- Example:

  key: deceptivewearediscoveredsav

  plaintext: wearediscoveredsaveyourself

  ciphertext: ZICVTWQNGKZEIIGASXSTSLVVWLA
- Even this scheme is vulnerable to cryptanalysis
  - Because the key and the plaintext share the same frequency distribution of letters, a statistical technique can be applied

Transposition Ciphers

- now consider classical **transposition** or **permutation** ciphers
- these hide the message by rearranging the letter order
- without altering the actual letters used
- can recognise these since have the same frequency distribution as the original text

Rail Fence Cipher

- Simplest transposition cipher
- Plaintext is written down as a sequence of diagonals and then read off as a sequence of rows
- To encipher the message “meet me after the toga party” with a rail fence of depth 2, we would write:

```
m e m a t r h t g p r y
  e t e f e t e o a a t
```

Encrypted message is:

MEMATRHTGPRYETEFETEOAAT

Row Transposition Cipher

- Is a more complex transposition
- Write the message in a rectangle, row by row, and read the message off, column by column, but permute the order of the columns
  - The order of the columns then becomes the key to the algorithm

Key: 4 3 1 2 5 6 7

Plaintext:
a t t a c k p
o s t p o n e
d u n t i l t
w o a mx y z

Ciphertext: TTNAAPTMTSUOAODWCOIXKNLYPETZ

Exercise-2

- Rail Fence Cipher:
  - message :“Happy Winter Vacation”
  - Depth : 3
- Row Transposition Cipher:
  - message :“Happy Winter Vacation”
  - Key : 4 3 1 2 5 6 7

Introduction to Stream Cipher

Stream Cipher

- Encrypts data one bit or byte at a time.
- Uses a keystream generated from a secret key.
- Ciphertext = Plaintext ⊕ Keystream.
- Examples: RC4, ChaCha20.

  - RC4 stands for Rivest Cipher 4, a fast and simple stream cipher algorithm invented by Ron Rivest in 1987, known for its use in protocols like SSL/TLS but now considered insecure due to vulnerabilities, leading to its prohibition in modern standards like TLS.

  - ChaCha20 is a fast, secure stream cipher by Daniel J. Bernstein, using Add-Rotate-XOR (ARX) operations on a 4x4 matrix of 32-bit words (512-bit state) to generate a keystream, which is then XORed with plaintext for encryption.

Stream Cipher

- A pseudorandom keystream is generated from the secret key.
- Each plaintext bit/byte is XORed with the keystream.
- Fast and suitable for real-time data (audio, video, streaming).
- If the same keystream is reused, security is broken.

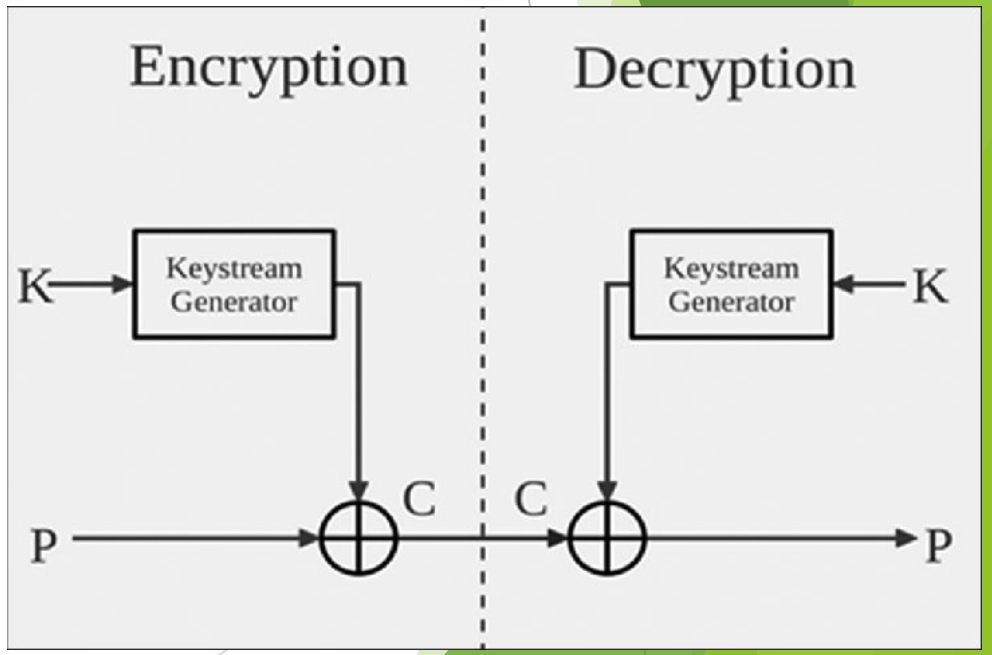
Stream Cipher

- Plaintext: "CRYPTOGRAPHY IS IMPORTANT"
- Each character is taken one by one.
- Each character is XORed with the keystream character.
- Output characters form the ciphertext stream.

Plaint Text : C R Y P T O G R A P H Y I S I M P O R T A N T

Key Streams : K1 K2 K3 K4 K5 K6 K7 K8 K9 K10 K11 K12 K13 ...

$$C \oplus K1 \rightarrow C1$$
$$R \oplus K2 \rightarrow C2$$
$$Y \oplus K3 \rightarrow C3$$
$$\ldots$$
$$T \oplus Kn \rightarrow Cn$$



Ciphertext is formed as a **stream**: C1 C2 C3 C4 C5 C6 ...

XOR

A	B	A ⊕ B
0	0	0
0	1	1
1	0	1
1	1	0

For two input bits **A** and **B**:

P = 10101010

K = 11001100

C = P ⊕ K

= 10101010 ⊕ 11001100

= 01100110

How multimedia are encrypted

- Plain Text → ASCII value → Binary → XOR with Key → Binary → Cipher Text

H- 72
E- 69
L- 76
L- 76
O- 79

H	72	01001000
E	69	01000101
L	76	01001100
L	76	01001100
O	79	01001111

Encryption: Key is “KEY”

- Padding:
  - PT: HELLO
  - K : KEYKE (padding)

K	75	01001011
E	69	01000101
Y	89	01011001
K	75	01001011
E	69	01000101

Plain Text	Key	Cipher Text
H	K	00000011
E	E	00000000
L	Y	00010101
L	K	00000111
O	E	00001010

Cipher Text: 00000011 00000000 00010101 00000111 00001010

Decryption:

Cipher Text : 00000011 00000000 00010101 00000111 00001010

Key : 01001011 01000101 01011001 01001011 01000101

K E Y K E

Cipher : 00000011

⊕

Key : 01001011 (K)

Plain : 01001000 → 72 → H (BINARY TO DECIMAL-ASCII values)

Cipher : 00000000

⊕

Key : 01000101 (E)

Plain : 01000101 → 69 → E

Block Cipher

- Encrypts data in fixed-size blocks
- Works block by block
- More structured
- Strong security

- Example block size: 8 or 16 characters
- Uses rounds of substitution and permutation

- Encrypts fixed-size blocks of data (e.g. 128 bits).
- Same key is used for each block.
- Examples: DES, AES

Block Cipher

- Plaintext is divided into fixed-size blocks.
- Plaint Text : C R Y P T O G R A P H Y I S I M P O R T A N T

  Block 1: CRYPTOGR

  Block 2: APHY IS

  Block 3: IMPORTAN

  Block 4: T (padded)
- Each block is encrypted independently or using modes.

  Block 1 + Key → Cipher Block 1

  Block 2 + Key → Cipher Block 2

  Block 3 + Key → Cipher Block 3

  Block 4 + Key → Cipher Block 4

- CRYPTOGR | APHY IS | IMPORTAN | T + padding
- ↓ ↓ ↓ ↓
- CB1 CB2 CB3 CB4

- CB1 | CB2 | CB3 | CB4

Block Cipher works in different modes

- Modes of operation: ECB, CBC, CFB, OFB.
- Provides strong security when used with proper modes.

- ECB – Electronic Codebook (Mode)
- CBC – Cipher Block Chaining (Mode)
- CFB – Cipher Feedback (Mode)
- OFB – Output Feedback (Mode)

Modes of operation: ECB, CBC, etc

- ECB and CBC are most commonly used modes.
- **Electronic Codebook (ECB) Mode:**
- Output of Block 1 is **NOT** used as input to Block 2

Plaintext Block 1 —Encrypt→ Cipher Block 1

Plaintext Block 2 —Encrypt→ Cipher Block 2

In ECB mode, each plaintext block is encrypted independently, and the ciphertext of one block does not affect the encryption of the next block.

Cipher Block Chaining (CBC) Mode:

- Output of Block 1 is used for Block 2

Plaintext Block 1 ⊕ IV —Encrypt→ Cipher Block 1

Plaintext Block 2 ⊕ Cipher Block 1 —Encrypt→ Cipher Block 2

Initialization Vector (IV) is a random or pseudo-random value used only for the first block of encryption in CBC mode.

Block Cipher

- Plaintext paragraph is split into blocks of equal size.
- Each block is encrypted separately.
- Example: Paragraph → Block1 | Block2 | Block3.
- Each encrypted block forms ciphertext.

Stream vs Block Cipher

- Stream: bit/byte by bit, fast, real-time data.
- Block: fixed-size blocks, stronger structure.
- Stream uses XOR; Block uses rounds and transformations.
- Both are widely used in modern cryptography.

DES- Data Encryption Standard

DES- Data Encryption Standard

- DES stands for Data Encryption Standard, a symmetric-key block cipher that encrypts data in 64-bit blocks.
- DES (Data Encryption Standard) uses a Feistel network with 16 identical rounds. Each round processes half of the data and mixes it with a round key.

DES Structure:

- Block size: 64 bits
- Rounds: 16
- Main Key : 64 bits
- Sub Key size: 56 bits (out of 64 bits only 56 bits only used, 8 bits are parity bits- last bit of every 8 bits pair is the parity bit)
- Round Key: 48 bits (key compression Permuted Choice-2)
- Structure: Feistel
- Generates 16 round keys

- DES uses a 16-round Feistel network where the 64-bit plaintext is split into two 32-bit halves.
- In each round, the right half is processed through the function F and XOR with the left half.
- The halves are swapped and after 16 rounds, a final permutation produces the ciphertext.

Claude Shannon and Substitution-Permutation Ciphers

- Claude Shannon introduced idea of substitution-permutation (S-P) networks in 1949 paper
- form basis of modern block ciphers
- S-P nets are based on the two primitive cryptographic operations seen before:
  - *substitution* (S-box)
  - *permutation* (P-box)
- provide *confusion* & *diffusion* of message & key

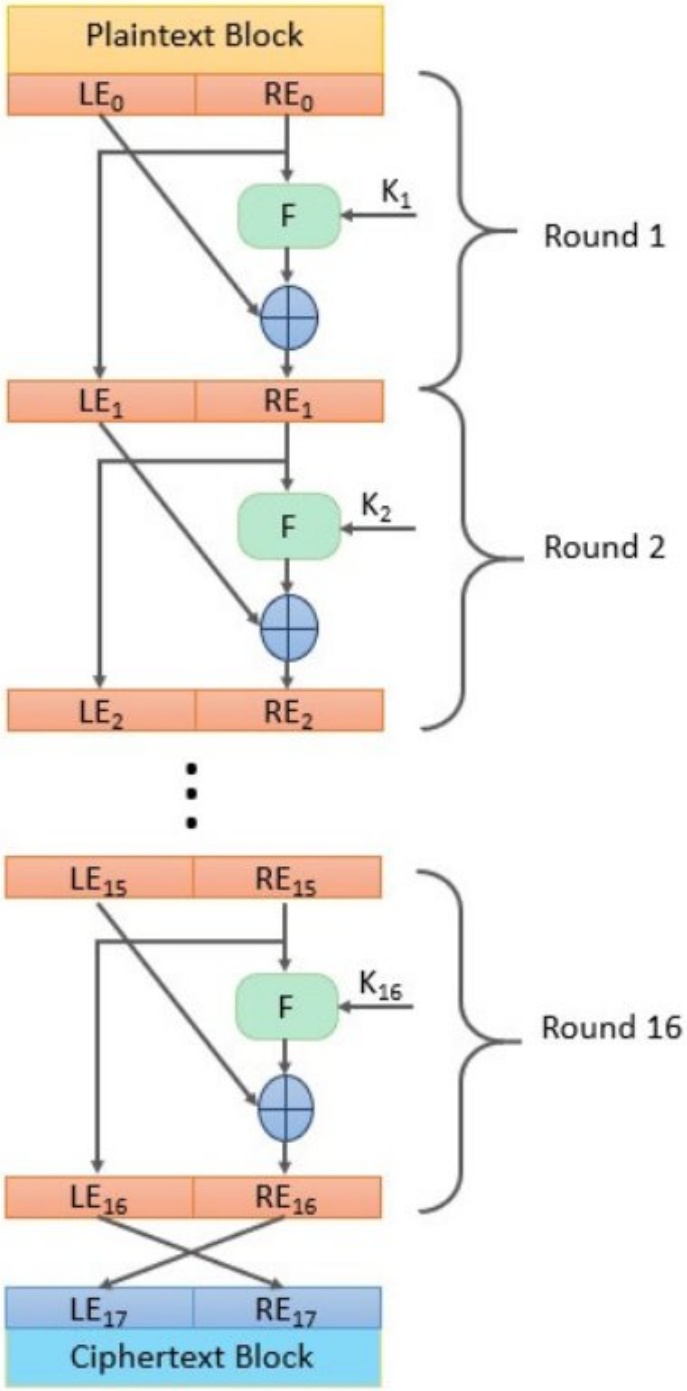
DES Block Cipher

- The Data Encryption Standard (DES):
- This algorithm adopted in 1977 by the National Institute of Standards and Technology (NIST). The algorithm itself is referred to as the Data Encryption Algorithm (DEA). For DES, data are encrypted in 64-bit blocks using a 56-bit key. The algorithm transforms 64-bit input in a series of steps into a 64-bit output. The same steps, with the same key, are used to reverse the encryption.

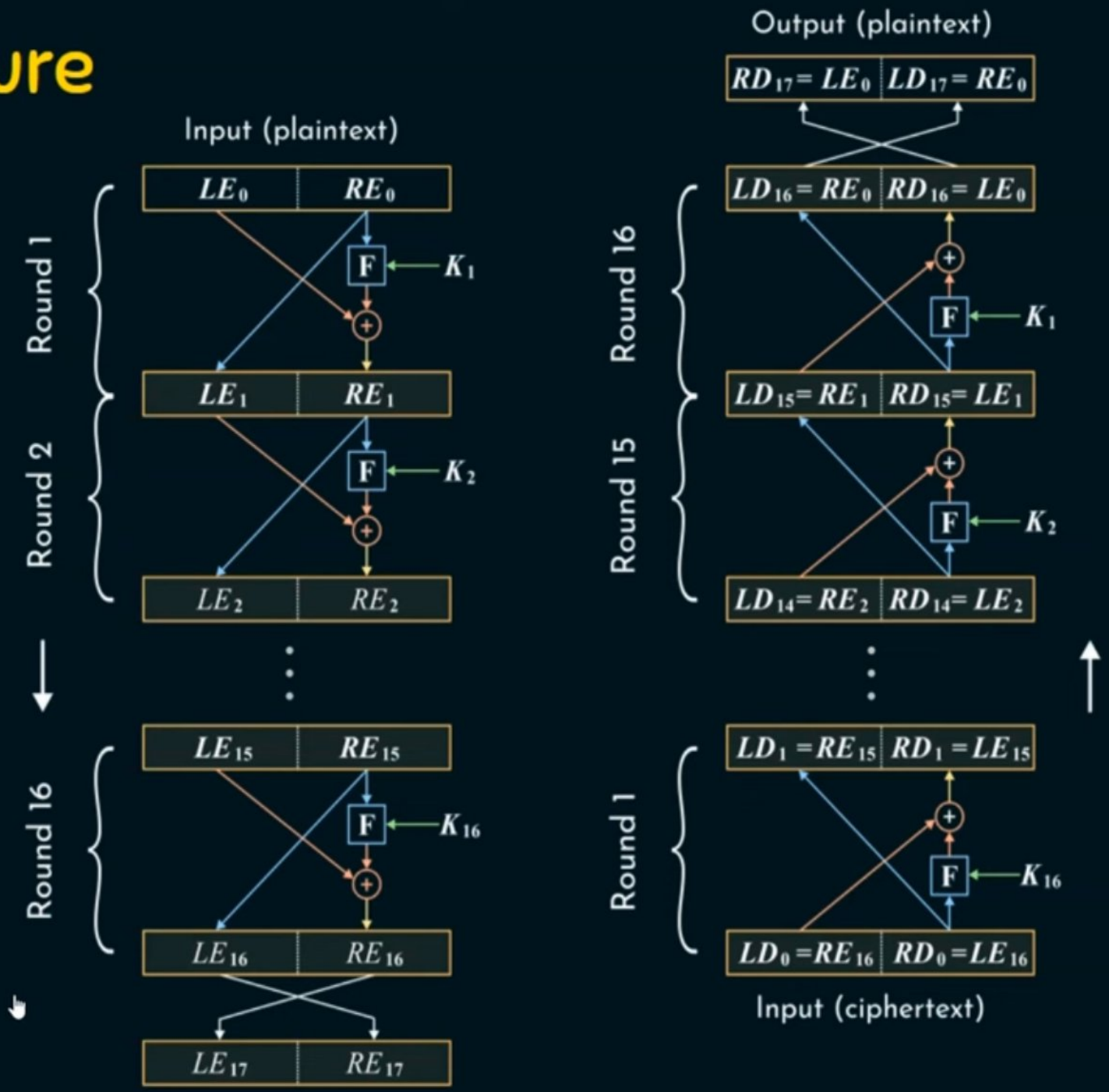
- DES encryption algorithm:
- The general structure of the DES consists of (1) key schedule, (2) round function and (3) initial and final permutation.
- Step1: Plaintext is broken into blocks of length 64 bits each.
- Step2: The 64-bit block undergoes an initial permutation (IP) using initial permutation IP table, IP(M). IP simply reorders bits.
- Step3: The 64-bit permuted input is divided into two 32-bit blocks: left (L) and right (R). The initial values of the left and right blocks are denoted L_0 and R_0.
- Step4: There are 16 rounds of operations on the L and R blocks. During each round, the following formula is applied:
- $L_n = R_{n-1}$
  $R_n = L_{n-1}$ XOR $F(R_{n-1}, K_n)$

$L_n = R_{n-1}$

$R_n = L_{n-1}\ XOR\ F(R_{n-1}, K_n)$



Feistel Structure

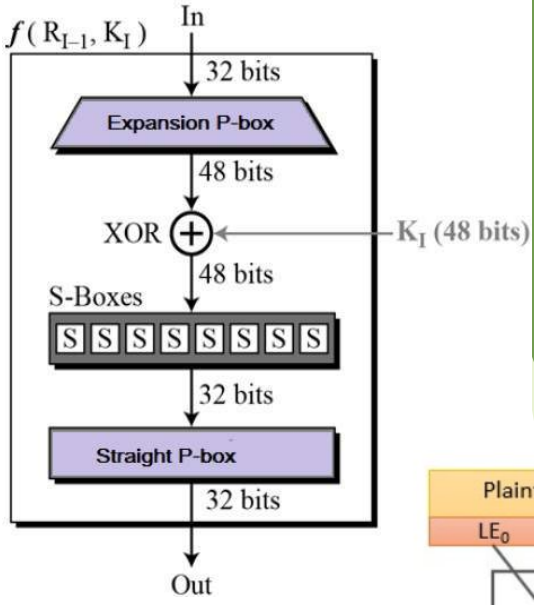


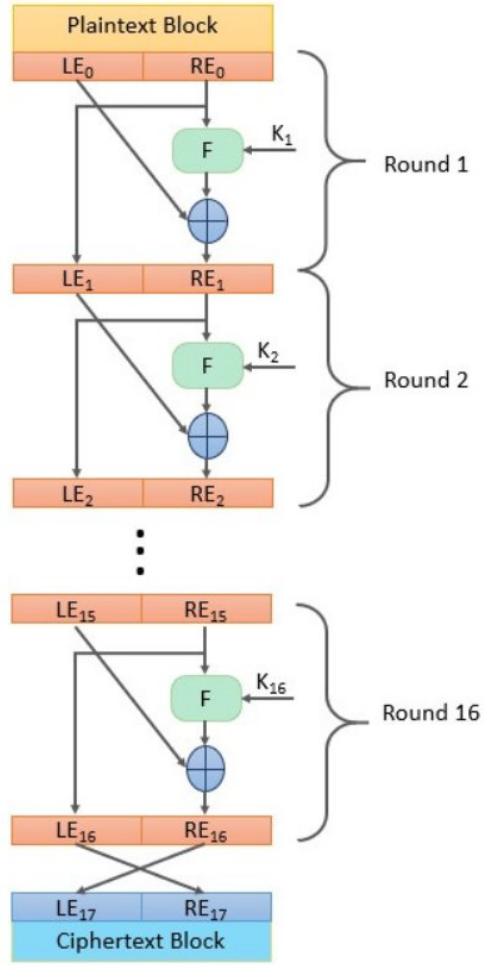
Feistel Cipher Design Elements

- block size
- key size
- number of rounds
- subkey generation algorithm
- round function
- fast software en/decryption
- ease of analysis

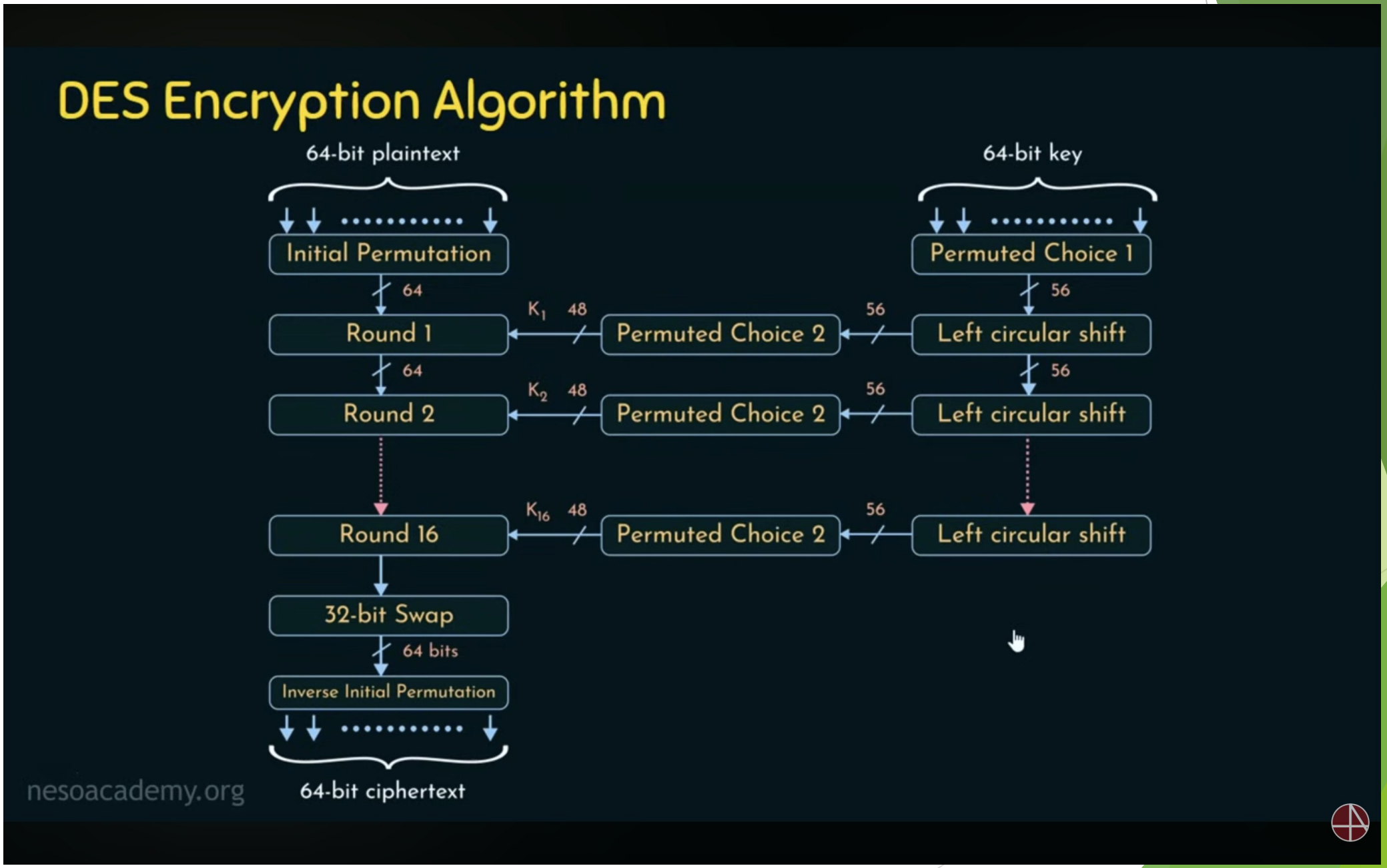
DES Block Cipher

- **Step5**: The function **F(.)** represents the heart of the DES algorithm. This function implements the following operations:
- **1-Expansion:** The right **32-bit** half-block is expanded to **48 bits** using the **expansion permutation** (**E**) table, $E(R_{n-1})$.
- **2-Key mixing:** The expanded result is combined with a **subkey** using an XOR operation. Sixteen 48-bit subkeys (one for each round) are derived from the main key using the **key schedule**, $K_n + E(R_{n-1})$.
- **3-Substitution:** After mixing in the subkeys, the block is divided into eight 6-bit pieces and fed into the substitution boxes (S-boxes), which implements nonlinear transformation. Each 6-bit piece uses as an address in the **S-boxes** where the first and last bits are used to address the **i^{th}** row and the middle four bits to address the **j^{th}** column in the S-boxes. The output of each **S-box is 4-bit length** piece. The output of all **eight S-boxes** is then combined into **32** bit section.
- $K_n + E(R_{n-1}) = B_1B_2B_3B_4B_5B_6B_7B_8$
- $S(Kn + E(Rn-1)) = S1(B_1)S2(B_2)S3(B_3)S4(B_4)S5(B_5)S6(B_6)S7(B_7)S8(B_8)$
- **4-Permutation:** The **32** bits outputs from the S-boxes are rearranged using the **P-box**, $F=P(S(K_n + E(R_{n-1})))$
- **Step6:** The results from the final DES round (i.e., L_{16} and R_{16}) are recombined into a 64-bit value and rearranged using an inverse initial permutation **(IP^{-1})** table. The output from IP^{-1} is the 64-bit ciphertext block.

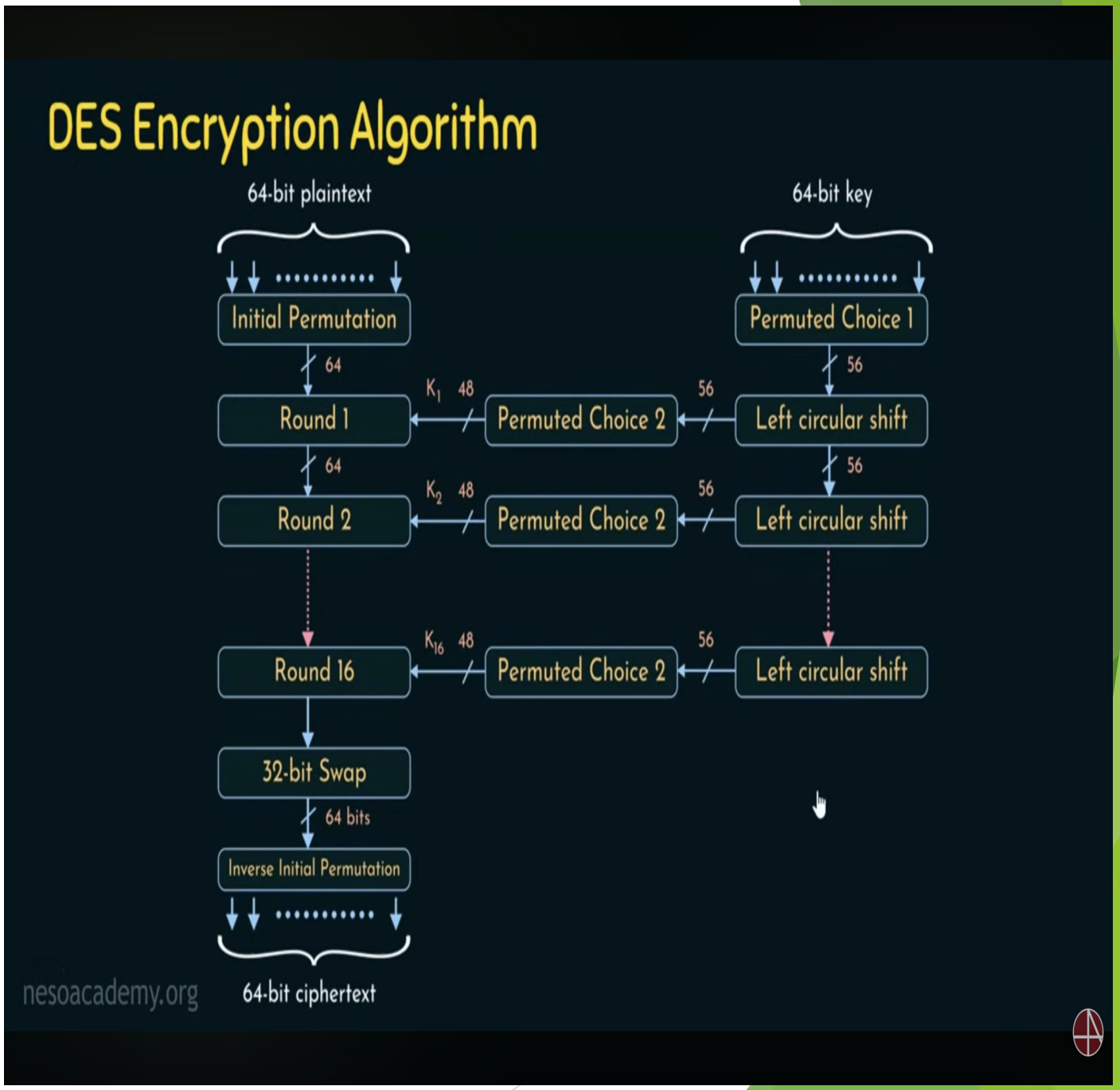




DES Encryption Architecture



Plaintext Process



Initial Permutation IP

- first step of the data computation
- IP reorders the input data bits
- even bits to LH half, odd bits to RH half
- quite regular in structure (easy in h/w)

IP Table:

58	50	42	34	26	18	10	2
60	52	44	36	28	20	12	4
62	54	46	38	30	22	14	6
64	56	48	40	32	24	16	8
57	49	41	33	25	17	9	1
59	51	43	35	27	19	11	3
61	53	45	37	29	21	13	5
63	55	47	39	31	23	15	7

- The table specifies the new order of the 64 bits, mapping each bit position in the input block to its new position in the permuted block.
- **Example of Initial Permutation (IP):**

1. **Input (Plaintext Block):**

Sample Plaintext = HELLOWORLD

Convert Plaintext to Binary

- Length = **10 characters- 80 bits**
- DES needs **64 bits = 8 bytes**,

01001000 01000101 01001100 01001100 -> HELL

01001111 01010111 01001111 01010010 -> OWOR

01001100 01000100 (next block)

- **Apply the IP Table**: Using the table, map the bits to their new positions:

- Bit 58 of the input goes to position 1 in the output,
- Bit 50 of the input goes to position 2 in the output,
- Bit 42 of the input goes to position 3 in the output, and so on
- Bit 7 of the input goes to position 64 in the output, ends

01001000 01000101 01001100 01001100 -> HELL
01001111 01010111 01001111 01010010 -> OWOR

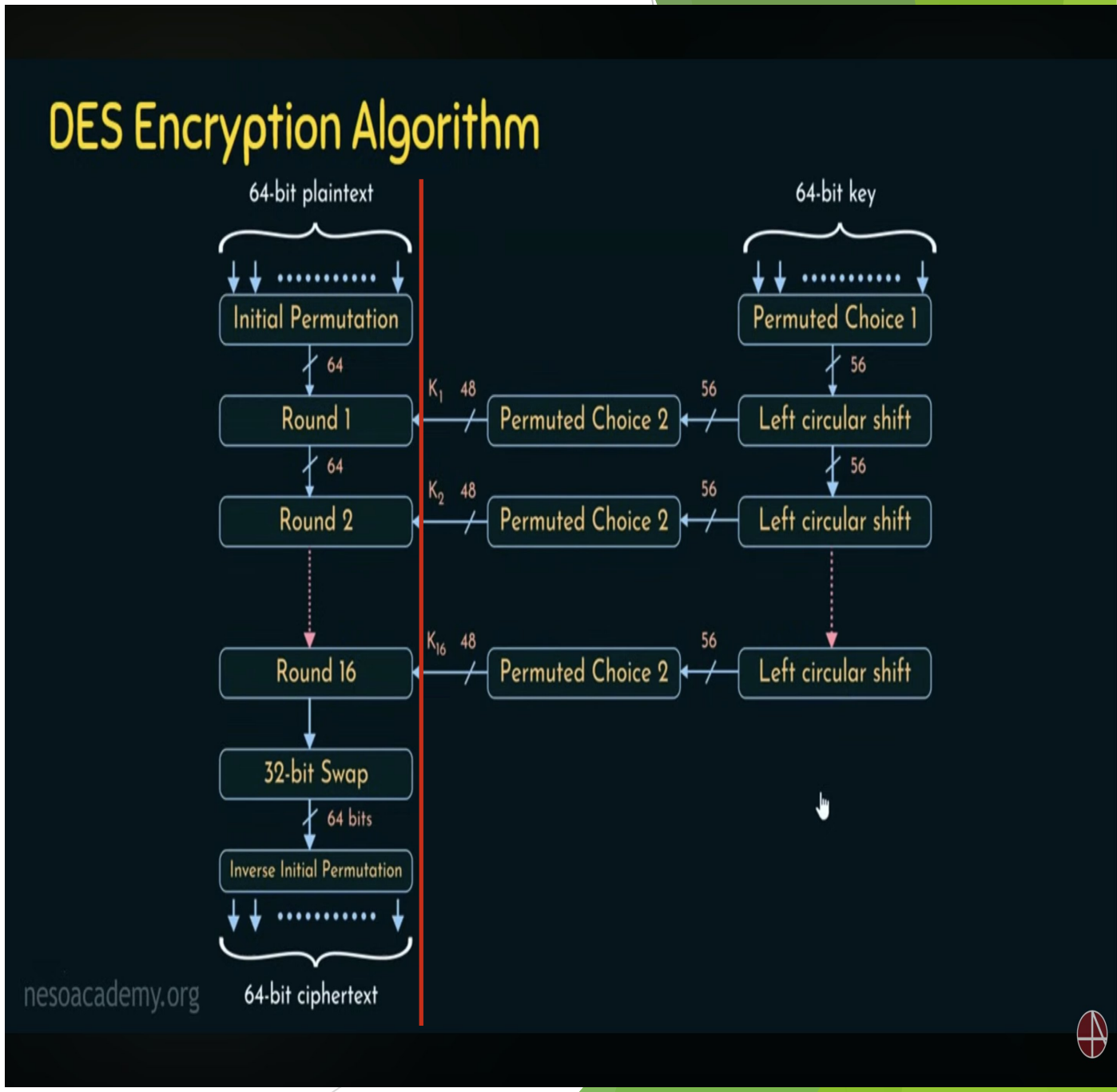
0 1 0 0 1 0 0 0
0 1 0 0 0 1 0 1
0 1 0 0 1 1 0 0
0 1 0 0 1 1 0 0
0 1 0 0 1 1 1 1
0 1 0 1 0 1 1 1
0 1 0 0 1 1 1 1
0 1 0 1 0 0 1 0

First 8 output bits ,

H- 01001000

Output bit	Input bit	Value
1	58	1
2	50	1
3	42	1
4	34	1
5	26	1
6	18	1
7	10	1
8	2	1

- 1 1 1 1 1 1 1 1



Final IP Output (64 bits)

01001000 01000101 01001100 01001100
01001111 01010111 01001111 01010010

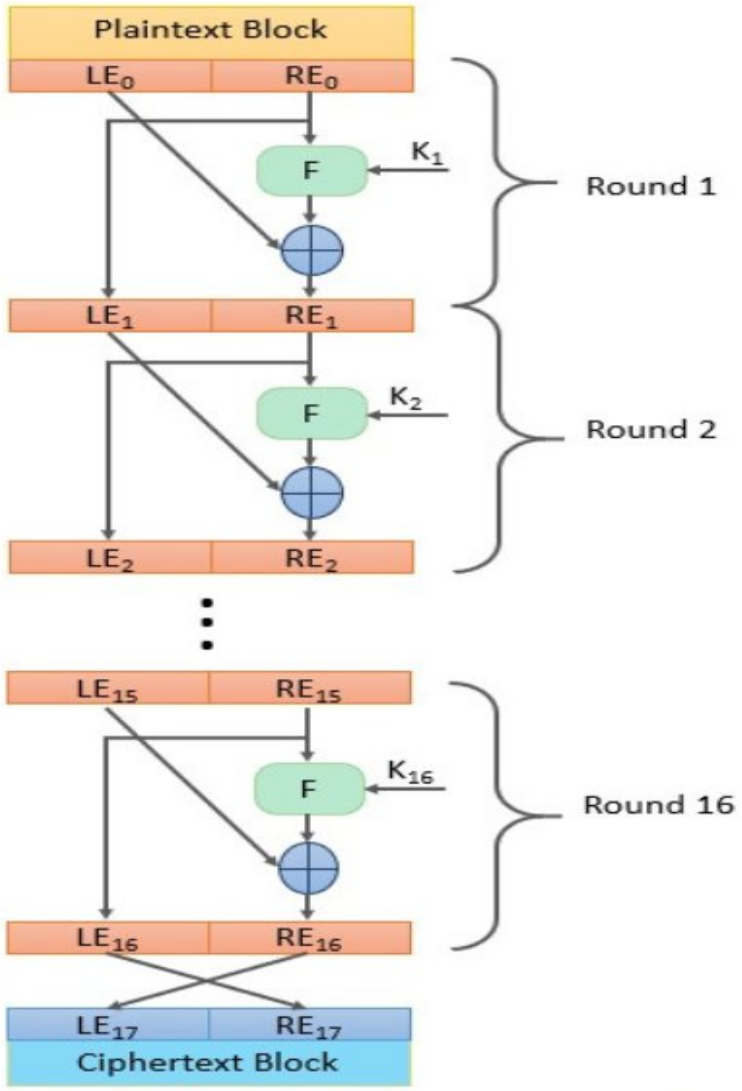
After applying IP to all 64 bits,

11111111 10100000 01111110 01110010
00000000 00000000 01011101 11110000

0 1 0 0 1 0 0 0
0 1 0 0 0 1 0 1
0 1 0 0 1 1 0 0
0 1 0 0 1 1 0 0
0 1 0 0 1 1 1 1
0 1 0 1 0 1 1 1
0 1 0 0 1 1 1 1
0 1 0 1 0 0 1 0

1 1 1 1 1 1 1 1
1 0 1 0 0 0 0 0
0 1 1 1 1 1 1 0
0 1 1 1 0 0 1 0
0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0
0 1 0 1 1 1 0 1
1 1 1 1 0 0 0 0

After Initial Permutation , the 64 bits is divided into 2 halves L and R, 32 bit each



1 1 1 1 1 1 1 1
1 0 1 0 0 0 0 0
0 1 1 1 1 1 1 0
0 1 1 1 0 0 1 0
0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0
0 1 0 1 1 1 0 1
1 1 1 1 0 0 0 0

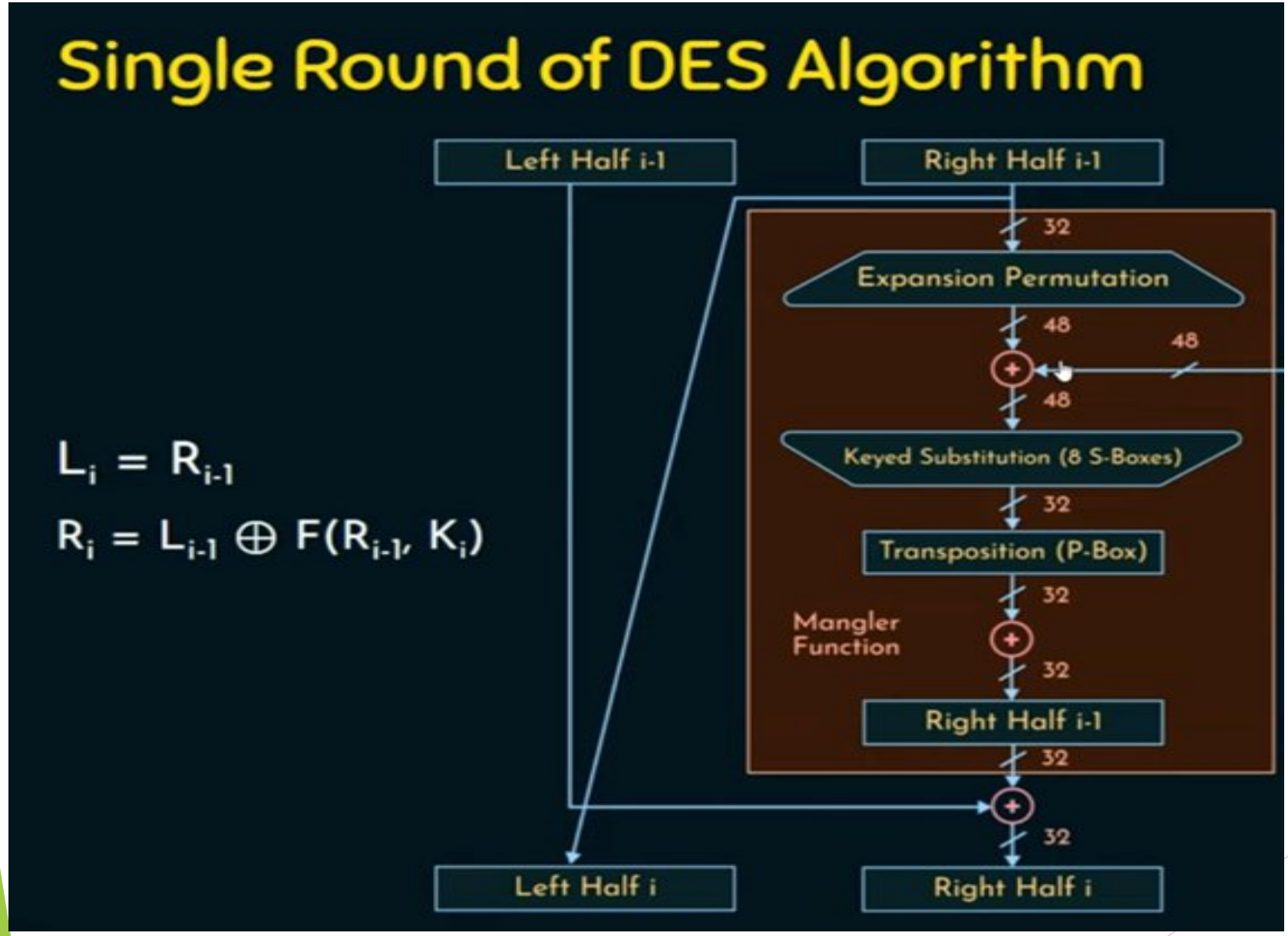
1 1 1 1 1 1 1 1
1 0 1 0 0 0 0 0
0 1 1 1 1 1 1 0
0 1 1 1 0 0 1 0

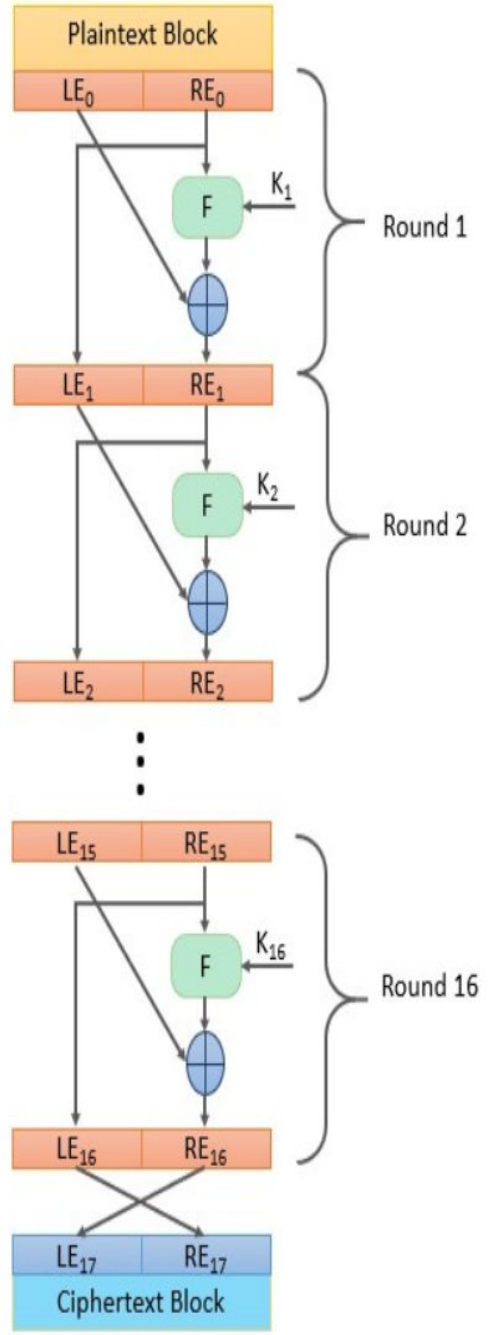
L -32 bit

0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0
0 1 0 1 1 1 0 1
1 1 1 1 0 0 0 0

R -32 bit

What happens in “F”





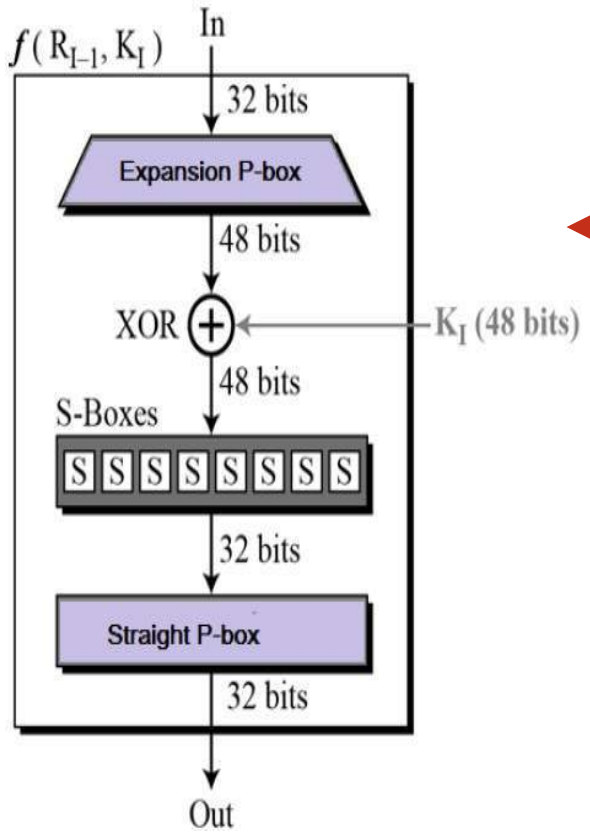
DES Round -1 , (Step-3)

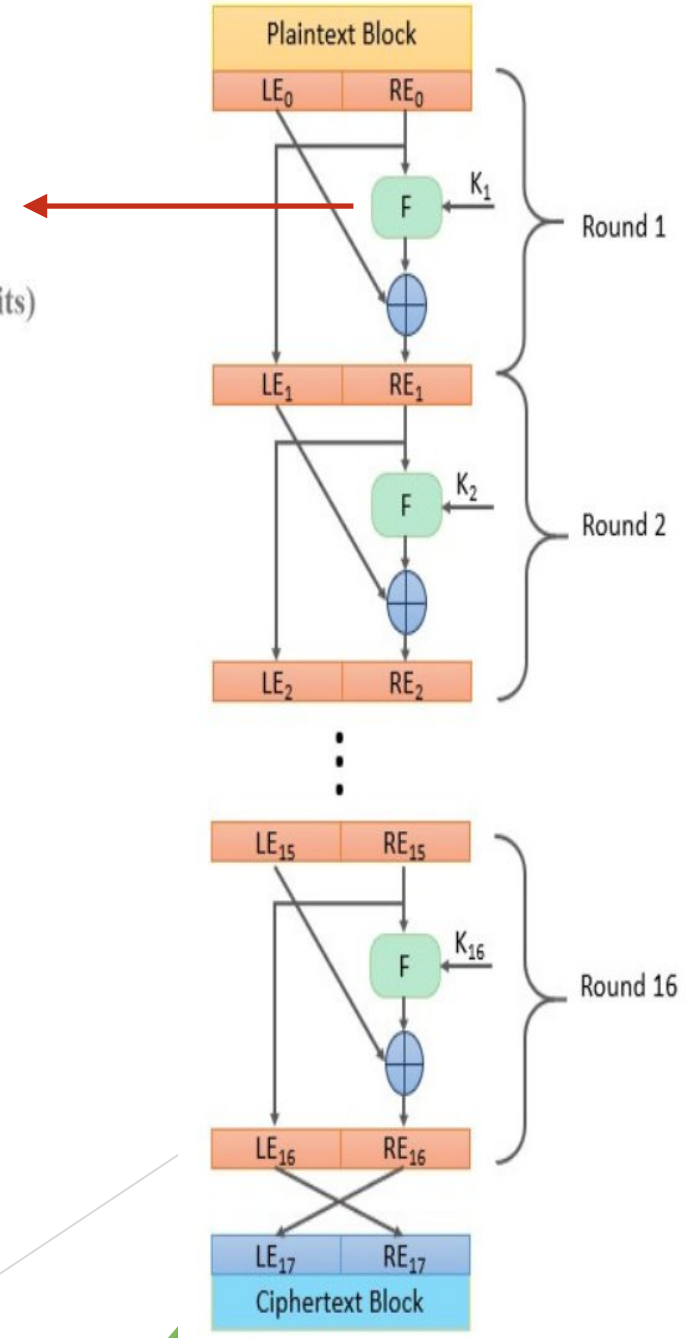
- uses two 32-bit L & R halves
- as for any Feistel cipher can describe as:

  $L_i = R_{i-1}$

  $R_i = L_{i-1} \oplus F(R_{i-1}, K_i)$

- F takes 32-bit R half and 48-bit subkey:
  - expands R to 48-bits using perm E
  - adds to subkey using XOR
  - passes through 8 S-boxes to get 32-bit result
  - finally permutes using 32-bit perm P



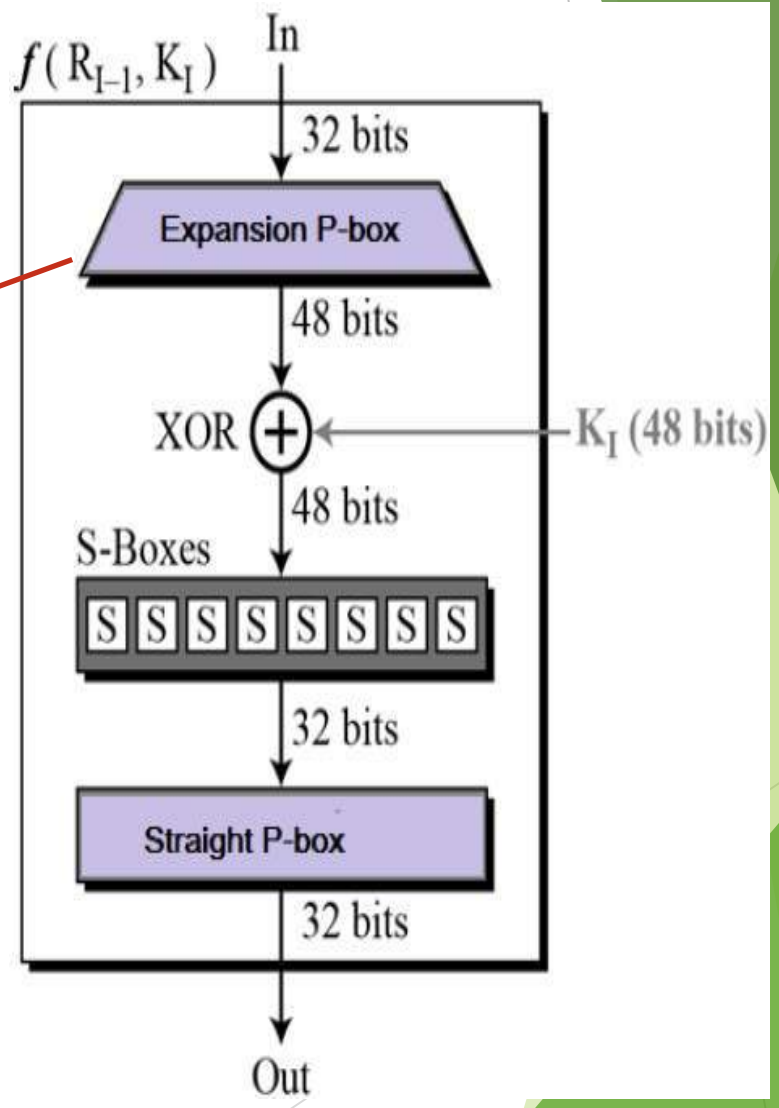


DES Round Structure (Expansion P –box)

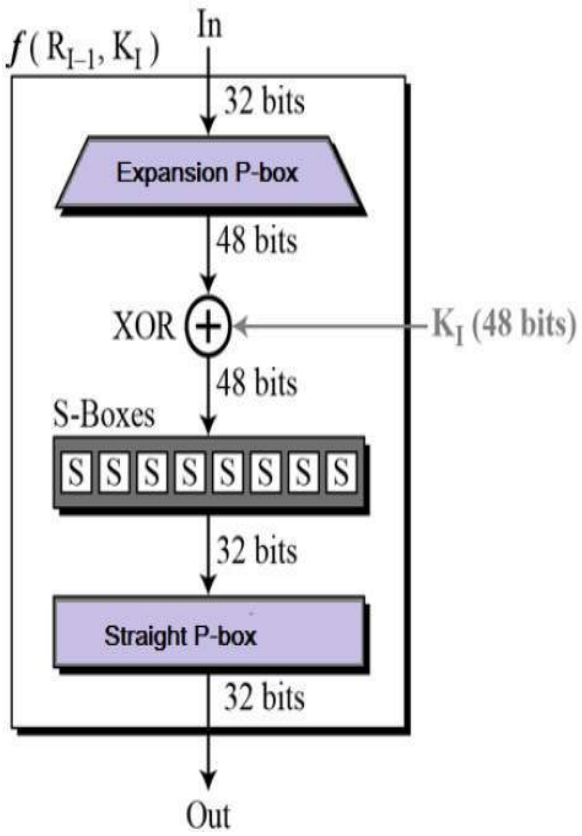
Expansion P-box table

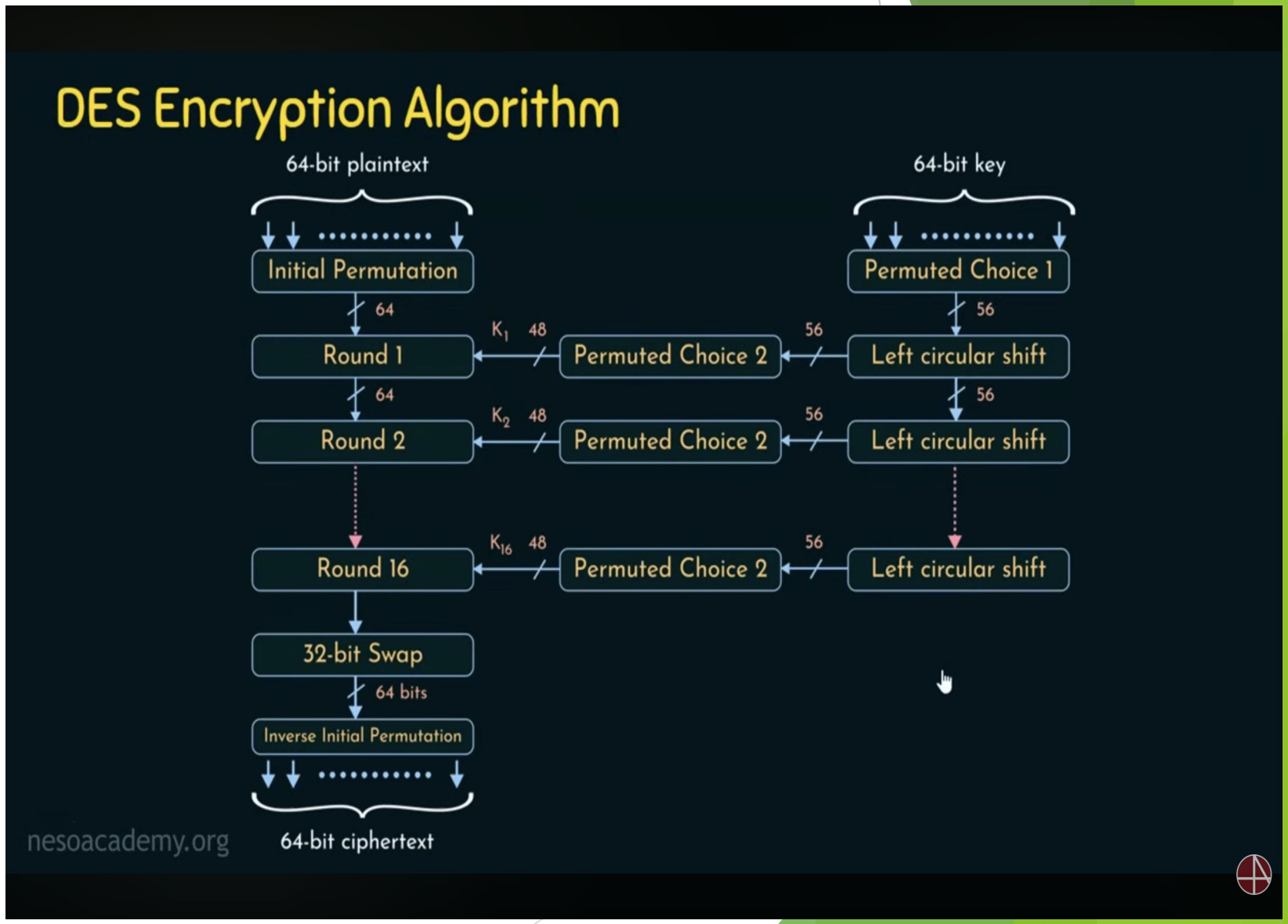
(c) Expansion Permutation (E)

32	1	2	3	4	5
4	5	6	7	8	9
8	9	10	11	12	13
12	13	14	15	16	17
16	17	18	19	20	21
20	21	22	23	24	25
24	25	26	27	28	29
28	29	30	31	32	1

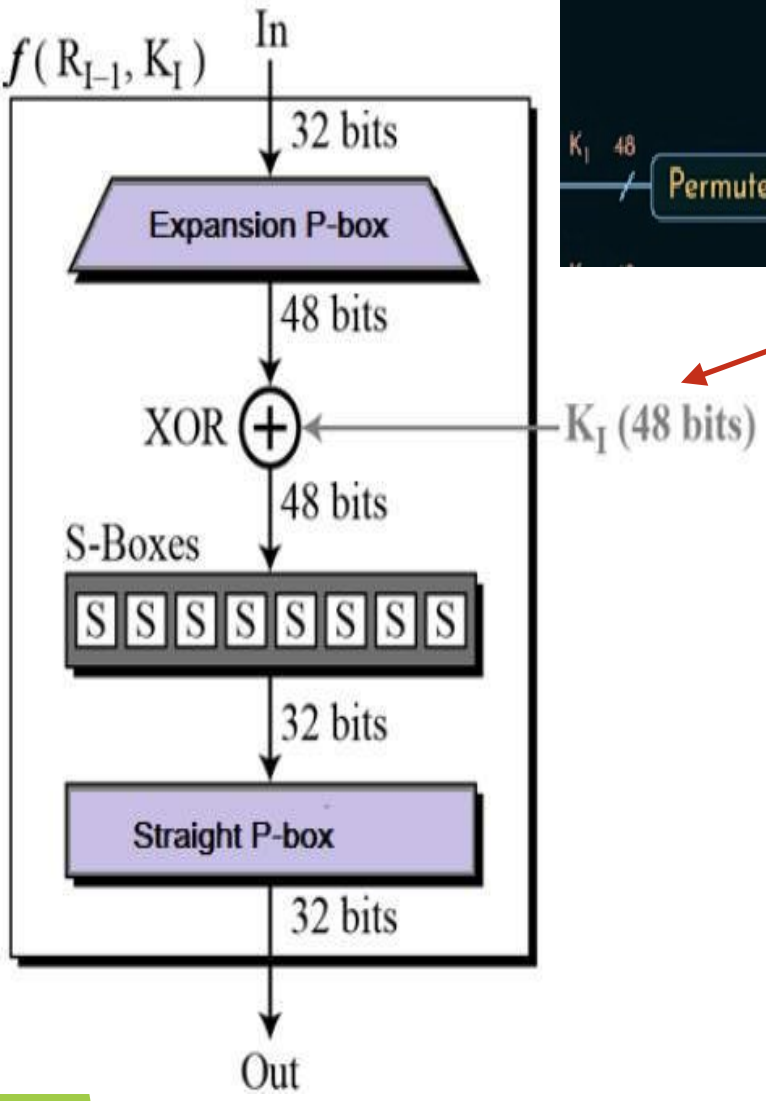


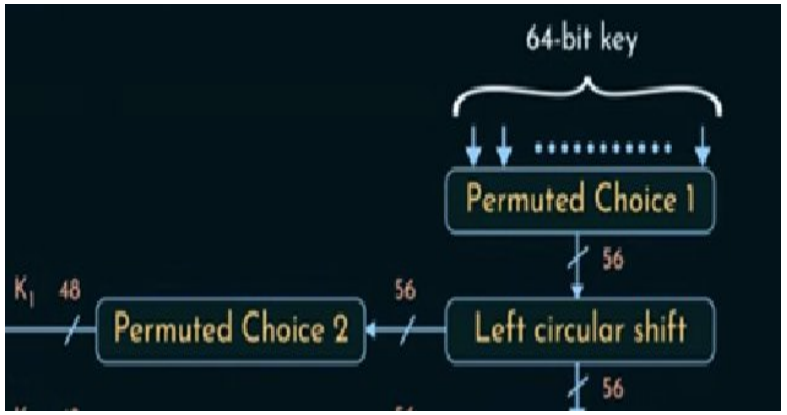
Next to the Expansion P-Box is Key estimation for first round to perform XOR operation





DES Round Structure (XOR)





(a) Input Key

1	2	3	4	5	6	7	8
9	10	11	12	13	14	15	16
17	18	19	20	21	22	23	24
25	26	27	28	29	30	31	32
33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48
49	50	51	52	53	54	55	56
57	58	59	60	61	62	63	64

(b) Permuted Choice One (PC-1)

57	49	41	33	25	17	9
1	58	50	42	34	26	18
10	2	59	51	43	35	27
19	11	3	60	52	44	36
63	55	47	39	31	23	15
7	62	54	46	38	30	22
14	6	61	53	45	37	29
21	13	5	28	20	12	4

(c) Permuted Choice Two (PC-2)

14	17	11	24	1	5	3	28
15	6	21	10	23	19	12	4
26	8	16	7	27	20	13	2
41	52	31	37	47	55	30	40
51	45	33	48	44	49	39	56
34	53	46	42	50	36	29	32

(d) Schedule of Left Shifts

Round Number	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Bits Rotated	1	1	2	2	2	2	2	2	1	2	2	2	2	2	2	1

Key Process : 64 bit to 56 bit using (PC1), Left Shift)

(a) Input Key

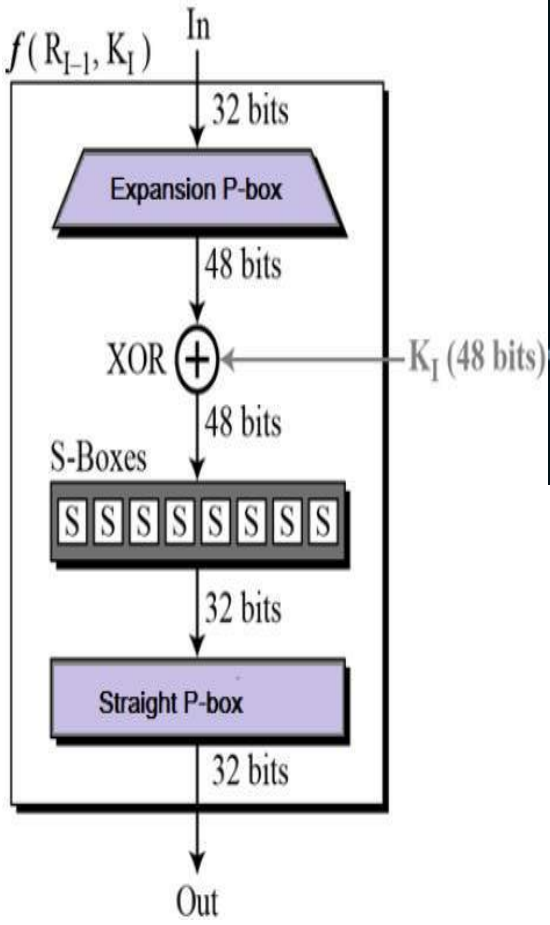
1	2	3	4	5	6	7	8
9	10	11	12	13	14	15	16
17	18	19	20	21	22	23	24
25	26	27	28	29	30	31	32
33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48
49	50	51	52	53	54	55	56
57	58	59	60	61	62	63	64

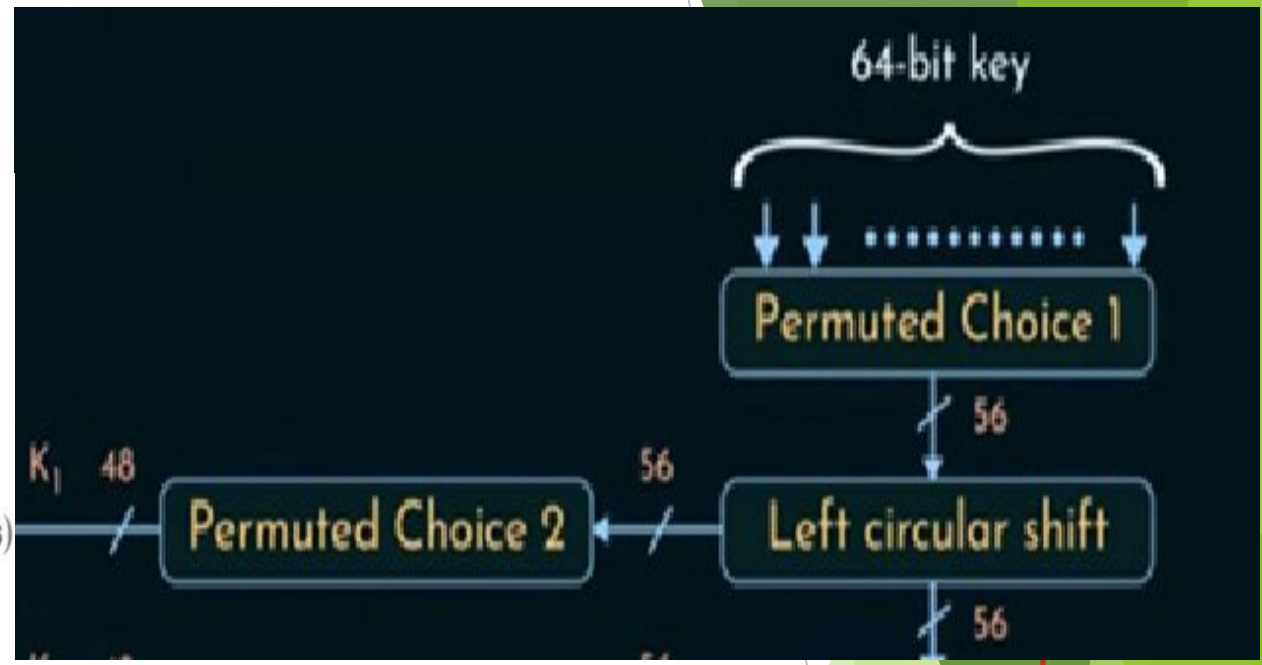
(b) Permuted Choice One (PC-1)

57	49	41	33	25	17	9
1	58	50	42	34	26	18
10	2	59	51	43	35	27
19	11	3	60	52	44	36
63	55	47	39	31	23	15
7	62	54	46	38	30	22
14	6	61	53	45	37	29
21	13	5	28	20	12	4

(c) Permuted Choice Two (PC-2)

14	17	11	24	1	5	3	28
15	6	21	10	23	19	12	4
26	8	16	7	27	20	13	2
41	52	31	37	47	55	30	40
51	45	33	48	44	49	39	56
34	53	46	42	50	36	29	32





Key Process : 56 bit to 48 using PC-2

(a) Input Key

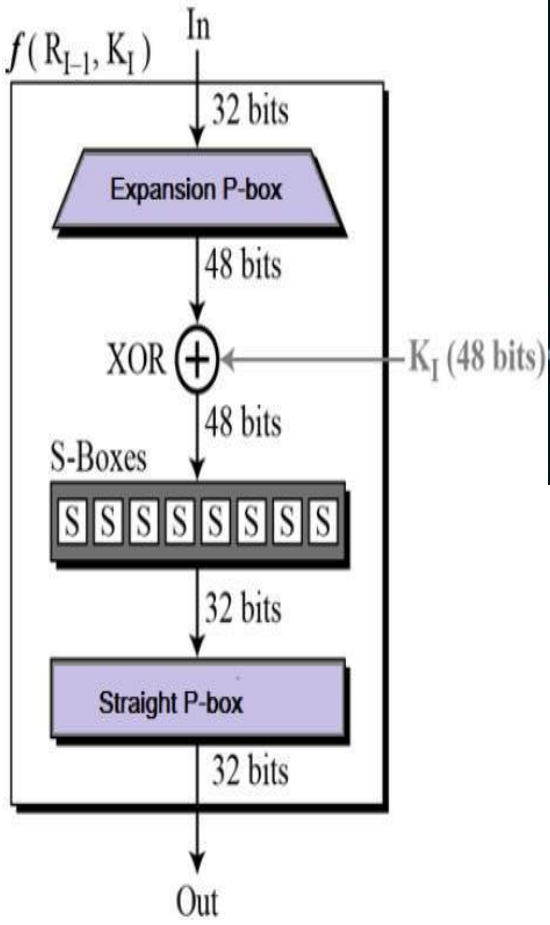
1	2	3	4	5	6	7	8
9	10	11	12	13	14	15	16
17	18	19	20	21	22	23	24
25	26	27	28	29	30	31	32
33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48
49	50	51	52	53	54	55	56
57	58	59	60	61	62	63	64

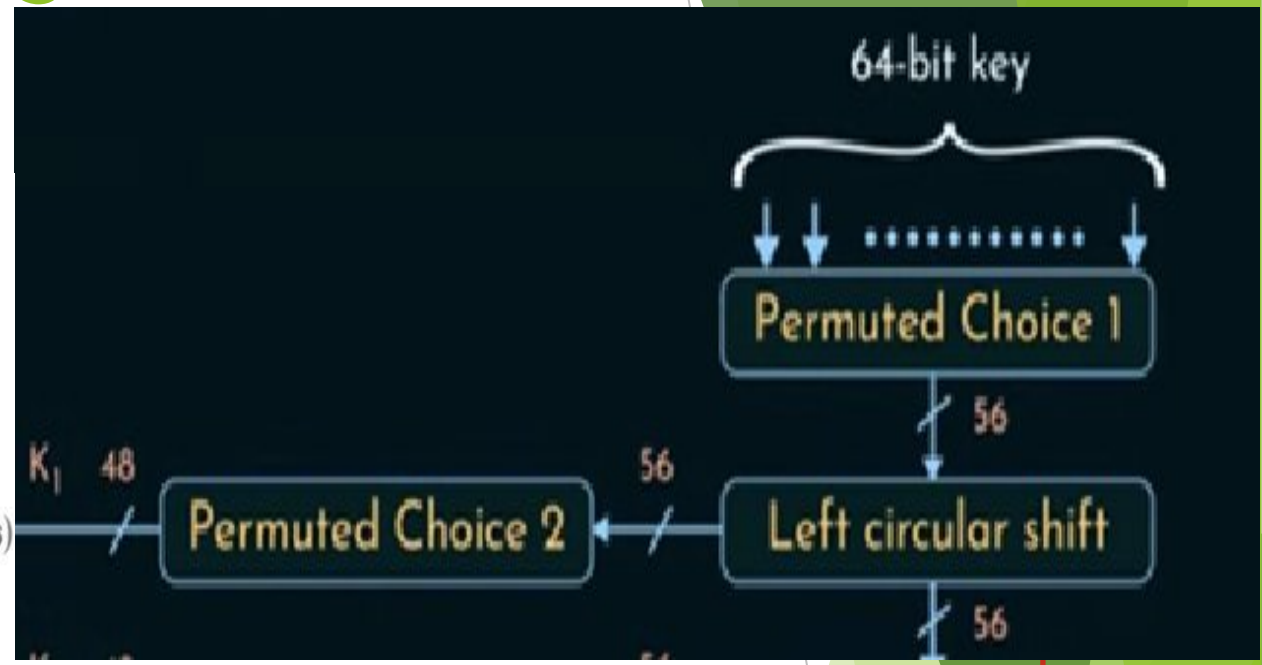
(b) Permuted Choice One (PC-1)

57	49	41	33	25	17	9
1	58	50	42	34	26	18
10	2	59	51	43	35	27
19	11	3	60	52	44	36
63	55	47	39	31	23	15
7	62	54	46	38	30	22
14	6	61	53	45	37	29
21	13	5	28	20	12	4

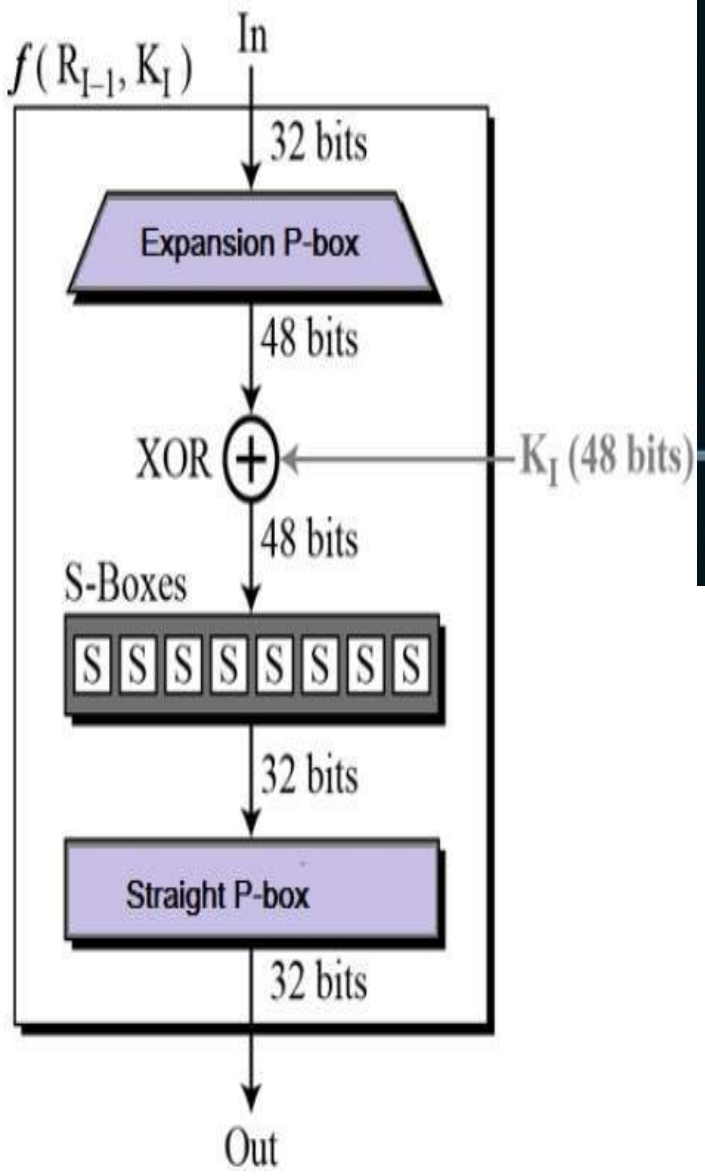
(c) Permuted Choice Two (PC-2)

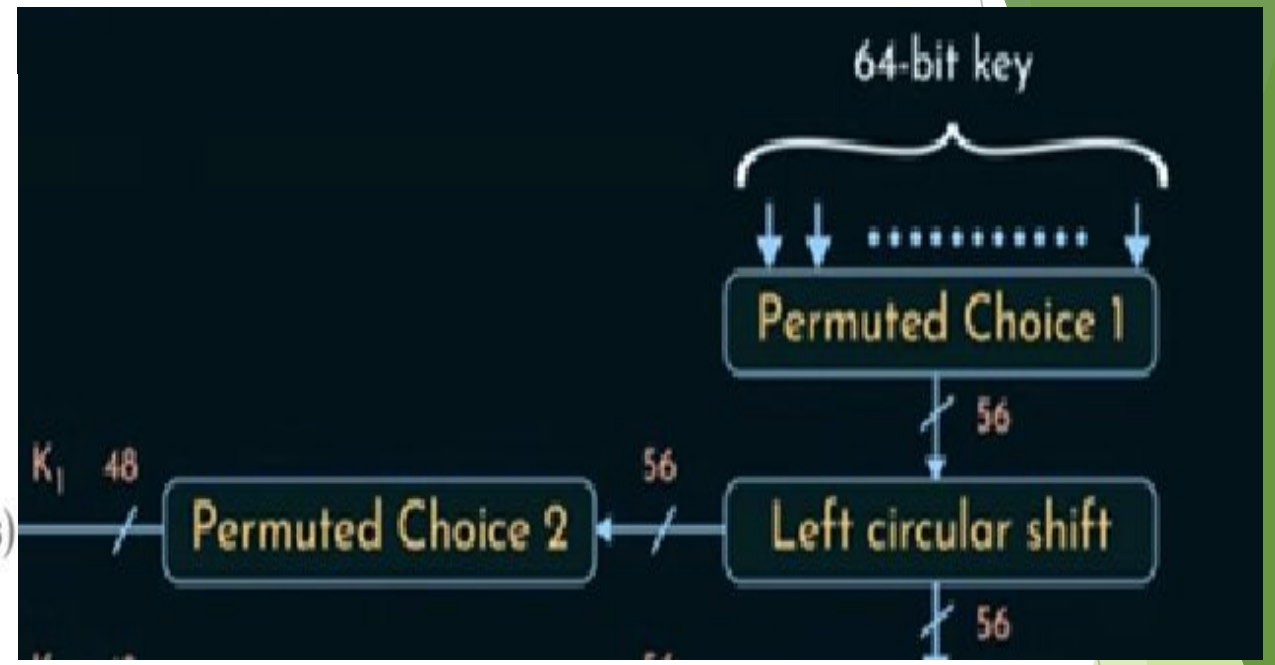
14	17	11	24	1	5	3	28
15	6	21	10	23	19	12	4
26	8	16	7	27	20	13	2
41	52	31	37	47	55	30	40
51	45	33	48	44	49	39	56
34	53	46	42	50	36	29	32



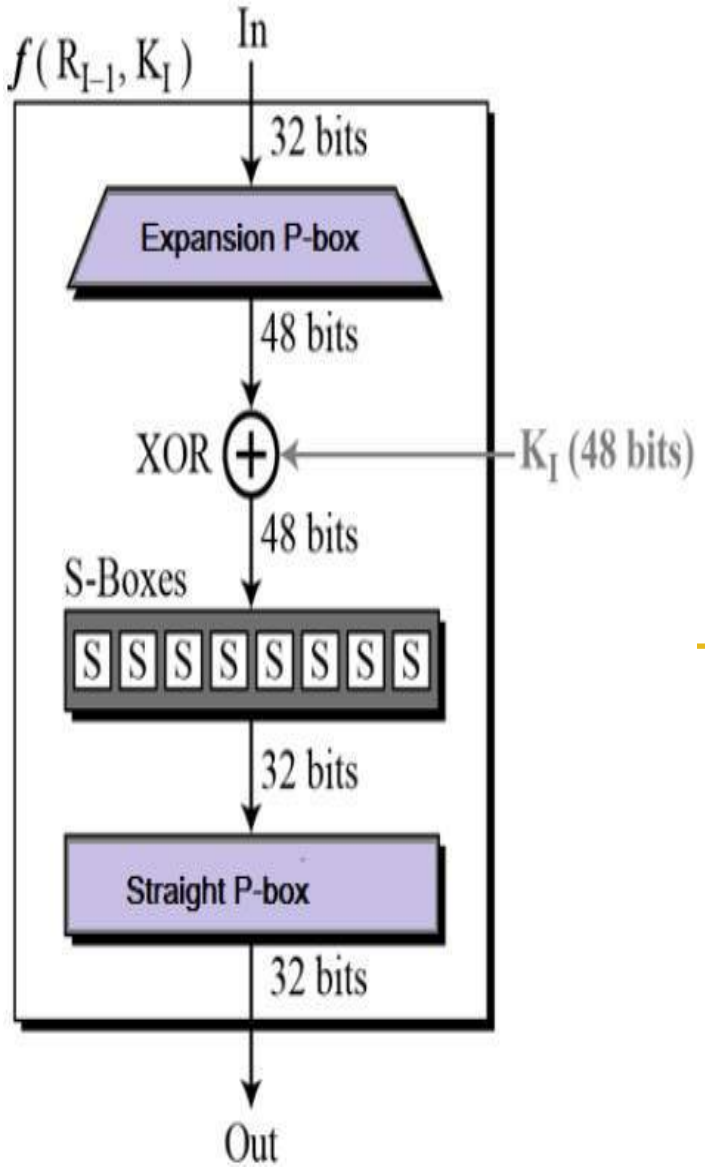


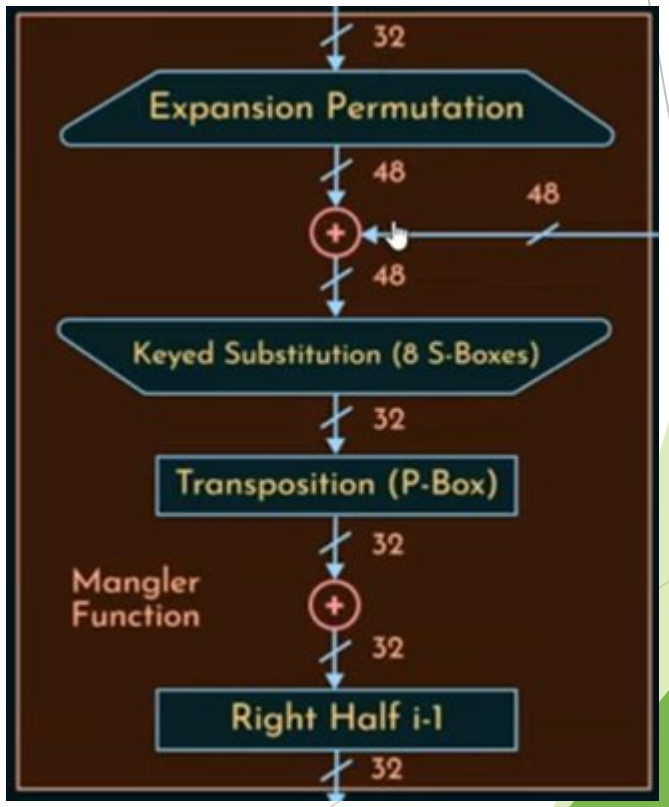
XOR happens and gives 48 bit as output





S-Box (Converting 48 bits to 32 bits)





S_1

14	4	13	1	2	15	11	8	3	10	6	12	5	9	0	7
0	15	7	4	14	2	13	1	10	6	12	11	9	5	3	8
4	1	14	8	13	6	2	11	15	12	9	7	3	10	5	0
15	12	8	2	4	9	1	7	5	11	3	14	10	0	6	13

S_2

15	1	8	14	6	11	3	4	9	7	2	13	12	0	5	10
3	13	4	7	15	2	8	14	12	0	1	10	6	9	11	5
0	14	7	11	10	4	13	1	5	8	12	6	9	3	2	15
13	8	10	1	3	15	4	2	11	6	7	12	0	5	14	9

S_3

10	0	9	14	6	3	15	5	1	13	12	7	11	4	2	8
13	7	0	9	3	4	6	10	2	8	5	14	12	11	15	1
13	6	4	9	8	15	3	0	11	1	2	12	5	10	14	7
1	10	13	0	6	9	8	7	4	15	14	3	11	5	2	12

S_4

7	13	14	3	0	6	9	10	1	2	8	5	11	12	4	15
13	8	11	5	6	15	0	3	4	7	2	12	1	10	14	9
10	6	9	0	12	11	7	13	15	1	3	14	5	2	8	4
3	15	0	6	10	1	13	8	9	4	5	11	12	7	2	14

S_5

2	12	4	1	7	10	11	6	8	5	3	15	13	0	14	9
14	11	2	12	4	7	13	1	5	0	15	10	3	9	8	6
4	2	1	11	10	13	7	8	15	9	12	5	6	3	0	14
11	8	12	7	1	14	2	13	6	15	0	9	10	4	5	3

S_6

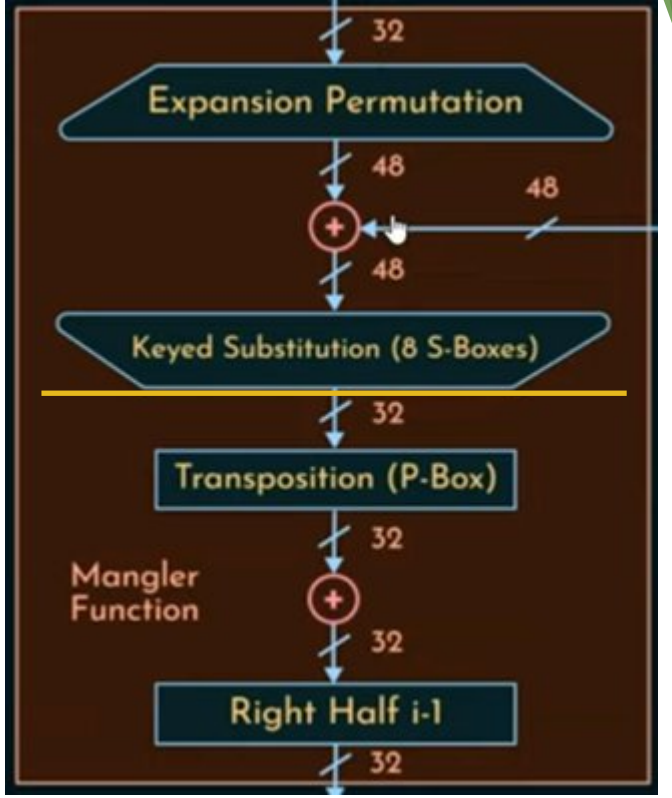
12	1	10	15	9	2	6	8	0	13	3	4	14	7	5	11
10	15	4	2	7	12	9	5	6	1	13	14	0	11	3	8
9	14	15	5	2	8	12	3	7	0	4	10	1	13	11	6
4	3	2	12	9	5	15	10	11	14	1	7	6	0	8	13

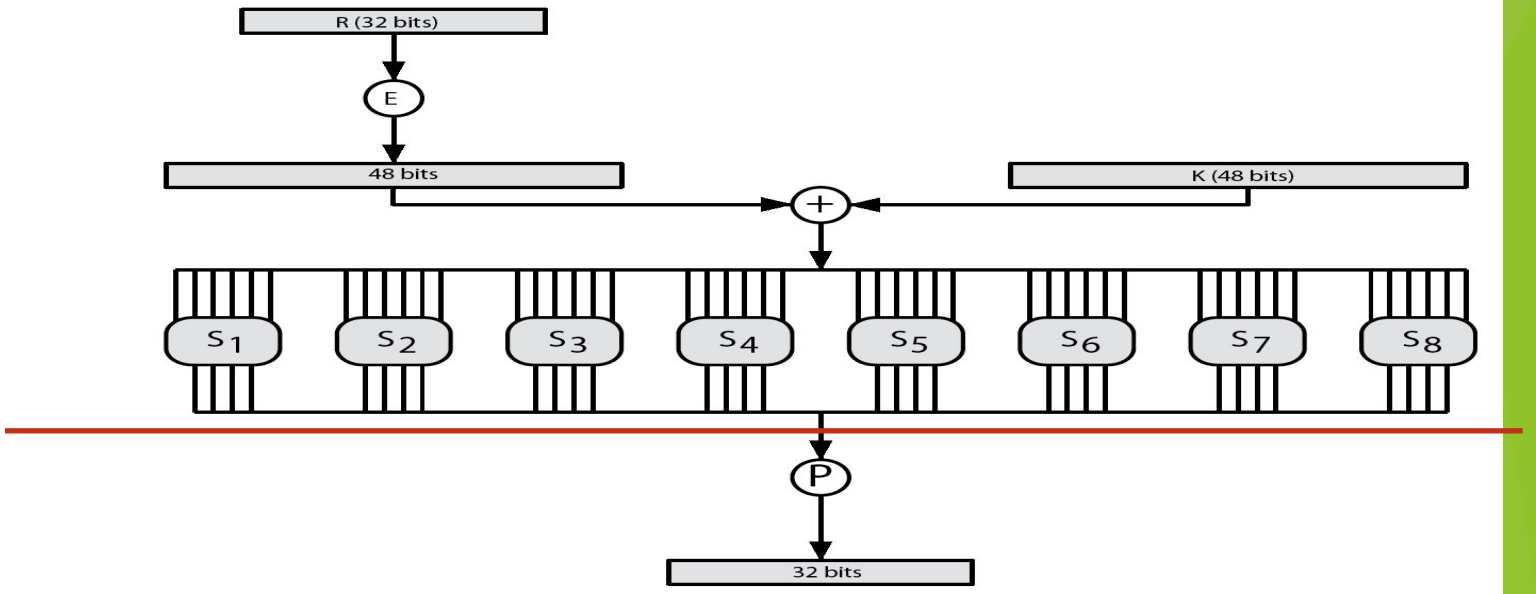
S_7

4	11	2	14	15	0	8	13	3	12	9	7	5	10	6	1
13	0	11	7	4	9	1	10	14	3	5	12	2	15	8	6
1	4	11	13	12	3	7	14	10	15	6	8	0	5	9	2
6	11	13	8	1	4	10	7	9	5	0	15	14	2	3	12

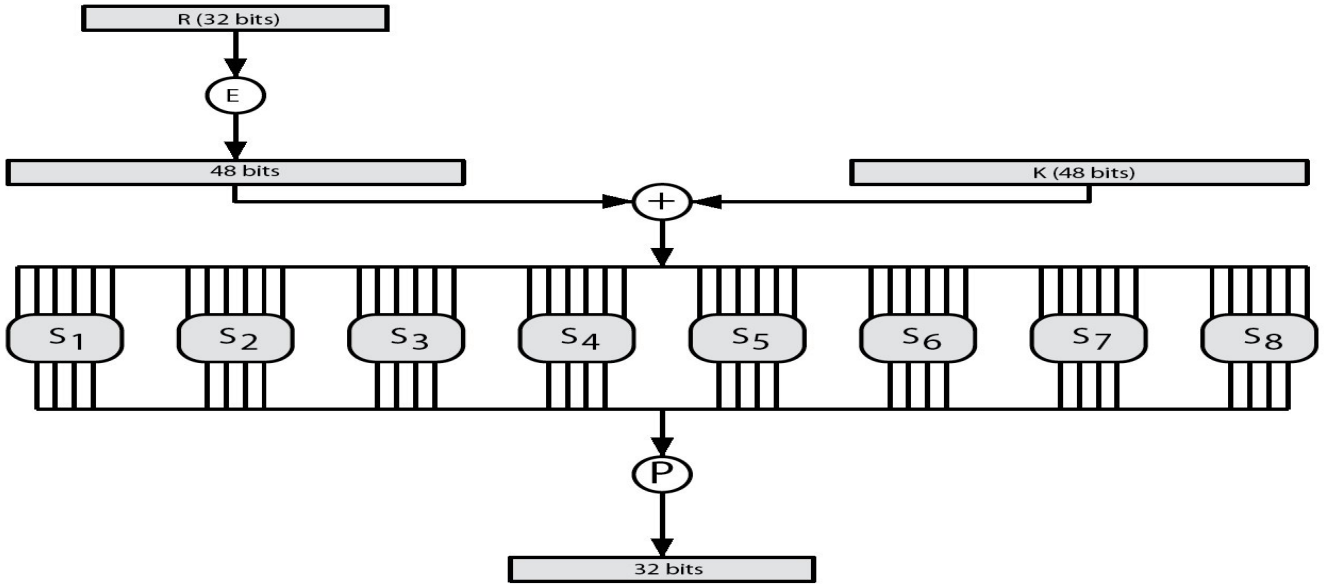
S_8

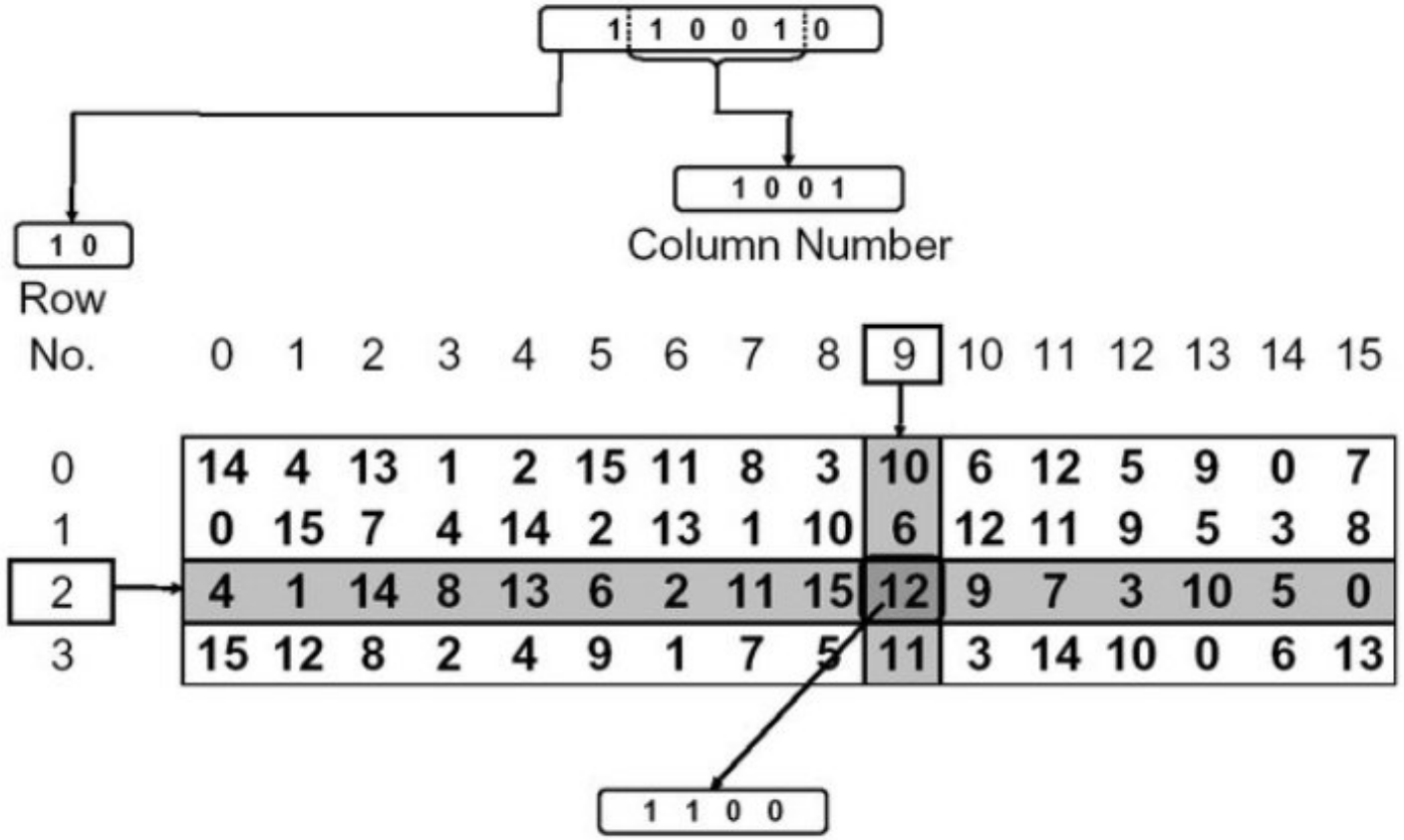
13	2	8	4	6	15	11	1	10	9	3	14	5	0	12	7
1	15	13	8	10	3	7	4	12	5	6	11	0	14	9	2
7	11	4	1	9	12	14	2	0	6	10	13	15	3	5	8
2	1	14	7	4	10	8	13	15	12	9	0	3	5	6	11





14	4	13	1	2	15	11	8	3	10	6	12	5	9	0	7
0	15	7	4	14	2	13	1	10	6	12	11	9	5	3	8
4	1	14	8	13	6	2	11	15	12	9	7	3	10	5	0
15	12	8	2	4	9	1	7	5	11	3	14	10	0	6	13

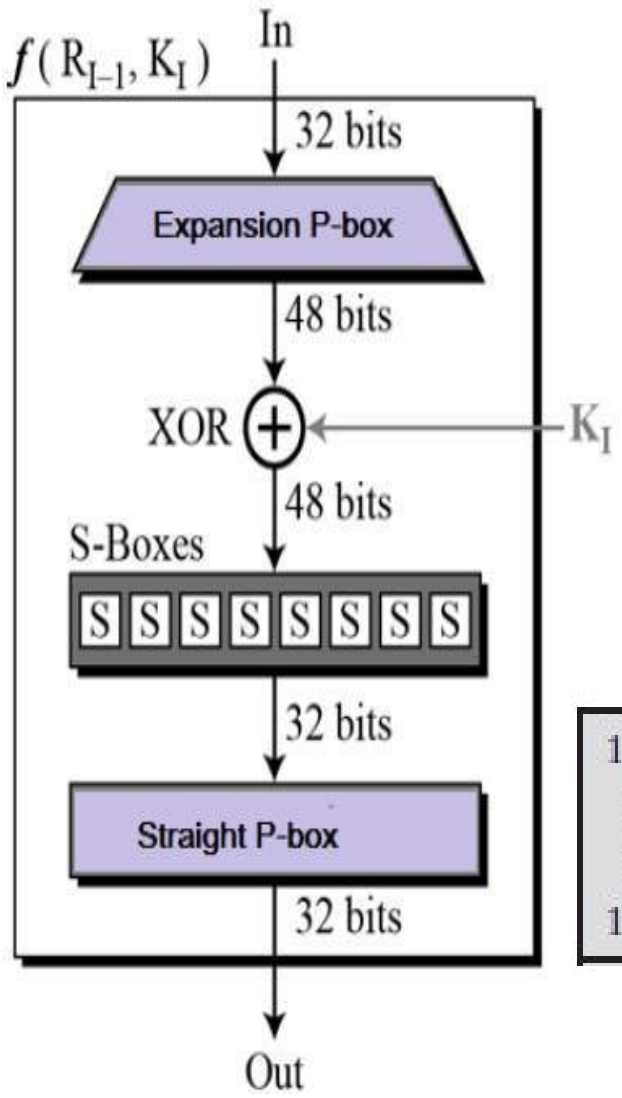


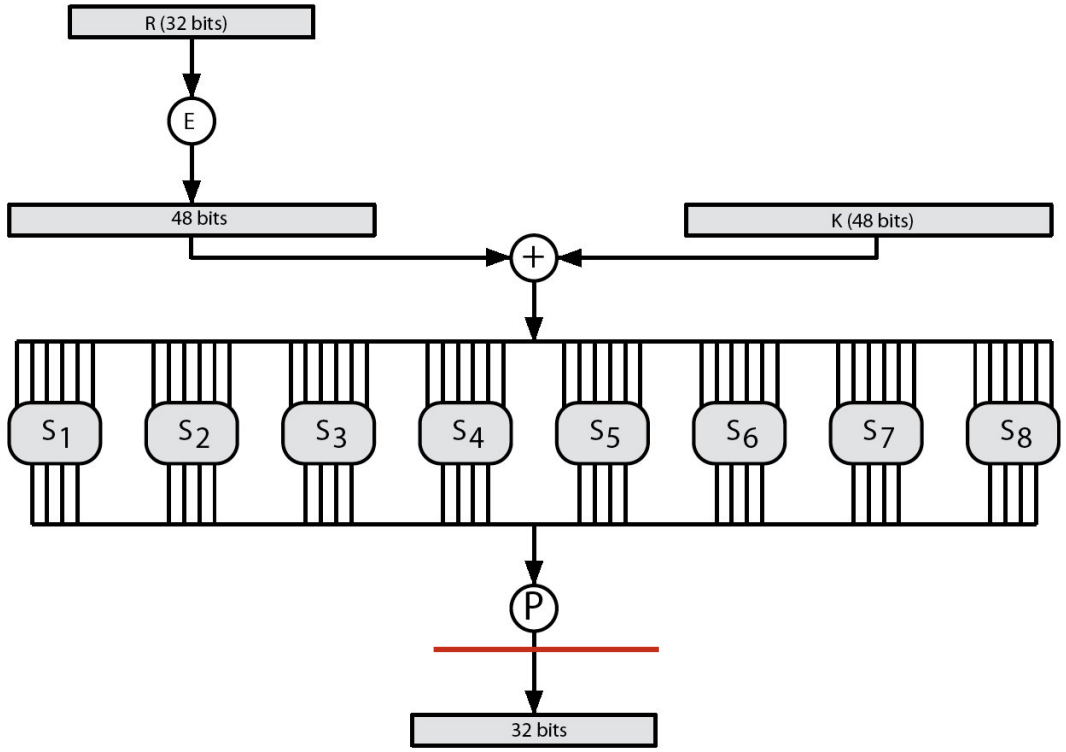


Application of S-box in DES Algorithm

Straight P-Box Table

16	7	20	21
29	12	28	17
1	15	23	26
5	18	31	10
2	8	24	14
32	27	3	9
19	13	30	6
22	11	4	25

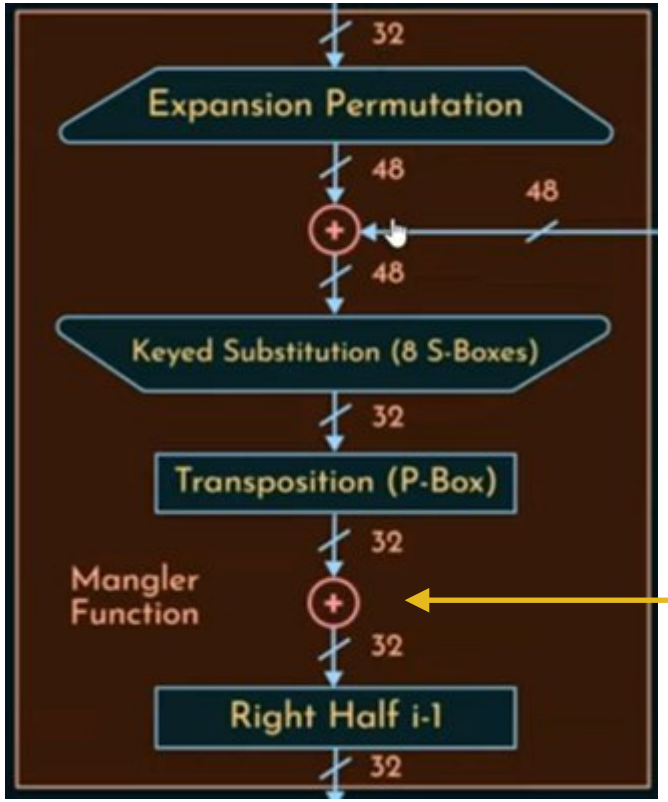


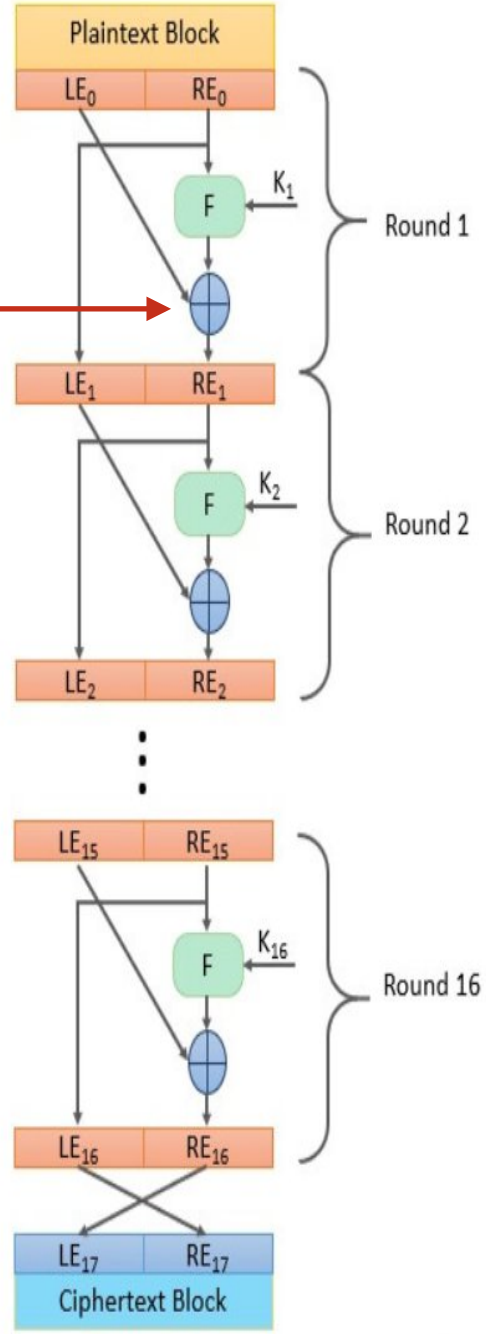


(d) Permutation Function (P)

16	7	20	21	29	12	28	17
1	15	23	26	5	18	31	10
2	8	24	14	32	27	3	9
19	13	30	6	22	11	4	25

Function “F” is completed and the output of the Function (32 bit) will undergo XOR operation with Left (32 bit) to complete Round-1





BOX used in DES (All elements represents positions of the bits)

(a) Initial Permutation (IP)

58	50	42	34	26	18	10	2
60	52	44	36	28	20	12	4
62	54	46	38	30	22	14	6
64	56	48	40	32	24	16	8
57	49	41	33	25	17	9	1
59	51	43	35	27	19	11	3
61	53	45	37	29	21	13	5
63	55	47	39	31	23	15	7

(b) Inverse Initial Permutation (IP^{-1})

40	8	48	16	56	24	64	32
39	7	47	15	55	23	63	31
38	6	46	14	54	22	62	30
37	5	45	13	53	21	61	29
36	4	44	12	52	20	60	28
35	3	43	11	51	19	59	27
34	2	42	10	50	18	58	26
33	1	41	9	49	17	57	25

(c) Expansion Permutation (E)

32	1	2	3	4	5
4	5	6	7	8	9
8	9	10	11	12	13
12	13	14	15	16	17
16	17	18	19	20	21
20	21	22	23	24	25
24	25	26	27	28	29
28	29	30	31	32	1

(d) Permutation Function (P)

16	7	20	21	29	12	28	17
1	15	23	26	5	18	31	10
2	8	24	14	32	27	3	9
19	13	30	6	22	11	4	25

Tables used in the DES algorithm

The DES S-Boxes

S_1	14	4	13	1	2	15	11	8	3	10	6	12	5	9	0	7
	0	15	7	4	14	2	13	1	10	6	12	11	9	5	3	8
	4	1	14	8	13	6	2	11	15	12	9	7	3	10	5	0
	15	12	8	2	4	9	1	7	5	11	3	14	10	0	6	13
S_2	15	1	8	14	6	11	3	4	9	7	2	13	12	0	5	10
	3	13	4	7	15	2	8	14	12	0	1	10	6	9	11	5
	0	14	7	11	10	4	13	1	5	8	12	6	9	3	2	15
	13	8	10	1	3	15	4	2	11	6	7	12	0	5	14	9
S_3	10	0	9	14	6	3	15	5	1	13	12	7	11	4	2	8
	13	7	0	9	3	4	6	10	2	8	5	14	12	11	15	1
	13	6	4	9	8	15	3	0	11	1	2	12	5	10	14	7
	1	10	13	0	6	9	8	7	4	15	14	3	11	5	2	12
S_4	7	13	14	3	0	6	9	10	1	2	8	5	11	12	4	15
	13	8	11	5	6	15	0	3	4	7	2	12	1	10	14	9
	10	6	9	0	12	11	7	13	15	1	3	14	5	2	8	4
	3	15	0	6	10	1	13	8	9	4	5	11	12	7	2	14
S_5	2	12	4	1	7	10	11	6	8	5	3	15	13	0	14	9
	14	11	2	12	4	7	13	1	5	0	15	10	3	9	8	6
	4	2	1	11	10	13	7	8	15	9	12	5	6	3	0	14
	11	8	12	7	1	14	2	13	6	15	0	9	10	4	5	3
S_6	12	1	10	15	9	2	6	8	0	13	3	4	14	7	5	11
	10	15	4	2	7	12	9	5	6	1	13	14	0	11	3	8
	9	14	15	5	2	8	12	3	7	0	4	10	1	13	11	6
	4	3	2	12	9	5	15	10	11	14	1	7	6	0	8	13
S_7	4	11	2	14	15	0	8	13	3	12	9	7	5	10	6	1
	13	0	11	7	4	9	1	10	14	3	5	12	2	15	8	6
	1	4	11	13	12	3	7	14	10	15	6	8	0	5	9	2
	6	11	13	8	1	4	10	7	9	5	0	15	14	2	3	12
S_8	13	2	8	4	6	15	11	1	10	9	3	14	5	0	12	7
	1	15	13	8	10	3	7	4	12	5	6	11	0	14	9	2
	7	11	4	1	9	12	14	2	0	6	10	13	15	3	5	8
	2	1	14	7	4	10	8	13	15	12	9	0	3	5	6	11

(a) Input Key

1	2	3	4	5	6	7	8
9	10	11	12	13	14	15	16
17	18	19	20	21	22	23	24
25	26	27	28	29	30	31	32
33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48
49	50	51	52	53	54	55	56
57	58	59	60	61	62	63	64

(b) Permuted Choice One (PC-1)

57	49	41	33	25	17	9
1	58	50	42	34	26	18
10	2	59	51	43	35	27
19	11	3	60	52	44	36
63	55	47	39	31	23	15
7	62	54	46	38	30	22
14	6	61	53	45	37	29
21	13	5	28	20	12	4

(c) Permuted Choice Two (PC-2)

14	17	11	24	1	5	3	28
15	6	21	10	23	19	12	4
26	8	16	7	27	20	13	2
41	52	31	37	47	55	30	40
51	45	33	48	44	49	39	56
34	53	46	42	50	36	29	32

(d) Schedule of Left Shifts

Round Number	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Bits Rotated	1	1	2	2	2	2	2	2	1	2	2	2	2	2	2	1

Tables used in DES key generator

Confusion vs diffusion

- S-Box → CONFUSION
- P-Boxes (Expansion & Straight) → DIFFUSION

Avalanche Effect

- A small change in input (1 bit) causes a large, unpredictable change in output.
- The avalanche effect occurs when a small change in either the plaintext (e.g., flipping a single bit) or the encryption key results in a substantial and unpredictable change in the ciphertext (typically, about 50% of the ciphertext bits should change).
- key desirable property of encryption alg
- where a change of **one** input or key bit results in changing approximately **half** output bits
- making attempts to "home-in" by guessing keys impossible
- DES exhibits strong avalanche

Strength of DES is Key Size

- 56-bit keys have $2^{56} = 7.2 \times 10^{16}$ values
- brute force search looks hard

- But,

- recent advances have shown is possible
  - on Internet in a few months
  - on dedicated h/w (EFF) in a few days
  - combined in 22hrs!
- still must be able to recognize plaintext
- must now consider alternatives to DES

Strength of DES – Analytic Attacks

- now have several analytic attacks on DES
- these utilise some deep structure of the cipher
  - by gathering information about encryptions
  - can eventually recover some/all of the sub-key bits
  - if necessary then exhaustively search for the rest
- generally these are statistical attacks, include
  - differential cryptanalysis
  - linear cryptanalysis
  - related key attacks

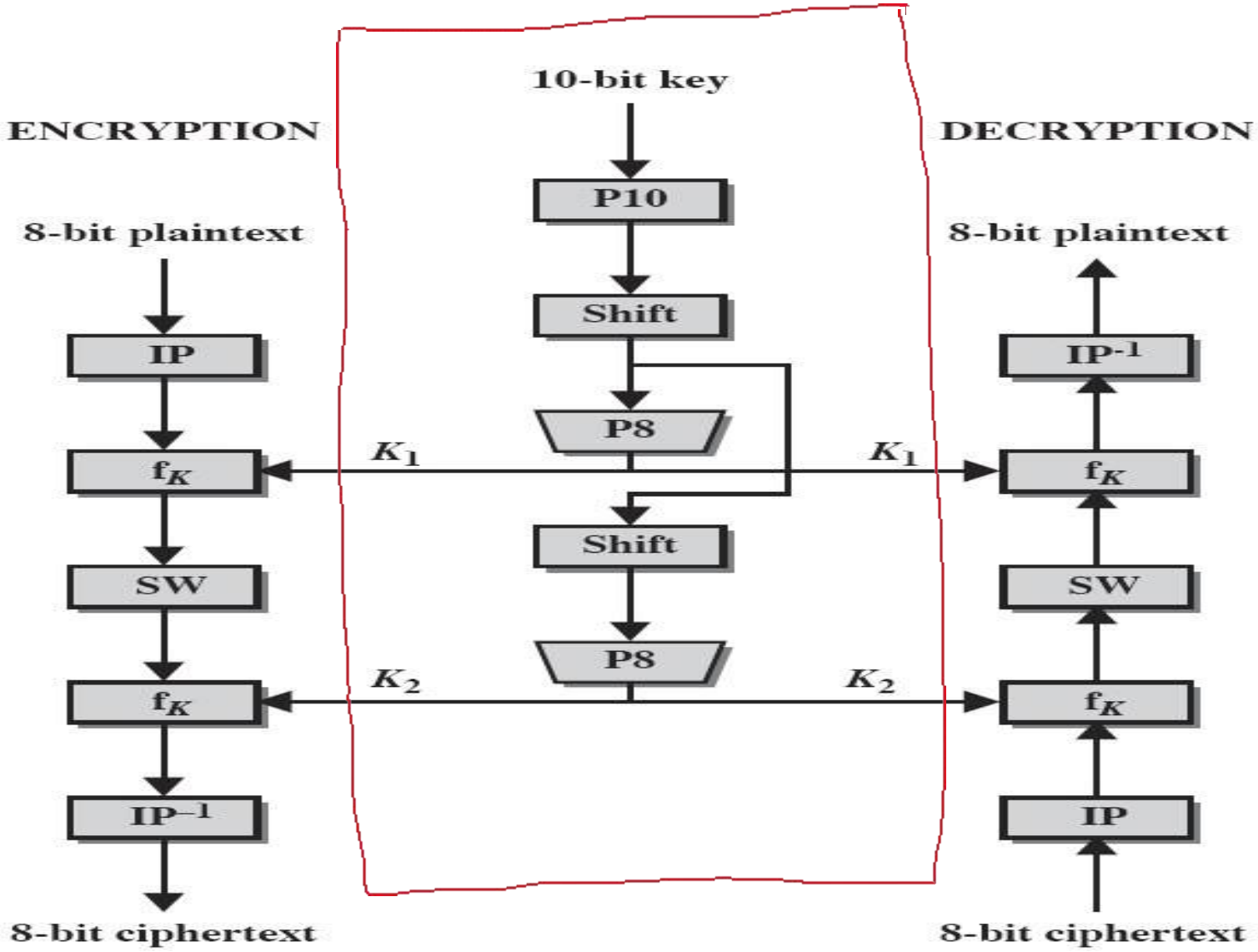
Strength of DES – Timing Attacks

- attacks actual implementation of cipher
- use knowledge of consequences of implementation to derive information about some/all subkey bits
- specifically use fact that calculations can take varying times depending on the value of the inputs to it
- particularly problematic on smartcards

S-DES

- Simplified Data Encryption Standard
- simple version of Data Encryption Standard having a 10-bit key and 8-bit plain text.
- It was developed for educational purpose so that understanding DES can become easy.
- It is a block cipher algorithm and uses a symmetric key for its algorithm i.e. they use the same key for both encryption and decryption.
- It has 2 rounds for encryption which use two different keys.
- First, we need to generate 2 keys before encryption.
- After generating keys, we pass them to each individual round for s-des encryption.

S-DES algorithm (2 rounds)

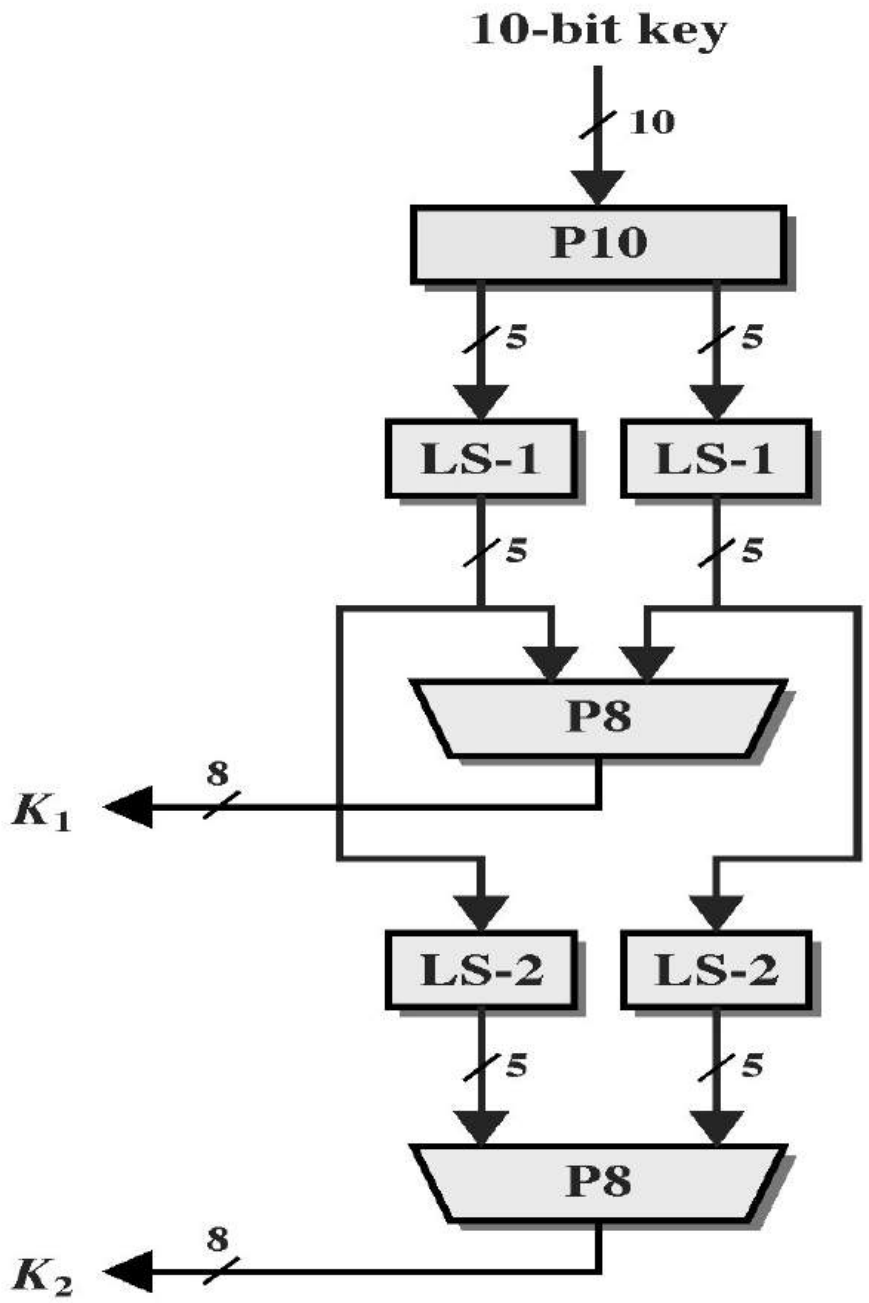


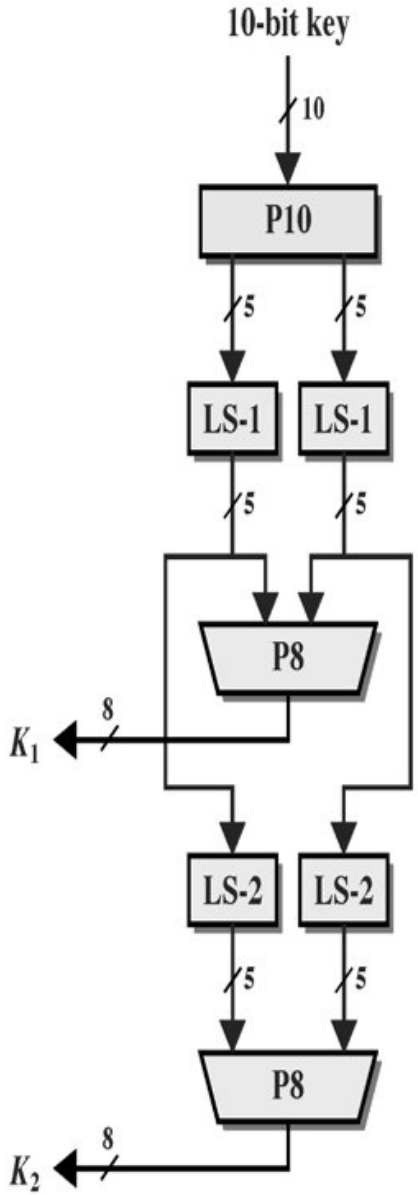
Example-1
S-DES Key generation

Find the subkeys K1 and K2 if the 10-bit Key is 1010000010 with the following permuted tables values:

P10 = { 3, 5, 2, 7, 4, 10, 1, 9, 8, 6}

P8 = { 6, 3, 7, 4, 8, 5, 10, 9}





Find the subkeys K1 and K2 if the 10-bit Key is 1010000010 with the following permuted tables values:

P10 = { 3, 5, 2, 7, 4, 10, 1, 9, 8, 6}

P8 = { 6, 3, 7, 4, 8, 5, 10, 9}

S-DES

i/p - 8 bit (plain text)

o/p - 8 bit (cipher text)

key - 10 bit (K_1, K_2)

Rounds - 2

1 2 3 4 5 6 7 8 9 10

Key - 1010000010

1) P_{10} = K_3 K_5 K_2 K_7 K_4 K_{10} K_1 K_9 K_8 K_6

1 0 0 0 0 0 0 1 1 0 0

2) Divide into 2 halves

l = 10000 r = 01100

3) Apply one-bit left shift

l = 00001 r = 11000

4) Combine both and permute bits by applying in P_8 table

00001 11000

P_8 = K_6 K_3 K_7 K_4 K_8 K_5 K_{10} K_9

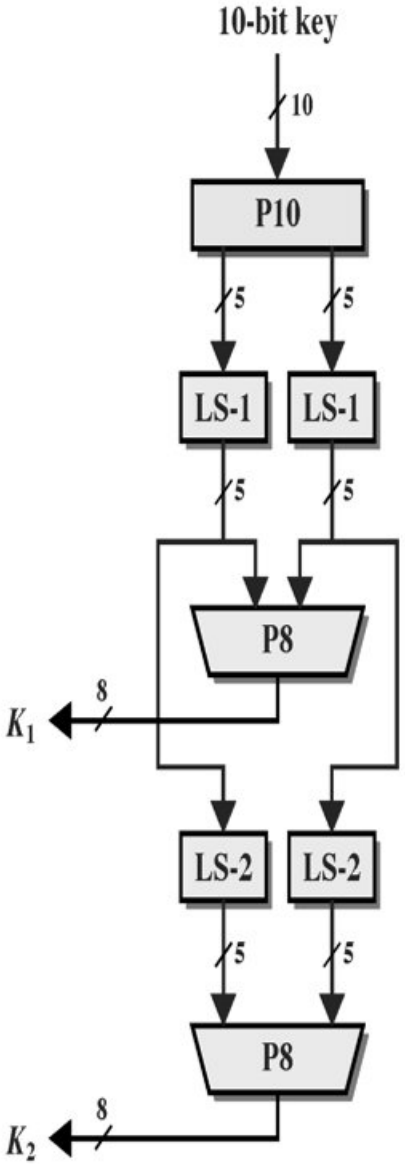
= 1 0 1 0 0 1 0 0

5) o/p from step 3

l = 00001 r = 11000

2-bit left shift

l = 00100 r = 00011



Find the subkeys K1 and K2 if the 10-bit Key is 1010000010 with the following permuted tables values:

P10 = { 3, 5, 2, 7, 4, 10, 1, 9, 8, 6}

P8 = { 6, 3, 7, 4, 8, 5, 10, 9}

b) Combine 2 halves of step 5 and permute by incl in P8

LS-2 = 00100000011

P8 = $K_6 K_3 K_7 K_4 K_8 K_5 K_{10} K_9$

= 0 1 0 0 0 0 1 1

K2

final o/p

Key 1 = 10100100

Key 2 = 01000011

Final sub keys are (also computed in upcoming 2 slides for reference)

Final Output:

Key-1 is: 1 0 1 0 0 1 0 0

Key-2 is: 0 1 0 0 0 0 1 1

Sub-key generation process

Step 1: We accepted a 10-bit key and permuted the bits by putting them in the P10 table.

- Key = 1 0 1 0 0 0 0 0 1 0(k1, k2, k3, k4, k5, k6, k7, k8, k9, k10) = (1, 0, 1, 0, 0, 0, 0, 0, 1, 0)
- P10 Permutation is:
- P10(k1, k2, k3, k4, k5, k6, k7, k8, k9, k10) = (k3, k5, k2, k7, k4, k10, k1, k9, k8, k6)
- After P10, we get 1 0 0 0 0 0 1 1 0 0

Step 2: We divide the key into 2 halves of 5-bit each.

l=1 0 0 0 0, r=0 1 1 0 0

Step 3: Now we apply one bit left-shift on each key.

- l = 0 0 0 0 1, r = 1 1 0 0 0

Step 4: Combine both keys after step 3 and permute the bits by putting them in the P8 table.

The output of the given table is the first key K1.

After LS-1 combined, we get 0 0 0 0 1 1 1 0 0 0

P8 permutation is: P8(k1, k2, k3, k4, k5, k6, k7, k8, k9, k10) = (k6, k3, k7, k4, k8, k5, k10, k9)

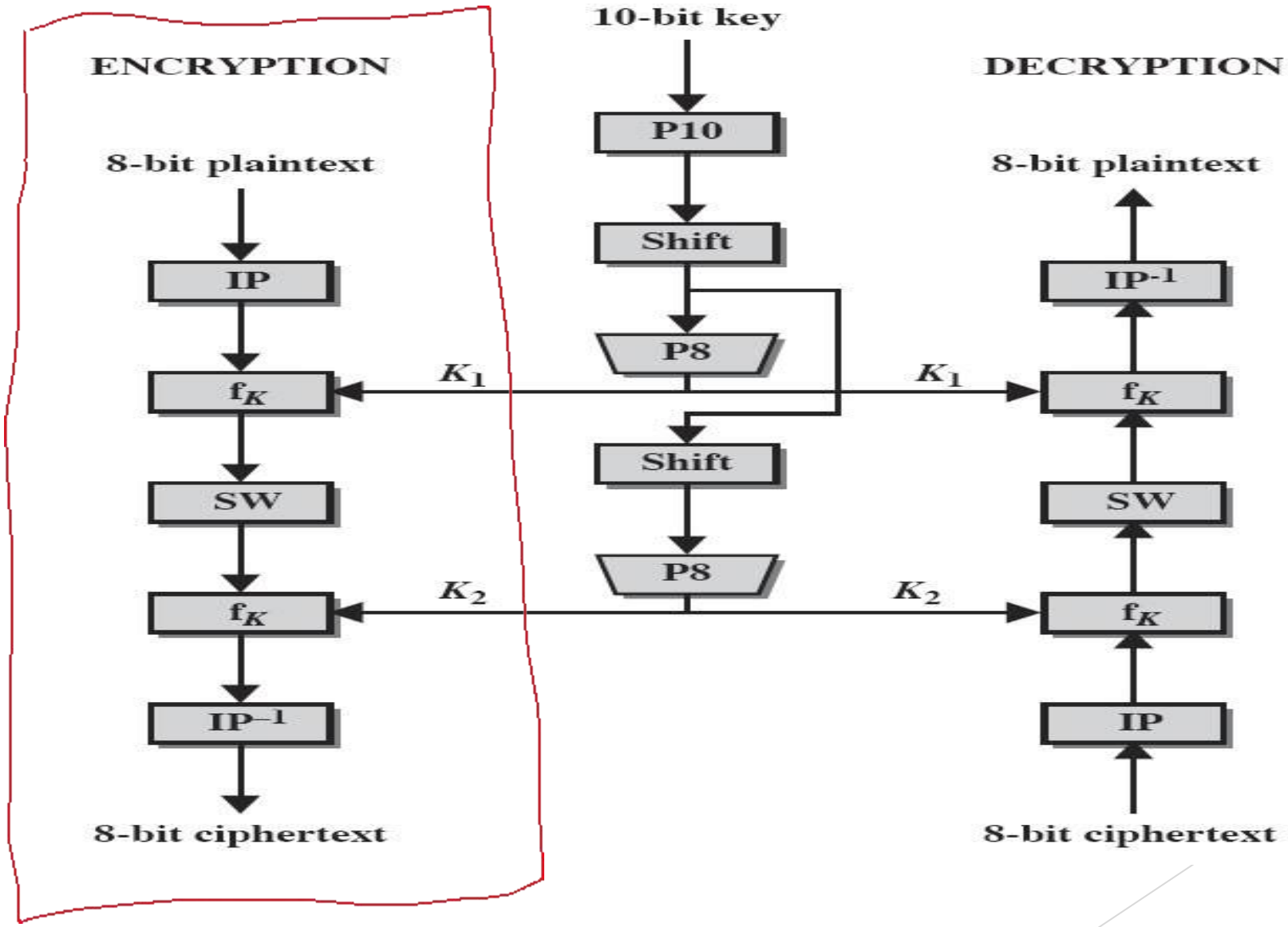
After P8, we get Key-1 : 1 0 1 0 0 1 0 0

Step 5: The output obtained from step 3
i.e. 2 halves after one bit left shift should again undergo the process of two-bit left shift.
Step 3 output - l = 0 0 0 0 1, r = 1 1 0 0 0
After two bit shift - l = 0 0 1 0 0, r = 0 0 0 1 1

Step 6: Combine the 2 halves obtained from step 5 and permute them by putting them in the P8 table.
The output of the given table is the second key K2.
After LS-2 combined = 0 0 1 0 0 0 0 0 1 1
P8 permutation is: P8(k1, k2, k3, k4, k5, k6, k7, k8, k9, k10) = (k6, k3, k7, k4, k8, k5, k10, k9)
After P8, we get Key-2 : 0 1 0 0 0 0 1 1

Final Output:
Key-1 is: 1 0 1 0 0 1 0 0 Key-2 is: 0 1 0 0 0 0 1 1

S-DES algorithm



S-DES Encryption Details

Find the ciphertext if the 8-bit plaintext is 10010111 with the following data:

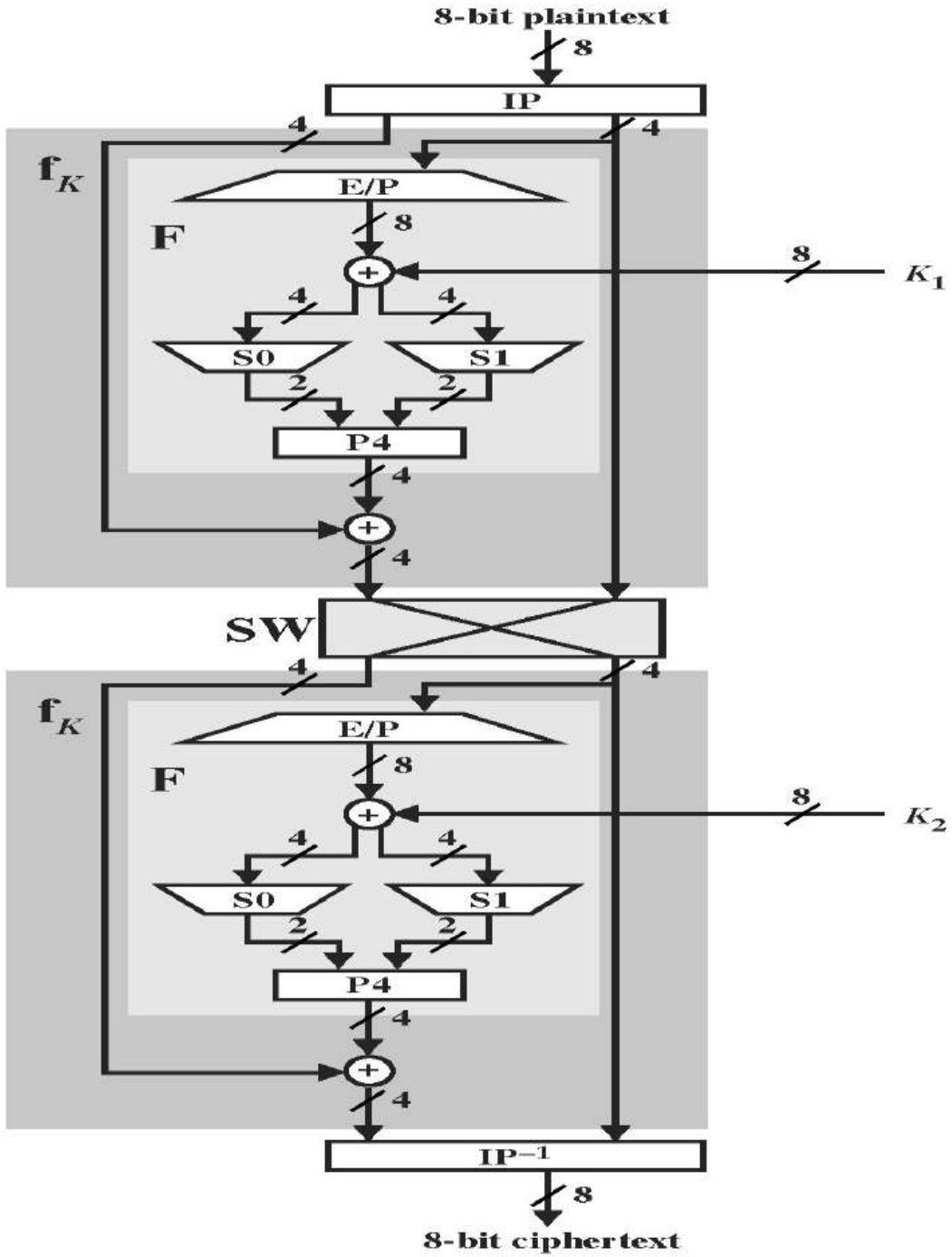
IP = (2,6,3,1,4,8,5,7}

EP = (4,1,2,3,2,3,4,1}

P4 = (2,4,3,1}

IP^{-1} = (4,1,3,5,7,2,8,6}

$$S0 = \begin{array}{c} \\ 0 \\ 1 \\ 2 \\ 3 \end{array} \begin{array}{c} \begin{array}{cccc} 0 & 1 & 2 & 3 \end{array} \\ \begin{bmatrix} 1 & 0 & 3 & 2 \\ 3 & 2 & 1 & 0 \\ 0 & 2 & 1 & 3 \\ 3 & 1 & 3 & 2 \end{bmatrix} \end{array} \qquad S1 = \begin{array}{c} \\ 0 \\ 1 \\ 2 \\ 3 \end{array} \begin{array}{c} \begin{array}{cccc} 0 & 1 & 2 & 3 \end{array} \\ \begin{bmatrix} 0 & 1 & 2 & 3 \\ 2 & 0 & 1 & 3 \\ 3 & 0 & 1 & 0 \\ 2 & 1 & 0 & 3 \end{bmatrix} \end{array}$$



Encryption

- **First, we need to generate 2 keys before encryption.**
- Consider, the entered 10-bit key is - 1 0 1 0 0 0 0 0 1 0
- Therefore,
- Key-1 is - 1 0 1 0 0 1 0 0
- Key-2 is - 0 1 0 0 0 0 1 1
- Entered 8-bit plaintext is - 1 0 0 1 0 1 1 1
- **Encryption process**
- **Step-1:** We perform initial permutation on our 8-bit plain text using the IP table. The initial permutation is defined as –
- After ip = 0 1 0 1 1 1 0 1

IP(k1, k2, k3, k4, k5, k6, k7, k8) = (k2, k6, k3, k1, k4, k8, k5, k7)

$P_P = \{2, 6, 3, 1, 4, 8, 5, 7\}$

$E_P = \{4, 1, 2, 3, 2, 3, 4, 1\}$

$P_4 = (2, 4, 3, 1)$

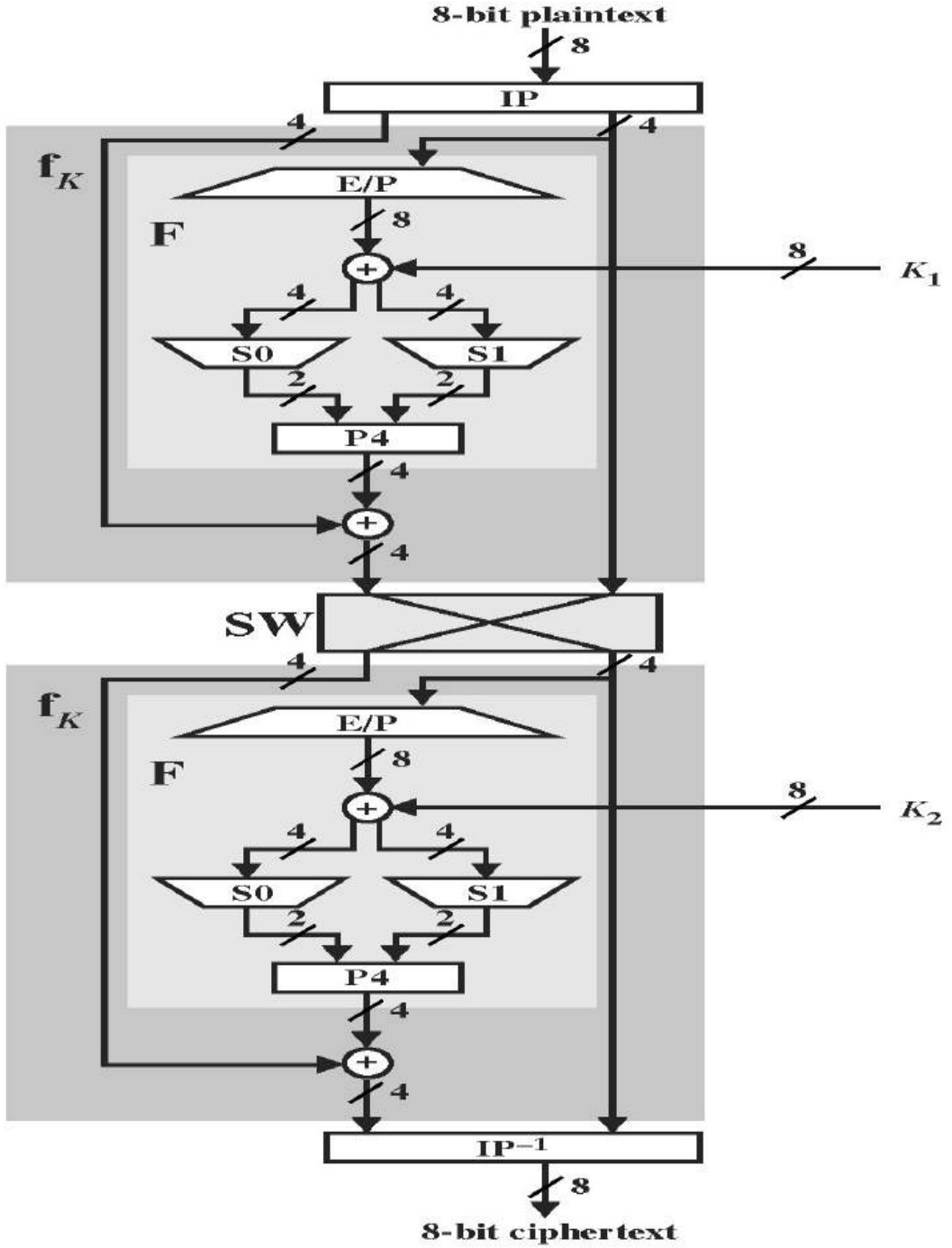
$P_P^{-1} = (4, 1, 3, 5, 7, 2, 8, 6)$

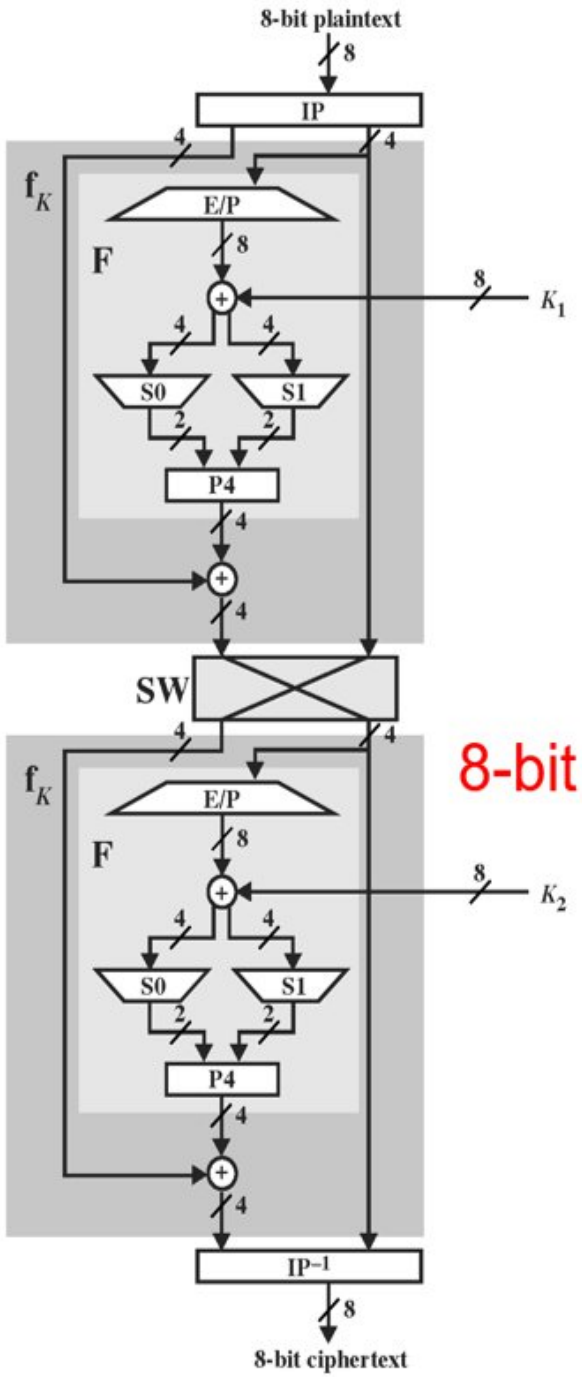
$S_0 =$

	0	1	2	3
0	1	0	3	2
1	3	2	1	0
2	0	2	1	3
3	3	1	3	2

$S_1 =$

	0	1	2	3
0	0	1	2	3
1	2	0	1	3
2	3	0	1	0
3	2	1	0	3





$P_P = \{2, 6, 3, 1, 4, 8, 5, 7\}$

$E_P = \{4, 1, 2, 3, 2, 3, 4, 1\}$

$P_4 = (2, 4, 3, 1)$

$P_P^{-1} = (4, 1, 3, 5, 7, 2, 8, 6)$

$S_0 =$

	0	1	2	3
0	1	0	3	2
1	3	2	1	0
2	0	2	1	3
3	3	1	3	2

$S_1 =$

	0	1	2	3
0	0	1	2	3
1	2	0	1	3
2	3	0	1	0
3	2	1	0	3

8-bit plaintext is - 1 0 0 1 0 1 1 1

P.T = 1 0 0 1 0 1 1 1 (1 2 3 4 5 6 7 8)

Encryption

$P_P = (2, 6, 3, 1, 4, 8, 5, 7)$

$= (0, 1, 0, 1, 1, 1, 0, 1)$

Step 2 – Divide into 2 halves

L = 0 1 0 1 R = 1 1 0 1

On Rt half, Expanded permutation Convert 4 bits into 8 bits

$E_P = (K_4\ K_1\ K_2\ K_3\ K_2\ K_3\ K_4\ K_1)$

$= 1\ 1\ 1\ 0\ 1\ 0\ 1\ 1$

Perform XOR operation using key K_1

$K_1 = 10100100$

(10100100) XOR (11101011)

$= 01001111$

Divide into 2 halves of 4 bits

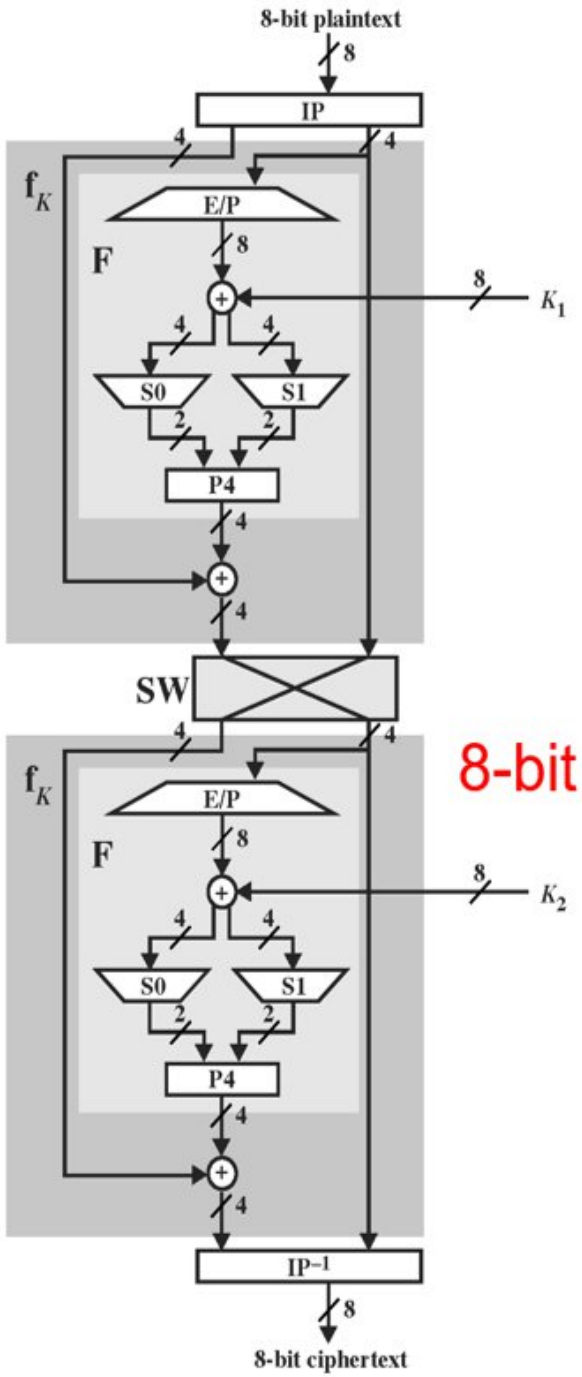
$l = 0100$ $r = 1111$

To $l = 0100$, row = 00, Column = 10 = 2

Check row 0, Column = 2 in S_0

$S_0 = 3 = 11$

	0	1	2	3
0	1	0	3	2
1	3	2	1	0
2	0	2	1	3
3	3	1	3	2



$P_P = \{2, 6, 3, 1, 4, 8, 5, 7\}$

$E_P = \{4, 1, 2, 3, 2, 3, 4, 1\}$

$P_4 = (2, 4, 3, 1)$

$P_P^{-1} = (4, 1, 3, 5, 7, 2, 8, 6)$

$S_0 =$

	0	1	2	3
0	1	0	3	2
1	3	2	1	0
2	0	2	1	3
3	3	1	3	2

$S_1 =$

	0	1	2	3
0	0	1	2	3
1	2	0	1	3
2	3	0	1	0
3	2	1	0	3

8-bit plaintext is - 1 0 0 1 0 1 1 1

for $r = \underbrace{11}_{\text{row}}\ \underbrace{11}_{\text{column}}$

row = 11 = 3, Column = 11 = 3

$S_1 =$

	0	1	2	3
0	0	1	2	3
1	2	0	1	3
2	3	0	1	0
3	2	1	0	3

Check row = 3 Column = 3 in S_1

$S_1 = 3 = 11$

Now Combine $S_0, S_1 = 1111$

Perform permutation using P_4 table

$P_4 = K_2\ K_4\ K_3\ K_1$

$= 1\ 1\ 1\ 1$

Now XOR o/p of P_4 table with left half of permutation table

L = 0101 (from IP)

(0101) XOR (1111)

= 1010

Now Combine right half of IP (1101) and o/p of IP (1010)

10101101

Step 3: Now Divide the op in 2 halves

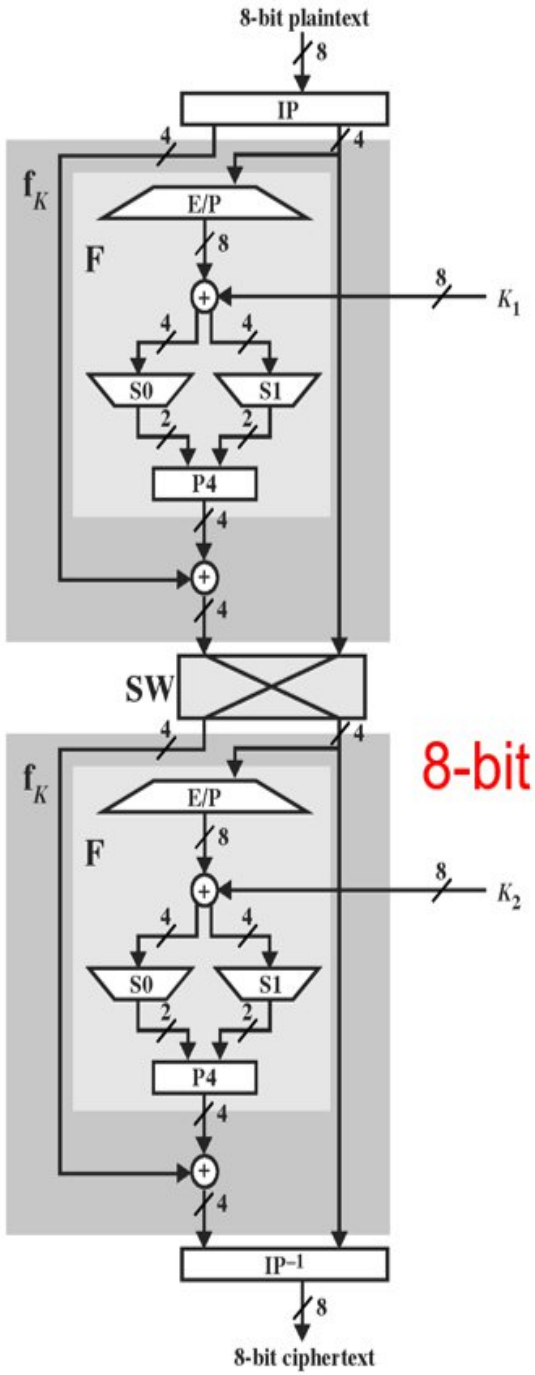
l = 1010 r = 1101

then Swap r to L L to r

l = 1101, r = 1010

Combine again

11011010



$P_P = \{2, 6, 3, 1, 4, 8, 5, 7\}$

$E_P = \{4, 1, 2, 3, 2, 3, 4, 1\}$

$P_4 = (2, 4, 3, 1)$

$P_P^{-1} = (4, 1, 3, 5, 7, 2, 8, 6)$

$S_0 =$

	0	1	2	3
0	1	0	3	2
1	3	2	1	0
2	0	2	1	3
3	3	1	3	2

$S_1 =$

	0	1	2	3
0	0	1	2	3
1	2	0	1	3
2	3	0	1	0
3	2	1	0	3

8-bit plaintext is - 1 0 0 1 0 1 1 1

Step - 4 : Perform step-2. Now we Key as K_2

rt half

EP = 4 1 2 3 2 3 4 1 = 1010

= 0 1 0 1 0 1 0 1

XOR it with Key 2, K_2 = 0 1 0 0 0 0 1 1

(0 1 0 1 0 1 0 1) XOR (0 1 0 0 0 0 1 1)

= 0 0 0 1 0 1 1 0

Divide into 2 halves

l = 0 0 0 1 r = 0 1 1 0

row = 01 = 1

Column = 00 = 0

S_0 = 3 = 11

row = 00 = 0

Column = 11 = 3

S_1 = 3 = 11

Now Combine S_0, S_1 = 1 1 1 1

Perform Permutation using P_4.

$P_4 = K_2\ K_4\ K_3\ K_1$

= 1 1 1 1

Now XOR o/p of P_4 with left half

L = 1 1 0 1

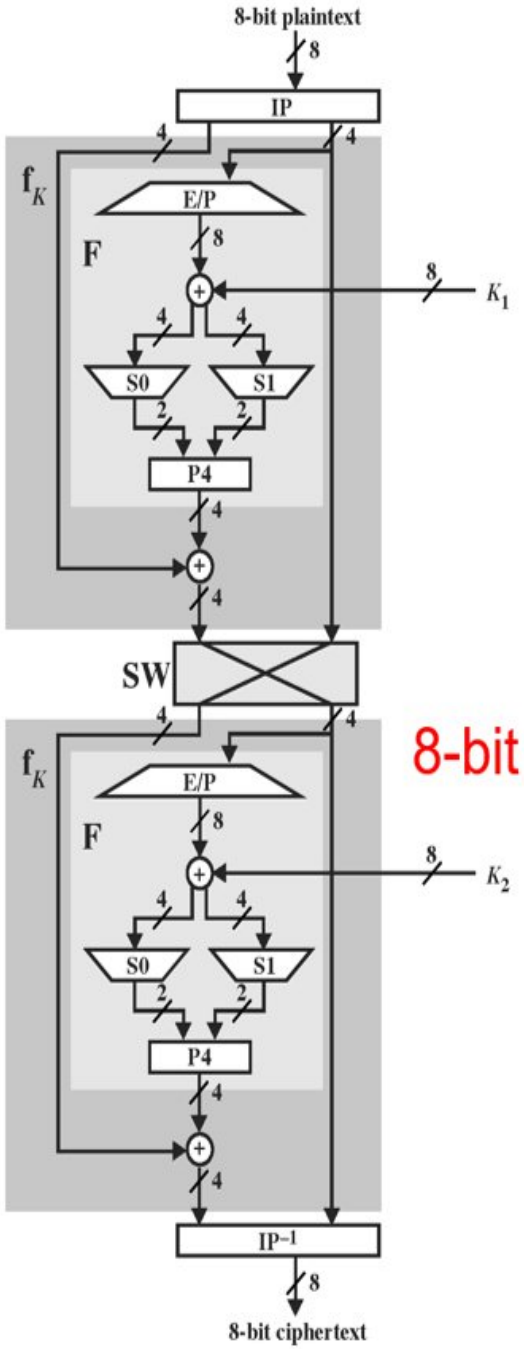
(1 1 0 1) XOR (1 1 1 1)

= 0 0 1 0

Now Combine right half (1010)

And o/p (0010)

= 0 0 1 0 1 0 1 0



$Pp = \{2, 6, 3, 1, 4, 8, 5, 7\}$

$Ep = \{4, 1, 2, 3, 2, 3, 4, 1\}$

$P_4 = (2, 4, 3, 1)$

$Pp^{-1} = (4, 1, 3, 5, 7, 2, 8, 6)$

$S_0 =$

	0	1	2	3
0	1	0	3	2
1	3	2	1	0
2	0	2	1	3
3	3	1	3	2

$S_1 =$

	0	1	2	3
0	0	1	2	3
1	2	0	1	3
2	3	0	1	0
3	2	1	0	3

8-bit plaintext is - 1 0 0 1 0 1 1 1

Now perform inverse initial Permutation

0 0 1 0 1 0 1 0

$Ip^{-1} = k_4\ k_1\ k_3\ k_5\ k_7\ k_2\ k_8\ k_6$

$= 0\ 0\ 1\ 1\ 1\ 0\ 0\ 0$

8-Bit Cipher text = 00111000

- **Step-2:** After the initial permutation, we get an 8-bit block of text which we divide into 2 halves of 4 bit each.
- l = 0 1 0 1 and r = 1 1 0 1
- On the right half, we perform expanded permutation using EP table which converts 4 bits into 8 bits. Expand permutation is defined as –
- After ep = 1 1 1 0 1 0 1 1

  EP(k1, k2, k3, k4) = (k4, k1, k2, k3, k2, k3, k4, k1)
- We perform XOR operation using the first key K1 with the output of expanded permutation.
- (1 0 1 0 0 1 0 0) XOR (1 1 1 0 1 0 1 1) = 0 1 0 0 1 1 1 1
- After XOR operation with 1st Key = 0 1 0 0 1 1 1 1

  Key-1 is - 1 0 1 0 0 1 0 0
- Again we divide the output of XOR into 2 halves of 4 bit each.
- l = 0 1 0 0 and r = 1 1 1 1

- We take the first and fourth bit as row and the second and third bit as a column for our S boxes.
- For l = 0 1 0 0
- row = 00 = 0, column = 10 = 2
- S0 = 3 = 11

S0 = [1,0,3,2
3,2,1,0
0,2,1,3
3,1,3,2]

- For r = 1 1 1 1
- row = 11 = 3, column = 11 = 3
- S1 = 3 = 11

S1= [0,1,2,3
2,0,1,3
3,0,1,0
2,1,0,3]

- After first S-Boxes combining S0 and S1 = 1 1 1 1
- S boxes gives a 2-bit output which we combine to get 4 bits and then perform permutation using the P4 table. P4 is defined as – P4(k1, k2, k3, k4) = (k2, k4, k3, k1)
- After P4 = 1 1 1 1
- We XOR the output of the P4 table with the left half of the initial permutation table i.e. IP table.
- (0 1 0 1) XOR (1 1 1 1) = 1 0 1 0
- After XOR operation with left nibble of after ip = 1 0 1 0
- We combine both halves i.e. right half of initial permutation and output of ip.
- Combine 1 1 0 1 and 1 0 1 0
- After combine = 1 0 1 0 1 1 0 1

- **Step-3:** Now, divide the output into two halves of 4 bit each. Combine them again, but now the left part should become right and the right part should become left. After step 3 = 1 1 0 1 1 0 1 0
- **Step-4:** Again perform step 2, but this time while doing XOR operation after expanded permutation use key 2 instead of key 1.
- After second ep = 0 1 0 1 0 1 0 1

Expand permutation is defined as - 4 1 2 3 2 3 4 1

- After XOR operation with 2nd Key = 0 0 0 1 0 1 1 0
- After second S-Boxes = 1 1 1 1
- After P4 = 1 1 1 1

P4 is defined as - 2 4 3 1

- After XOR operation with left nibble of after first part = 0 0 1 0
- After second part = 0 0 1 0 1 0 1 0
- l = 1 1 0 1 and r = 1 0 1 0
- On the right half, we perform expanded permutation using EP table which converts 4 bits into 8 bits. Expand permutation is defined as –
- After second ep = 0 1 0 1 0 1 0 1

EP(k1, k2, k3, k4) = (k4, k1, k2, k3, k2, k3, k4, k1)

- We perform XOR operation using second key K2 with the output of expanded permutation
- (0 1 0 0 0 0 1 1) XOR (0 1 0 1 0 1 0 1) = 0 0 0 1 0 1 1 0
- After XOR operation with 2nd Key = 0 0 0 1 0 1 1 0
- Again we divide the output of XOR into 2 halves of 4 bit each.

Key-2 is - 0 1 0 0 0 0 1 1

l = 0 0 0 1 and r = 0 1 1 0

- We take the first and fourth bit as row and the second and third bit as a column for our S boxes.
- For l = 0 0 0 1
- row = 01 = 1 , column = 00 = 0
- S0 = 3 = 11

S0 = [1,0,3,2
3,2,1,0
0,2,1,3
3,1,3,2]

- For r = 0 1 1 0
- row = 00 = 0 , column = 11 = 3
- S1 = 3 = 11

S1 = [0,1,2,3
2,0,1,3
3,0,1,0
2,1,0,3]

- After first S-Boxes combining S0 and S1 = 1 1 1 1
- S boxes gives a 2-bit output which we combine to get 4 bits and then perform permutation using the P4 table. P4 is defined as –

P4(k1, k2, k3, k4) = (k2, k4, k3, k1)

- After P4 = 1 1 1 1
- We XOR the output of the P4 table with the left half of the initial permutation table i.e. IP table.
- (1 1 0 1) XOR (1 1 1 1) = 0 0 1 0
- After XOR operation with left nibble of after first part = 0 0 1 0
- We combine both halves i.e. right half of initial permutation and output of ip.
- Combine 1 0 1 0 and 0 0 1 0
- After combine = 0 0 1 0 1 0 1 0
- After second part = 0 0 1 0 1 0 1 0

- **Step-5:**Perform inverse initial permutation. The output of this table is the cipher text of 8 bit.
- Output of step 4 : 0 0 1 0 1 0 1 0
- Inverse Initial permutation is defined as –

  IP-1(k1, k2, k3, k4, k5, k6, k7, k8) = (k4, k1, k3, k5, k7, k2, k8, k6)

- **8-bit Cipher Text will be = 0 0 1 1 1 0 0 0**

S-DES Example-2

- Encrypt and decrypt the plaintext 9A (hexadecimal) using Simplified DES (S-DES) with the 10-bit key 1100101110. Show all intermediate steps.

- Steps to follow ,
  - Since Hexadecimal is asked, you should utilize hexadecimal to binary conversion

Hex digit	Binary
9	**1001**
A	**1010**

  - For Text as Plaintext , Plaintext-> ASCII-> Binary
  - For hexadecimal, Hexadecimal-> Binary

S-DES Example-3

- Encrypt the plaintext “00101000” using S-DES for the key value “1100011110” with the following data:

IP = (2,6,3,1,4,8,5,7}

EP = (4,1,2,3,2,3,4,1}

P4 = (2,4,3,1}

IP^{-1} = (4,1,3,5,7,2,8,6}

P10 = { 3, 5, 2, 7, 4, 10, 1, 9, 8, 6}

P8 = { 6, 3, 7, 4, 8, 5, 10, 9}

$$S0 = \begin{array}{c} \\ 0 \\ 1 \\ 2 \\ 3 \end{array} \begin{array}{c} \begin{array}{cccc} 0 & 1 & 2 & 3 \end{array} \\ \begin{bmatrix} 1 & 0 & 3 & 2 \\ 3 & 2 & 1 & 0 \\ 0 & 2 & 1 & 3 \\ 3 & 1 & 3 & 2 \end{bmatrix} \end{array} \qquad S1 = \begin{array}{c} \\ 0 \\ 1 \\ 2 \\ 3 \end{array} \begin{array}{c} \begin{array}{cccc} 0 & 1 & 2 & 3 \end{array} \\ \begin{bmatrix} 0 & 1 & 2 & 3 \\ 2 & 0 & 1 & 3 \\ 3 & 0 & 1 & 0 \\ 2 & 1 & 0 & 3 \end{bmatrix} \end{array}$$

2.1 Key Generation

The keys k_1 and k_2 are derived using the functions $P10$, Shift, and $P8$.
$P10$ is defined as follows:

P10									
3	5	2	7	4	10	1	9	8	6

$P8$ is defined to be as follows:

P8							
6	3	7	4	8	5	10	9

The first key k_1 is therefore equal to:

Bit #	1	2	3	4	5	6	7	8	9	10
K	1	1	0	0	0	1	1	1	1	0
$P10(K)$	0	0	1	1	0	0	1	1	1	1
$Shift(P10(K))$	0	1	1	0	0	1	1	1	1	0
$P8(Shift(P10(K)))$	1	1	1	0	1	0	0	1		

The second key k_2 is derived in a similar manner:

Bit #	1	2	3	4	5	6	7	8	9	10
K	1	1	0	0	0	1	1	1	1	0
$P10(K)$	0	0	1	1	0	0	1	1	1	1
$Shift^3(P10(K)))$	1	0	0	0	1	1	1	0	1	1
$P8(Shift^2(P10(K)))$	1	0	1	0	0	1	1	1		

So we have the two keys $k_1 = \{1110\ 1001\}$ and $k_2 = \{1010\ 0111\}$

2.2 Initial and Final Permutation

The plaintext undergoes an initial permutation when it enters the encryption function, IP. It undergoes a reverse final permutation at the end IP^{-1}.

The function IP is defined as follows:

IP							
2	6	3	1	4	8	5	7

The function IP^{-1} is defined as follows:

IP^{-1}							
4	1	3	5	7	2	8	6

Applied to the input, we have the following after the initial permutation:

Bit #	1	2	3	4	5	6	7	8
P	0	0	1	0	1	0	0	0
$IP(P)$	0	0	1	0	0	0	1	0

2.3 Functions f_K, SW, K

- The function f_k is defined as follows. Let $P = (L, R)$, then $f_K(L, R) = (L \oplus F(R, SK), R)$.
- The function SW just switches the two halves of the plaintext, so $SW(L, R) \rightarrow (R, L)$
- The function $F(p, k)$ takes a four bit string p and eight bit key k and produces a four bit output. It performs the following steps.

1. First it runs an expansion permutation E/P:

E/P							
4	1	2	3	2	3	4	1

2. Then it XORs the key with the result of the E/P function
3. Then it substitutes the two halves based on the S-Boxes.
4. Finally, the output from the S-Boxes undergoes the $P4$ permutation:

P4			
2	4	3	1

Applying the functions, we must perform the following steps: $IP^{-1} \circ f_{K_2} \circ SW \circ f_{K_1} \circ IP$

1. We have already calculated $IP(P) = \{0010\ 0010\}$. Applying the next functions:
2. $f_{K_1}(L, R) = f_{\{1110\ 1001\}}(0010\ 0010) = (0010 \oplus F(0010, \{1110\ 1001\}), 0010)$
3. $F(0010, \{1110\ 1001\}) = P4 \circ SBoxes \circ \{1110\ 1001\} \oplus (E/P(0010))$
4. The steps are:

Bit #	1	2	3	4	5	6	7	8
R	0	0	1	0				
E/P(R)	0	0	0	1	0	1	0	0
k_1	1	1	1	0	1	0	0	1
E/P(R)$\oplus k_1$	1	1	1	1	1	1	0	1
SBoxes(E/P(R)$\oplus k_1$)	1	0	0	0				
P4(Sboxes(E/P(R)$\oplus k_1$))	0	0	0	1				

5. The result from F is therefore 0001
6. Calculating we then have $f_{k_1}(L, R) = (0010 \oplus 0001, 0010) = (0011, 0010)$
7. So far, then $L = 0011$ and $R = 0010$. SW just swaps them so $R = 0011$ and $L = 0010$.
8. We now do the calculation of $f_{k_2}(L, R) = f_{\{1010\ 0111\}}(0010\ 0011) = (0010 \oplus F(0011, \{1010\ 0111\}, 0011))$

9. The steps for F are as above:

Bit #	1	2	3	4	5	6	7	8
R	0	0	1	1				
E/P(R)	1	0	0	1	0	1	1	0
k_2	1	0	1	0	0	1	1	1
E/P(R)$\oplus k_2$	0	0	1	1	0	0	0	1
SBoxes(E/P(R)$\oplus k_2$)	1	0	1	0				
P4(Sboxes(E/P(R)$\oplus k_2$))	0	0	1	1				

10. So now we have the outcome of F as 0011

11. Calculating we then have $f_{k_2}(L, R) = (0010 \oplus 0011, 0011) = (0001, 0011)$

12. Last, we perform the IP^{-1} permutation:

Bit #	1	2	3	4	5	6	7	8
R,L	0	0	0	1	0	0	1	1
IP^{-1}(R,L)	1	0	0	0	1	0	1	0

13. So the final result of the encryption is 1000 1010.

Advanced Encryption Standard

AES Requirements

- private key symmetric block cipher
- 128-bit data, 128/192/256-bit keys
- stronger & faster than Triple-DES
- provide full specification & design details
- both C & Java implementations

AES Evaluation Criteria

- initial criteria:
  - security – effort for practical cryptanalysis
  - cost – in terms of computational efficiency
  - algorithm & implementation characteristics
- final criteria
  - general security
  - ease of software & hardware implementation
  - implementation attacks
  - flexibility (in en/decrypt, keying, other factors)

The AES Cipher - Rijndael

- designed by Rijmen-Daemen in Belgium
- Plaintext /Ciphertext :128 bit data and AES has 128/192/256 bit keys,

•**AES-128:** 10 rounds
•**AES-192:** 12 rounds
•**AES-256:** 14 rounds

- an **iterative** rather than **feistel** cipher
  - processes data as block of 4 columns of 4 bytes
  - operates on entire data block in every round
- designed to be:
  - resistant against known attacks
  - speed and code compactness on many CPUs
  - design simplicity

DES vs AES

The AES Cipher - Rijndael

- designed by Rijmen-Daemen in Belgium
- has 128/192/256 bit keys, 128 bit data
- an **iterative** rather than **feistel** cipher
  - processes data as block of 4 columns of 4 bytes
  - operates on entire data block in every round
- designed to be:
  - resistant against known attacks
  - speed and code compactness on many CPUs
  - design simplicity

Data Encryption Standard (DES)

- most widely used block cipher in world
- adopted in 1977 by NIST
- **encrypts 64-bit data using 56-bit key**
- has widespread use
- has been considerable controversy over its security

Strength of DES – Key Size

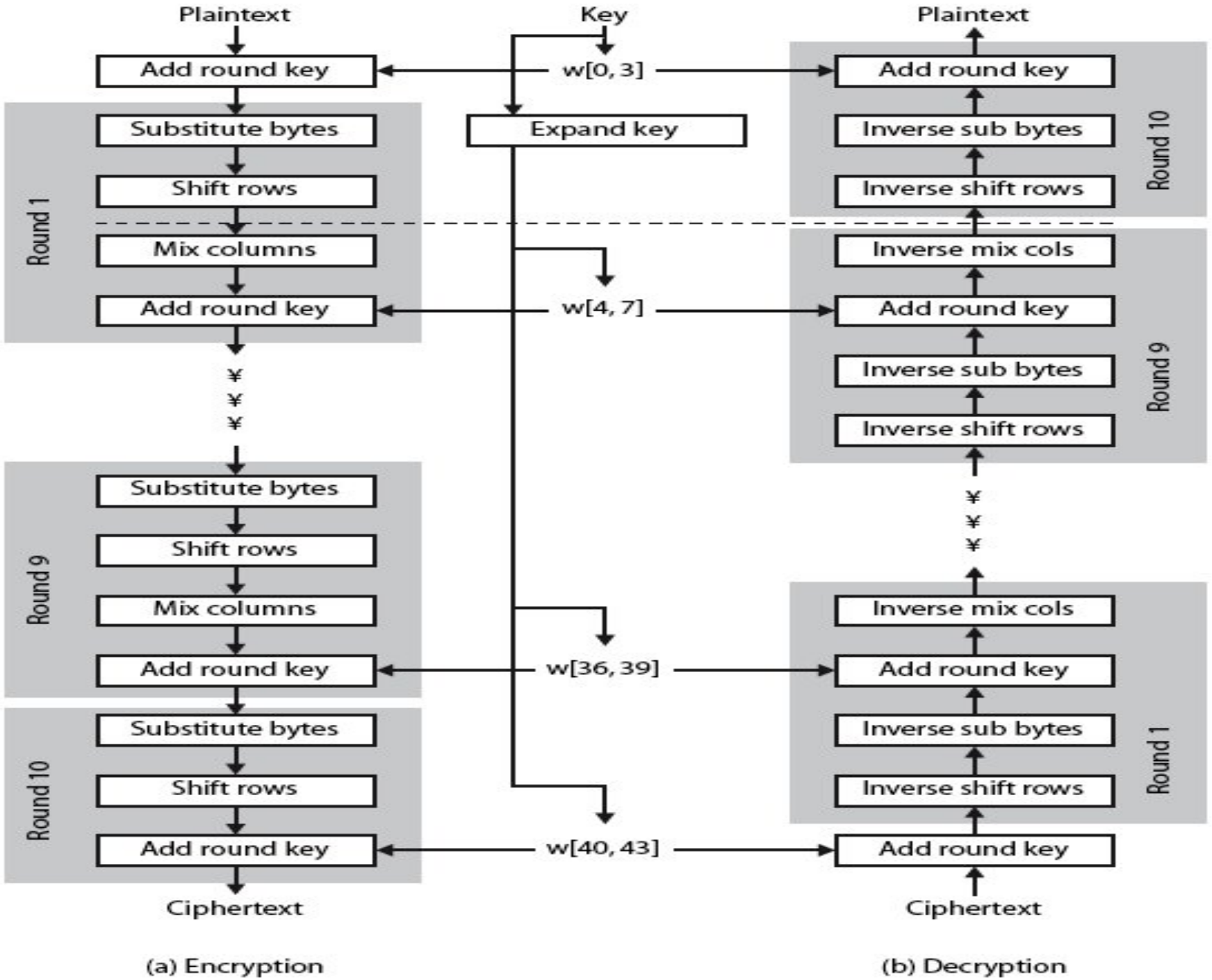
- 56-bit keys have $2^{56} = 7.2 \times 10^{16}$ values
- brute force search looks hard
- recent advances have shown is possible
  - on Internet in a few months
  - on dedicated h/w (EFF) in a few days
  - combined in 22hrs!
- still must be able to recognize plaintext
- must now consider alternatives to DES

Rijndael

- data block of 4 columns of 4 bytes is state
- key is expanded to array of words
- has 9/11/13 rounds in which state undergoes:
  - byte substitution (1 S-box used on every byte)
  - shift rows (permute bytes between groups/columns)
  - mix columns (subs using matrix multipy of groups)
  - add round key (XOR state with key material)
  - view as alternating XOR key & scramble data bytes
- initial XOR key material & incomplete last round
- with fast XOR & table lookup implementation

AES ARCHITECTURE

Plaintext, Key and CipherText: 128 bits and 10 rounds


Plaintext
Key
Plaintext
Add round key
w[0, 3]
Add round key
Substitute bytes
Expand key
Inverse sub bytes
Round 10
Shift rows
Inverse shift rows
Round 1
Mix columns
Inverse mix cols
Add round key
w[4, 7]
Add round key
Inverse sub bytes
Round 9
Inverse shift rows
Substitute bytes
Shift rows
Inverse mix cols
Mix columns
Add round key
w[36, 39]
Add round key
Substitute bytes
Inverse sub bytes
Round 1
Shift rows
Inverse shift rows
Round 10
Add round key
w[40, 43]
Add round key
Ciphertext
Ciphertext
(a) Encryption
(b) Decryption

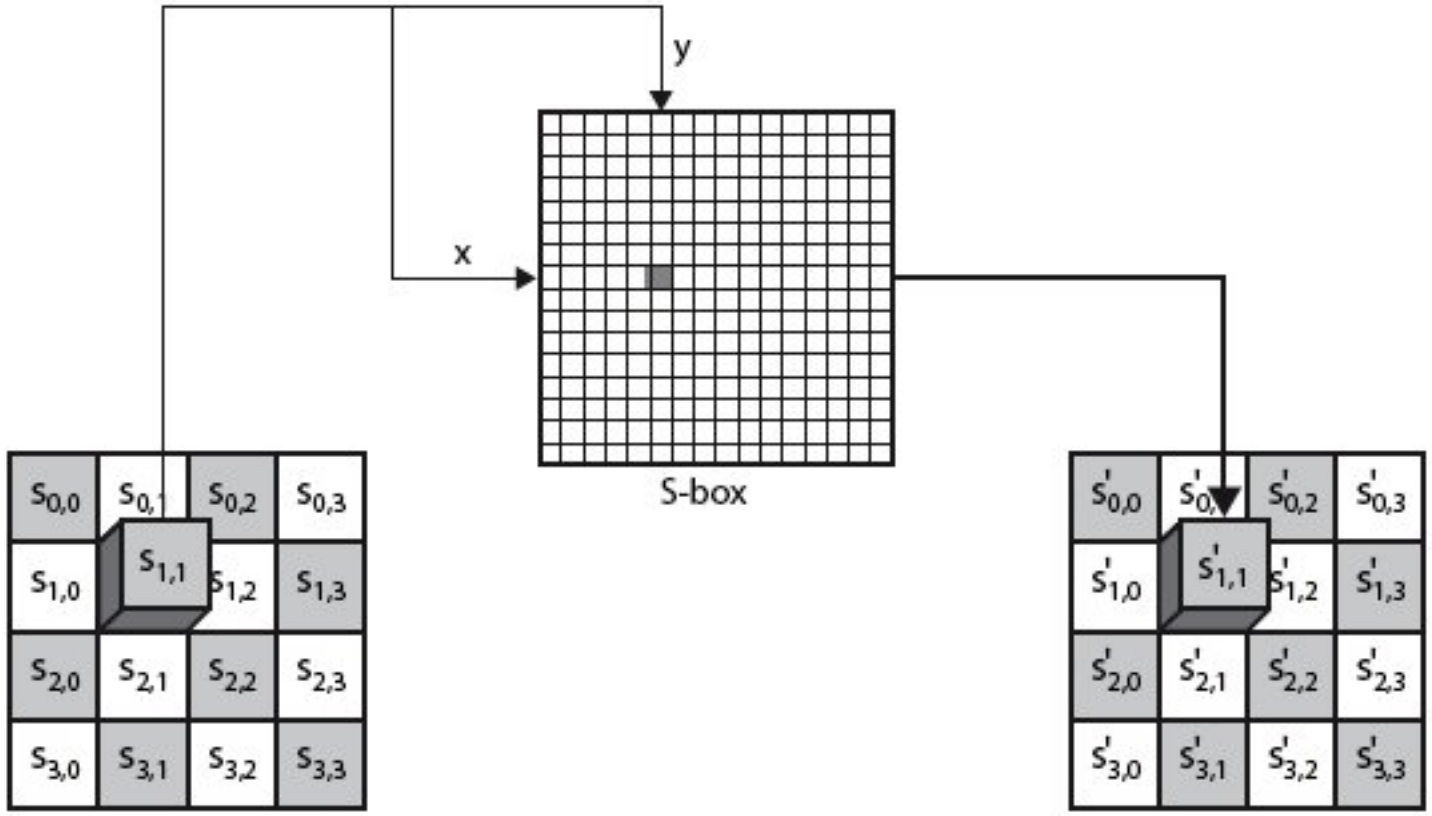
AES steps

- Byte Substitution
- Shift Rows
- Mix Columns
- Add Round Key

Byte Substitution

- a simple substitution of each byte
- uses one table of 16x16 bytes containing a permutation of all 256 8-bit values
- each byte of state is replaced by byte indexed by row (left 4-bits) & column (right 4-bits)
  - eg. byte {95} is replaced by byte in row 9 column 5
  - which has value {2A}
- S-box constructed using defined transformation of values in GF(2^8)
- designed to be resistant to all known attacks

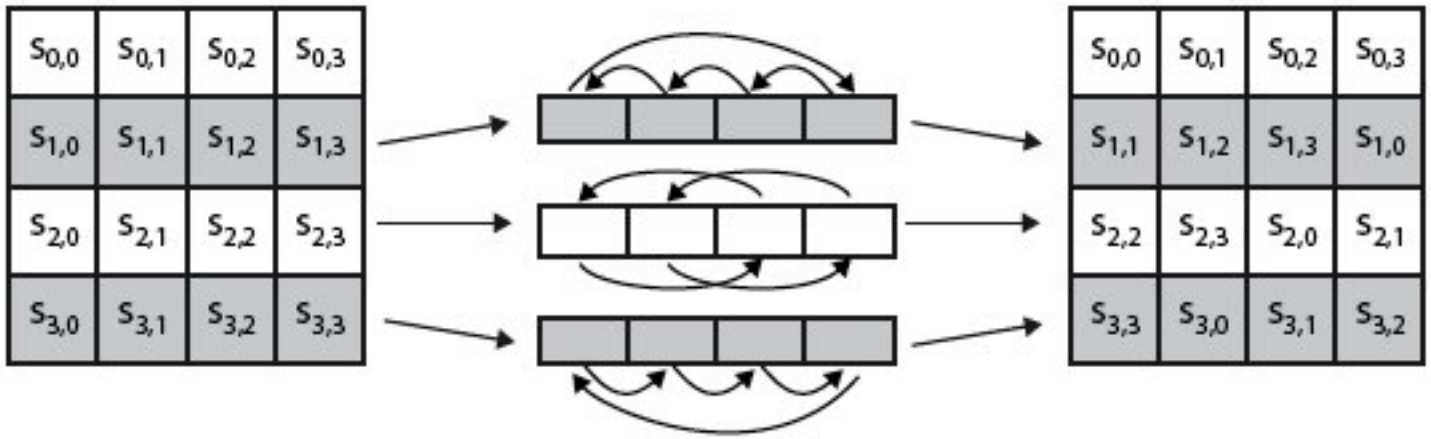
Byte Substitution



Shift Rows

- a circular byte shift in each each
  - 1st row is unchanged
  - 2nd row does 1 byte circular shift to left
  - 3rd row does 2 byte circular shift to left
  - 4th row does 3 byte circular shift to left
- decrypt inverts using shifts to right
- since state is processed by columns, this step permutes bytes between the columns

Shift Rows

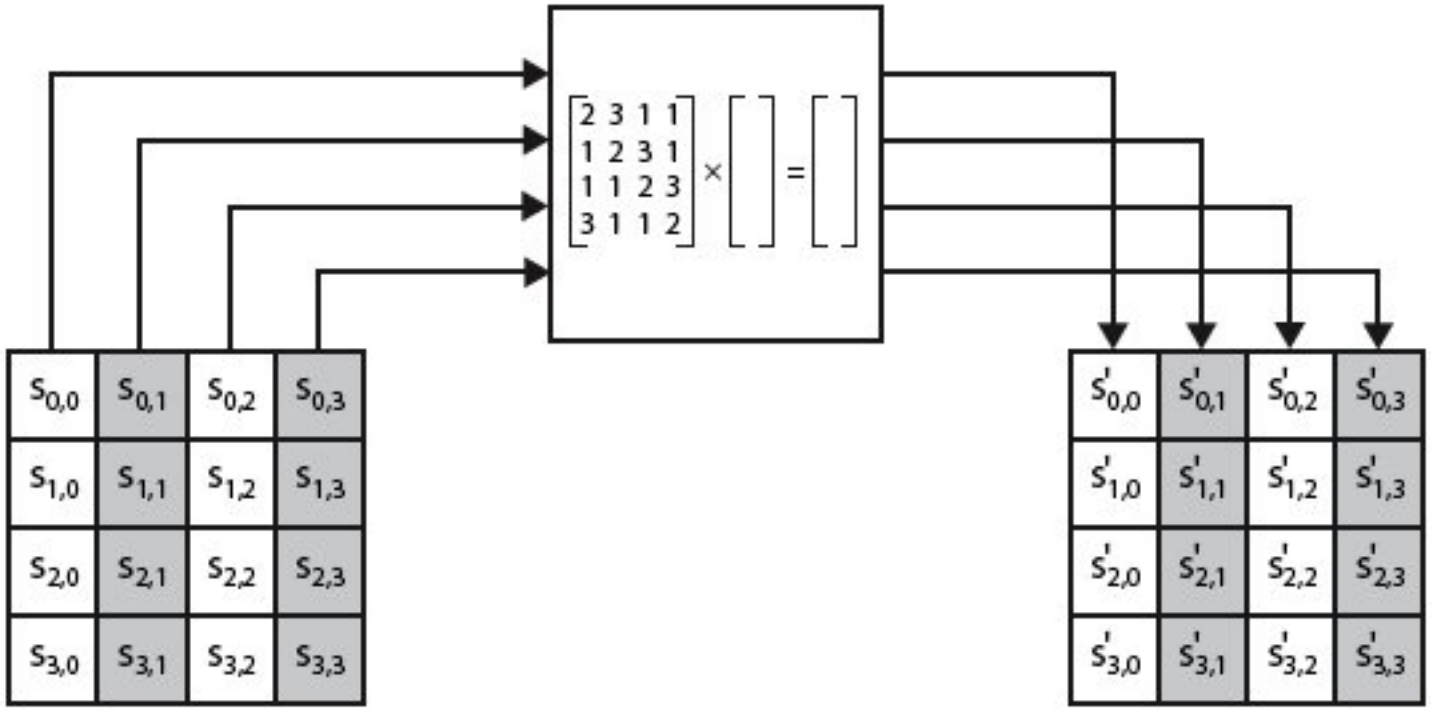

$S_{0,0}$ $S_{0,1}$ $S_{0,2}$ $S_{0,3}$
$S_{1,0}$ $S_{1,1}$ $S_{1,2}$ $S_{1,3}$
$S_{2,0}$ $S_{2,1}$ $S_{2,2}$ $S_{2,3}$
$S_{3,0}$ $S_{3,1}$ $S_{3,2}$ $S_{3,3}$
$S_{0,0}$ $S_{0,1}$ $S_{0,2}$ $S_{0,3}$
$S_{1,1}$ $S_{1,2}$ $S_{1,3}$ $S_{1,0}$
$S_{2,2}$ $S_{2,3}$ $S_{2,0}$ $S_{2,1}$
$S_{3,3}$ $S_{3,0}$ $S_{3,1}$ $S_{3,2}$

Mix Columns

- each column is processed separately
- each byte is replaced by a value dependent on all 4 bytes in the column
- effectively a matrix multiplication in (Galois Field)GF(2^8) using prime poly $m(x) = x^8+x^4+x^3+x+1$

$$\begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \begin{bmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,0} & s_{1,1} & s_{1,2} & s_{1,3} \\ s_{2,0} & s_{2,1} & s_{2,2} & s_{2,3} \\ s_{3,0} & s_{3,1} & s_{3,2} & s_{3,3} \end{bmatrix} = \begin{bmatrix} s'_{0,0} & s'_{0,1} & s'_{0,2} & s'_{0,3} \\ s'_{1,0} & s'_{1,1} & s'_{1,2} & s'_{1,3} \\ s'_{2,0} & s'_{2,1} & s'_{2,2} & s'_{2,3} \\ s'_{3,0} & s'_{3,1} & s'_{3,2} & s'_{3,3} \end{bmatrix}$$

Mix Columns


2 3 1 1
1 2 3 1
1 1 2 3
3 1 1 2
×
=
$s_{0,0}$ $s_{0,1}$ $s_{0,2}$ $s_{0,3}$
$s_{1,0}$ $s_{1,1}$ $s_{1,2}$ $s_{1,3}$
$s_{2,0}$ $s_{2,1}$ $s_{2,2}$ $s_{2,3}$
$s_{3,0}$ $s_{3,1}$ $s_{3,2}$ $s_{3,3}$
$s'_{0,0}$ $s'_{0,1}$ $s'_{0,2}$ $s'_{0,3}$
$s'_{1,0}$ $s'_{1,1}$ $s'_{1,2}$ $s'_{1,3}$
$s'_{2,0}$ $s'_{2,1}$ $s'_{2,2}$ $s'_{2,3}$
$s'_{3,0}$ $s'_{3,1}$ $s'_{3,2}$ $s'_{3,3}$

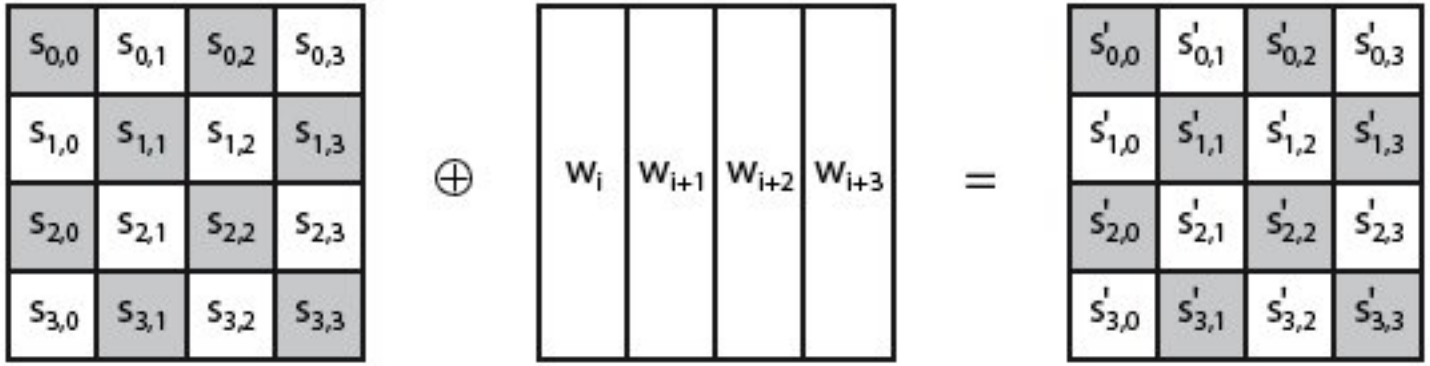
Mix Columns

- can express each col as 4 equations
  - to derive each new byte in col
- decryption requires use of inverse matrix
  - with larger coefficients, hence a little harder
- have an alternate characterisation
  - each column a 4-term polynomial
  - with coefficients in GF(2^8)
  - and polynomials multiplied modulo (x^4+1)

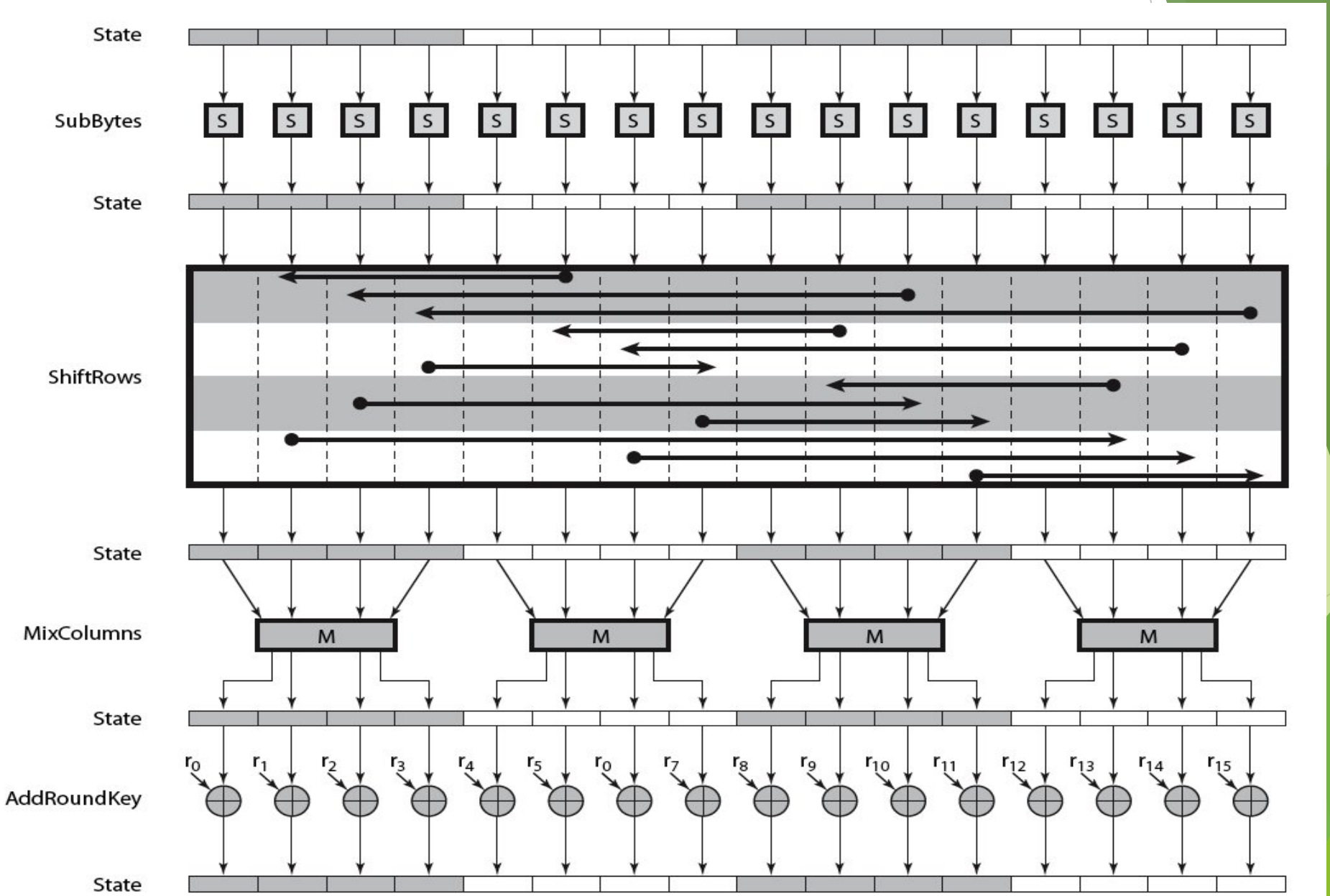
Add Round Key

- XOR state with 128-bits of the round key
- again processed by column (though effectively a series of byte operations)
- inverse for decryption identical
  - since XOR own inverse, with reversed keys
- designed to be as simple as possible
  - a form of Vernam cipher on expanded key
  - requires other stages for complexity / security

Add Round Key



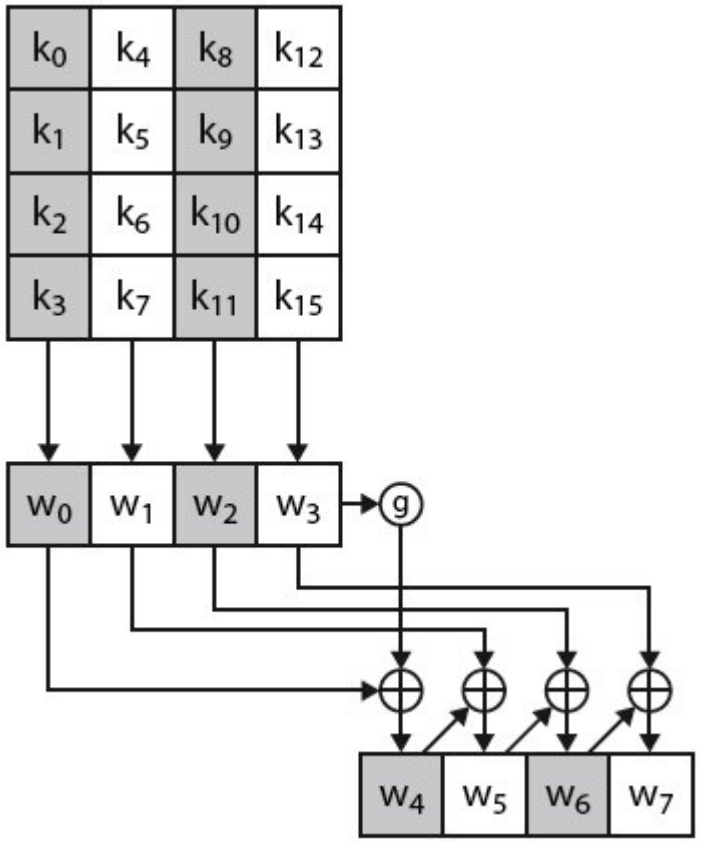
AES Round


State
SubBytes
S
State
ShiftRows
State
MixColumns
M
State
AddRoundKey
r0 r1 r2 r3 r4 r5 r0 r7 r8 r9 r10 r11 r12 r13 r14 r15
State

AES Key Expansion

- takes 128-bit (16-byte) key and expands into array of 44/52/60 32-bit words
- start by copying key into first 4 words
- then loop creating words that depend on values in previous & 4 places back
  - in 3 of 4 cases just XOR these together
  - 1st word in 4 has rotate + S-box + XOR round constant on previous, before XOR 4th back

AES Key Expansion



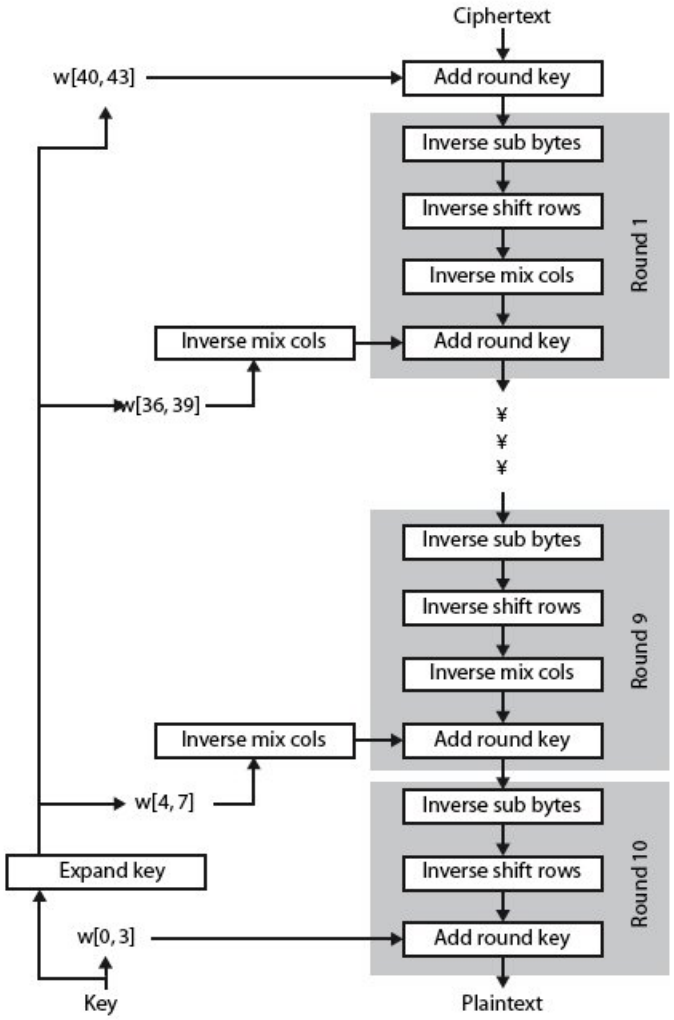
Key Expansion Rationale

- designed to resist known attacks
- design criteria included
  - knowing part key insufficient to find many more
  - invertible transformation
  - fast on wide range of CPU's
  - use round constants to break symmetry
  - diffuse key bits into round keys
  - enough non-linearity to hinder analysis
  - simplicity of description

AES Decryption

- AES decryption is not identical to encryption since steps done in reverse
- but can define an equivalent inverse cipher with steps as for encryption
  - but using inverses of each step
  - with a different key schedule
- works since result is unchanged when
  - swap byte substitution & shift rows
  - swap mix columns & add (tweaked) round key

AES Decryption (128 bit key size)



AES Round Transformation Function

Solved example

Round Functions

- Sub Byte (Encrypt: S box, Decrypt : Inv S box)
- Shift Row (Encrypt: Left Shift, Decrypt : Right Shift)
- Mix Column (Encrypt: Mix col, Decrypt : Inv Mix Col)
- Add Round Key

		y															
		0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
x	0	63	7C	77	7B	F2	6B	6F	C5	30	01	67	2B	FE	D7	AB	76
	1	CA	82	C9	7D	FA	59	47	F0	AD	D4	A2	AF	9C	A4	72	C0
	2	B7	FD	93	26	36	3F	F7	CC	34	A5	E5	F1	71	D8	31	15
	3	04	C7	23	C3	18	96	05	9A	07	12	80	E2	EB	27	B2	75
	4	09	83	2C	1A	1B	6E	5A	A0	52	3B	D6	B3	29	E3	2F	84
	5	53	D1	00	ED	20	FC	B1	5B	6A	CB	BE	39	4A	4C	58	CF
	6	D0	EF	AA	FB	43	4D	33	85	45	F9	02	7F	50	3C	9F	A8
	7	51	A3	40	8F	92	9D	38	F5	BC	B6	DA	21	10	FF	F3	D2
	8	CD	0C	13	EC	5F	97	44	17	C4	A7	7E	3D	64	5D	19	73
	9	60	81	4F	DC	22	2A	90	88	46	EE	B8	14	DE	5E	0B	DB
	A	E0	32	3A	0A	49	06	24	5C	C2	D3	AC	62	91	95	E4	79
	B	E7	C8	37	6D	8D	D5	4E	A9	6C	56	F4	EA	65	7A	AE	08
	C	BA	78	25	2E	1C	A6	B4	C6	E8	DD	74	1F	4B	BD	8B	8A
	D	70	3E	B5	66	48	03	F6	0E	61	35	57	B9	86	C1	1D	9E
	E	E1	F8	98	11	69	D9	8E	94	9B	1E	87	E9	CE	55	28	DF
	F	8C	A1	89	0D	BF	E6	42	68	41	99	2D	0F	B0	54	BB	16

(a) S-box

		y														
		0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
x	0	52	09	6A	D5	30	36	A5	38	BF	40	A3	9E	81	F3	D7	FB
	1	7C	E3	39	82	9B	2F	FF	87	34	8E	43	44	C4	DE	E9	CB
	2	54	7B	94	32	A6	C2	23	3D	EE	4C	95	0B	42	FA	C3	4E
	3	08	2E	A1	66	28	D9	24	B2	76	5B	A2	49	6D	8B	D1	25
	4	72	F8	F6	64	86	68	98	16	D4	A4	5C	CC	5D	65	B6	92
	5	6C	70	48	50	FD	ED	B9	DA	5E	15	46	57	A7	8D	9D	84
	6	90	D8	AB	00	8C	BC	D3	0A	F7	E4	58	05	B8	B3	45	06
	7	D0	2C	1E	8F	CA	3F	0F	02	C1	AF	BD	03	01	13	8A	6B
	8	3A	91	11	41	4F	67	DC	EA	97	F2	CF	CE	F0	B4	E6	73
	9	96	AC	74	22	E7	AD	35	85	E2	F9	37	E8	1C	75	DF	6E
	A	47	F1	1A	71	1D	29	C5	89	6F	B7	62	0E	AA	18	BE	1B
	B	FC	56	3E	4B	C6	D2	79	20	9A	DB	C0	FE	78	CD	5A	F4
	C	1F	DD	A8	33	88	07	C7	31	B1	12	10	59	27	80	EC	5F
	D	60	51	7F	A9	19	B5	4A	0D	2D	E5	7A	9F	93	C9	9C	EF
	E	A0	E0	3B	4D	AE	2A	F5	B0	C8	EB	BB	3C	83	53	99	61
	F	17	2B	04	7E	BA	77	D6	26	E1	69	14	63	55	21	0C	7D

(b) Inverse S-box

128 bit Plaintext and Key

- **Plaintext: EA835CF00445332D655D98AD8596B0C5**
- **Key : AC7766F319FADC2128D12941575C006A.**

Note: AES reads/writes plaintext and key in column-wise order (top to bottom, left to right).

Step-1:Byte Substituition

x \ y	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	63	7C	77	7B	F2	6B	6F	C5	30	01	67	2B	FE	D7	AB	76
1	CA	82	C9	7D	FA	59	47	F0	AD	D4	A2	AF	9C	A4	72	C0
2	B7	FD	93	26	36	3F	F7	CC	34	A5	E5	F1	71	D8	31	15
3	04	C7	23	C3	18	96	05	9A	07	12	80	E2	EB	27	B2	75
4	09	83	2C	1A	1B	6E	5A	A0	52	3B	D6	B3	29	E3	2F	84
5	53	D1	00	ED	20	FC	B1	5B	6A	CB	BE	39	4A	4C	58	CF
6	D0	EF	AA	FB	43	4D	33	85	45	F9	02	7F	50	3C	9F	A8
7	51	A3	40	8F	92	9D	38	F5	BC	B6	DA	21	10	FF	F3	D2
8	CD	0C	13	EC	5F	97	44	17	C4	A7	7E	3D	64	5D	19	73
9	60	81	4F	DC	22	2A	90	88	46	EE	B8	14	DE	5E	0B	DB
A	E0	32	3A	0A	49	06	24	5C	C2	D3	AC	62	91	95	E4	79
B	E7	C8	37	6D	8D	D5	4E	A9	6C	56	F4	EA	65	7A	AE	08
C	BA	78	25	2E	1C	A6	B4	C6	E8	DD	74	1F	4B	BD	8B	8A
D	70	3E	B5	66	48	03	F6	0E	61	35	57	B9	86	C1	1D	9E
E	E1	F8	98	11	69	D9	8E	94	9B	1E	87	E9	CE	55	28	DF
F	8C	A1	89	0D	BF	E6	42	68	41	99	2D	0F	B0	54	BB	16

(a) S-box

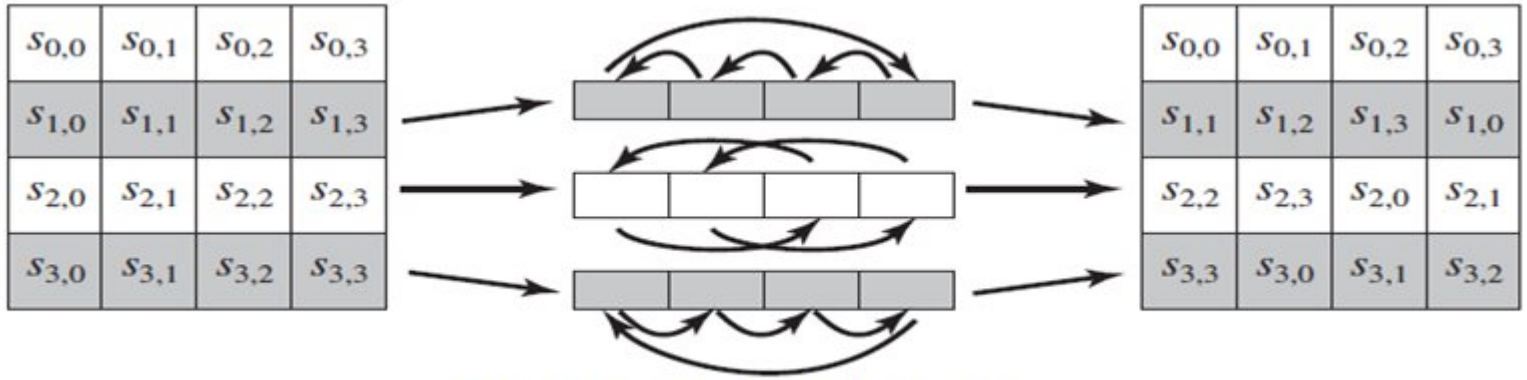
an example of the SubBytes transformation:

EA	04	65	85
83	45	5D	96
5C	33	98	B0
F0	2D	AD	C5

→

87	F2	4D	97
EC	6E	4C	90
4A	C3	46	E7
8C	D8	95	A6

Step 2: Shift Rows



(a) Shift row transformation

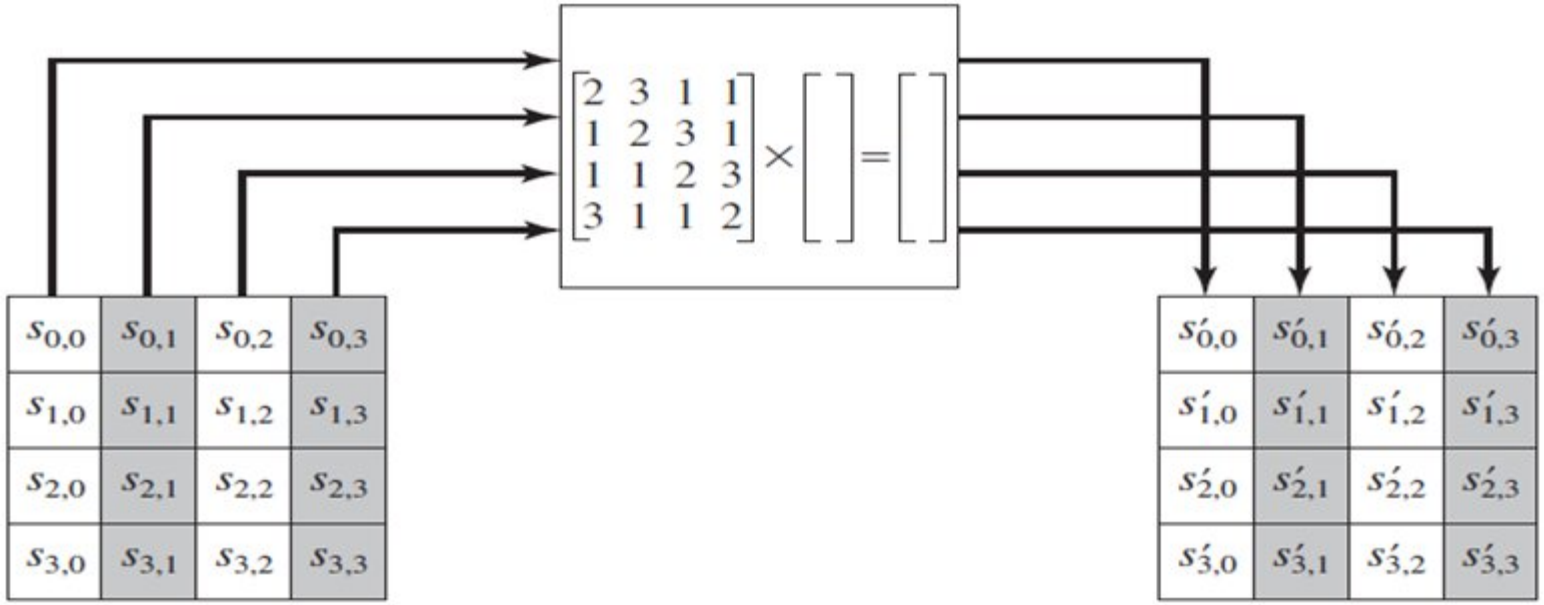
The following is an example of ShiftRows.

87	F2	4D	97
EC	6E	4C	90
4A	C3	46	E7
8C	D8	95	A6

→

87	F2	4D	97
6E	4C	90	EC
46	E7	4A	C3
A6	8C	D8	95

Step 3: Mix columns



(b) Mix column transformation

AES Mix Column

2	3	1	1
1	2	3	1
1	1	2	3
3	1	1	2

*

87	F2	4D	97
6E	4C	90	EC
46	E7	4A	C3
A6	8C	D8	95

=

?			

$\{02\} * \{87\} \oplus \{03\} * \{6E\} \oplus \{01\} * \{46\} \oplus \{01\} * \{A6\}$

$X^7 + X^6 + X^5 + X^4 + X^3 + X^2 + X^1 + 1$

$0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 1 \quad 0$

$02 = 0000\ 0010 = X$

$87 = 1000\ 0111 = X^7 + X^2 + X + 1$

$X^7 + X^6 + X^5 + X^4 + X^3 + X^2 + X^1 + 1$

$1 \quad 0 \quad 0 \quad 0 \quad 0 \quad 1 \quad 1 \quad 1$

$\{02\} * \{87\} = X * (X^7 + X^2 + X + 1)$

$= X^8 + X^3 + X^2 + X$

$= X^4 + X^3 + X + 1 + X^3 + X^2 + X$

$= X^4 + X^2 + 1$

$= 0001\ 0101$

Use irreducible Polynomial Theorem, GF (2^3)

$X^8 = X^4 + X^3 + X + 1$

$X^7 + X^6 + X^5 + X^4 + X^3 + X^2 + X^1 + 1$

$0 \quad 0 \quad 0 \quad 1 \quad 0 \quad 1 \quad 0 \quad 1$

AES Mix Column

2	3	1	1
1	2	3	1
1	1	2	3
3	1	1	2

*

87	F2	4D	97
6E	4C	90	EC
46	E7	4A	C3
A6	8C	D8	95

=

?			

$\{02\} * \{87\} \oplus \{03\} * \{6E\} \oplus \{01\} * \{46\} \oplus \{01\} * \{A6\}$

$02 = 0000\ 0010 = X$

$87 = 1000\ 0111 = X^7 + X^2 + X + 1$

$02 * 87 = X * (X^7 + X^2 + X + 1)$

$= X^8 + X^3 + X^2 + X$

$= X^4 + X^3 + X + 1 + X^3 + X^2 + X$

$= X^4 + X^2 + 1$

$= 0001\ 0101$

$03 = 0000\ 0011 = X + 1$

$6E = 0110\ 1110 = X^6 + X^5 + X^3 + X^2 + X$

$03 * 6E = (X+1) * (X^6 + X^5 + X^3 + X^2 + X)$

$= X^7 + X^6 + X^4 + X^3 + X^2 + X^6 + X^5 + X^3 + X^2 + X$

$= X^7 + X^5 + X^4 + X$

$X^7 + X^6 + X^5 + X^4 + X^3 + X^2 + X^1 + 1$

1 0 1 1 0 0 1 0

AES Mix Column

2	3	1	1
1	2	3	1
1	1	2	3
3	1	1	2

*

87	F2	4D	97
6E	4C	90	EC
46	E7	4A	C3
A6	8C	D8	95

=

?			

$X^7 + X^6 + X^5 + X^4 + X^3 + X^2 + X^1 + 1$

{02} * {87} ⊕ {03} * {6E} ⊕ {01} * {46} ⊕ {01} * {A6}

02 = 0000 0010 = X

87 = 1000 0111 = $X^7 + X^2 + X + 1$

02 * 87 = $X * (X^7 + X^2 + X + 1)$

= $X^8 + X^3 + X^2 + X$

= $X^4 + \cancel{X^3} + \cancel{X} + 1 + \cancel{X^3} + X^2 + \cancel{X}$

= $X^4 + X^2 + 1$

= 0001 0101

03 = 0000 0011 = X + 1

6E = 0110 1110 = $X^6 + X^5 + X^3 + X^2 + X$

03 * 6E = $(X+1) * (X^6 + X^5 + X^3 + X^2 + X)$

= $X^7 + \cancel{X^6} + X^4 + \cancel{X^3} + \cancel{X^2} + \cancel{X^6} + X^5 + \cancel{X^3} + \cancel{X^2} + X$

= $X^7 + X^5 + X^4 + X$

= 1011 0010

01 * 46 = 46 = 0100 0110

01 * A6 = A6 = 1010 0110

AES Mix Column

2	3	1	1
1	2	3	1
1	1	2	3
3	1	1	2

*

87	F2	4D	97
6E	4C	90	EC
46	E7	4A	C3
A6	8C	D8	95

=

47			

{02} * {87} ⊕ {03} * {6E} ⊕ {01} * {46} ⊕ {01} * {A6} = {47}

02 * 87 = 0 0 0 1 0 1 0 1
03 * 6E = 1 0 1 1 0 0 1 0
01 * 46 = 0 1 0 0 0 1 1 0
01 * A6 = 1 0 1 0 0 1 1 0

= 0 1 0 0 0 1 1 1 = {47}

4 7

In Ex-or
Odd time 1's = 1
Even time 1's = 0

AES Mix Column

2	3	1	1
1	2	3	1
1	1	2	3
3	1	1	2

*

87	F2	4D	97
6E	4C	90	EC
46	E7	4A	C3
A6	8C	D8	95

=

47			
?			

{01} * {87} ⊕ {02} * {6E} ⊕ {03} * {46} ⊕ {01} * {A6}

01 * 87 = 87 = 1000 0111

02 = 0000 0010 = X

6E = 0110 1110 = $X^6 + X^5 + X^3 + X^2 + X$

02 * 6E = X * ($X^6 + X^5 + X^3 + X^2 + X$)

= $X^7 + X^6 + X^4 + X^3 + X^2$

= 1101 1100

03 = 0000 0110 = X + 1

46 = 0100 0110 = $X^6 + X^2 + X$

03 * 46 = (X + 1) * ($X^6 + X^2 + X$)

= $X^7 + X^3 + X^2 + X^6 + X^2 + X$

= $X^7 + X^6 + X^3 + X$

= 1100 1010

01 * A6 = A6 = 1010 0110

AES Mix Column

2	3	1	1
1	2	3	1
1	1	2	3
3	1	1	2

*

87	F2	4D	97
6E	4C	90	EC
46	E7	4A	C3
A6	8C	D8	95

=

47			
37			

{01} * {87} ⊕ {02} * {6E} ⊕ {03} * {46} ⊕ {01} * {A6} = {37}

01 * 87 = 1 0 0 0 0 1 1 1
02 * 6E = 1 1 0 1 1 1 0 0
03 * 46 = 1 1 0 0 1 0 1 0
01 * A6 = 1 0 1 0 0 1 1 0

= 0 0 1 1 0 1 1 1 = {37}

3 7

AES Mix Column

2	3	1	1
1	2	3	1
1	1	2	3
3	1	1	2

*

87	F2	4D	97
6E	4C	90	EC
46	E7	4A	C3
A6	8C	D8	95

=

47			
37			
?			

$\{01\} * \{87\} \oplus \{01\} * \{6E\} \oplus \{02\} * \{46\} \oplus \{03\} * \{A6\}$

$01 * 87 = 87 = 1000\ 0111$

$01 * 6E = 6E = 0110\ 1110$

$02 = 0000\ 0010 = X$

$46 = 0100\ 0110 = X^6 + X^2 + X$

$02 * 46 = X * (X^6 + X^2 + X)$

$= X^7 + X^3 + X^2$

$= 1000\ 1100$

$03 = 0000\ 0110 = X + 1$

$A6 = 1010\ 0110 = X^7 + X^5 + X^2 + X$

$03 * A6 = (X + 1) * (X^7 + X^5 + X^2 + X)$

$= X^8 + X^6 + X^3 + X^2 + X^7 + X^5 + X^2 + X$

$= X^4 + X^3 + X + 1 + X^6 + X^3 + X^7 + X^5 + X$

$= X^7 + X^6 + X^5 + X^4 + 1$

$= 1111\ 0001$

Use irreducible Polynomial Theorem, GF (2^3)

$X^8 = X^4 + X^3 + X + 1$

AES Mix Column

2	3	1	1
1	2	3	1
1	1	2	3
3	1	1	2

*

87	F2	4D	97
6E	4C	90	EC
46	E7	4A	C3
A6	8C	D8	95

=

47			
37			
94			

{01} * {87} ⊕ {01} * {6E} ⊕ {02} * {46} ⊕ {03} * {A6} = {94}

01 * 87 = 1 0 0 0 0 1 1 1
01 * 6E = 0 1 1 0 1 1 1 0
02 * 46 = 1 0 0 0 1 1 0 0
03 * A6 = 1 1 1 1 0 0 0 1

= 1 0 0 1 0 1 0 0 = {94}

9 4

AES Mix Column

2	3	1	1
1	2	3	1
1	1	2	3
3	1	1	2

*

87	F2	4D	97
6E	4C	90	EC
46	E7	4A	C3
A6	8C	D8	95

=

47			
37			
94			
?			

$\{03\} * \{87\} \oplus \{01\} * \{6E\} \oplus \{01\} * \{46\} \oplus \{02\} * \{A6\}$

$03 = 0000\ 0011 = X + 1$

$87 = 1000\ 0111 = X^7 + X^2 + X + 1$

$03 * 87 = (X + 1) * (X^7 + X^2 + X + 1)$

$= \boxed{X^8} + X^3 + X^2 + X + X^7 + X^2 + X + 1$

$= X^4 + X^3 + X + 1 + X^3 + X^7 + 1$

$= X^7 + X^4 + X$

$= 1001\ 0010$

$01 * 6E = 6E = 0110\ 1110$

$01 * 46 = 46 = 0100\ 0110$

$02 = 0000\ 0010 = X$

$A6 = 1010\ 0110 = X^7 + X^5 + X^2 + X$

$02 * A6 = X * (X^7 + X^5 + X^2 + X)$

$= \boxed{X^8} + X^6 + X^3 + X^2$

$= X^4 + X^3 + X + 1 + X^6 + X^3 + X^2$

$= X^6 + X^4 + X^2 + X + 1$

Use irreducible Polynomial Theorem, GF

$\mathbf{X^8 = X^4 + X^3 + X + 1}$

AES Mix Column

2	3	1	1
1	2	3	1
1	1	2	3
3	1	1	2

*

87	F2	4D	97
6E	4C	90	EC
46	E7	4A	C3
A6	8C	D8	95

=

47			
37			
94			
?			

$\{03\} * \{87\} \oplus \{01\} * \{6E\} \oplus \{01\} * \{46\} \oplus \{02\} * \{A6\}$

$03 = 0000\ 0011 = X + 1$

$87 = 1000\ 0111 = X^7 + X^2 + X + 1$

$03 * 87 = (X + 1) * (X^7 + X^2 + X + 1)$

$= \boxed{X^8} + X^3 + \cancel{X^2} + \cancel{X} + X^7 + \cancel{X^2} + \cancel{X} + 1$

$= X^4 + \cancel{X^3} + X + \cancel{1} + \cancel{X^3} + X^7 + \cancel{1}$

$= X^7 + X^4 + X$

$= 1001\ 0010$

$01 * 6E = 6E = 0110\ 1110$

$01 * 46 = 46 = 0100\ 0110$

$02 = 0000\ 0010 = X$

$A6 = 1010\ 0110 = X^7 + X^5 + X^2 + X$

$02 * A6 = X * (X^7 + X^5 + X^2 + X)$

$= \boxed{X^8} + X^6 + X^3 + X^2$

$= X^4 + \cancel{X^3} + X + 1 + X^6 + \cancel{X^3} + X^2$

$= X^6 + X^4 + X^2 + X + 1$

$= 0101\ 0111$

Use irreducible Polynomial Theorem, GF (2^3)

$X^8 = X^4 + X^3 + X + 1$

AES Mix Column

2	3	1	1
1	2	3	1
1	1	2	3
3	1	1	2

*

87	F2	4D	97
6E	4C	90	EC
46	E7	4A	C3
A6	8C	D8	95

=

47			
37			
94			
ED			

{03} * {87} ⊕ {01} * {6E} ⊕ {01} * {46} ⊕ {02} * {A6} = {ED}

03 * 87 = 1 0 0 1 0 0 1 0
01 * 6E = 0 1 1 0 1 1 1 0
01 * 46 = 0 1 0 0 0 1 1 0
02 * A6 = 0 1 0 1 0 1 1 1

= 1 1 1 0 1 1 0 1 = {ED}

E D

Add Round Key

AES Add Round key

47	40	A3	4C
37	D4	70	9F
94	E4	3A	42
ED	A5	A6	BC

$\oplus$

AC	19	28	57
77	FA	D1	5C
66	DC	29	00
F3	21	41	6A

Key

=

?			

AES Add Round key

47	40	A3	4C
37	D4	70	9F
94	E4	3A	42
ED	A5	A6	BC

$\oplus$

AC	19	28	57
77	FA	D1	5C
66	DC	29	00
F3	21	41	6A

=

EB	59	8B	1B
40	2E	A1	C3
F2	38	13	42
1E	84	E7	D6

4C $\oplus$ 57 = ?

4C = 0100 1100

57 = 0101 0111

= 0001 1011

1 B = {1B}

9F $\oplus$ 5C = ?

9F = 1001 1111

5C = 0101 1100

= 1100 0011

C 3 = {C3}

42 $\oplus$ 00 = ?

42 = 0100 0010

00 = 0000 0000

= 0100 0010

4 2 = {42}

BC $\oplus$ 6A = ?

BC = 1011 1100

6A = 0110 1010

= 1101 0110

D 6 = {D6}

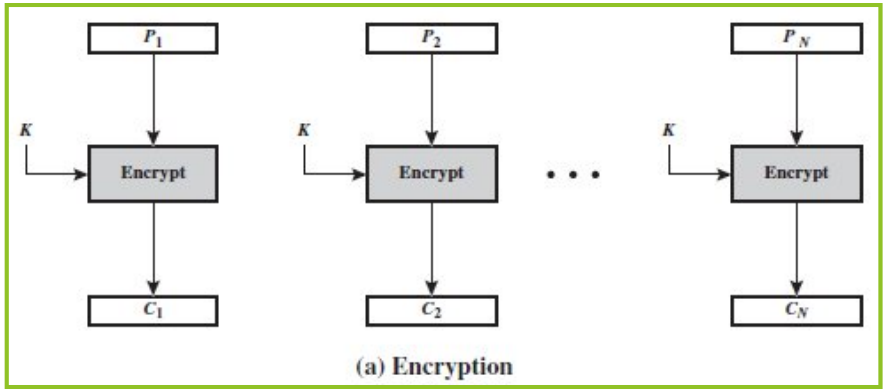
Block Cipher Modes of Operation

Block Cipher Modes of Operation

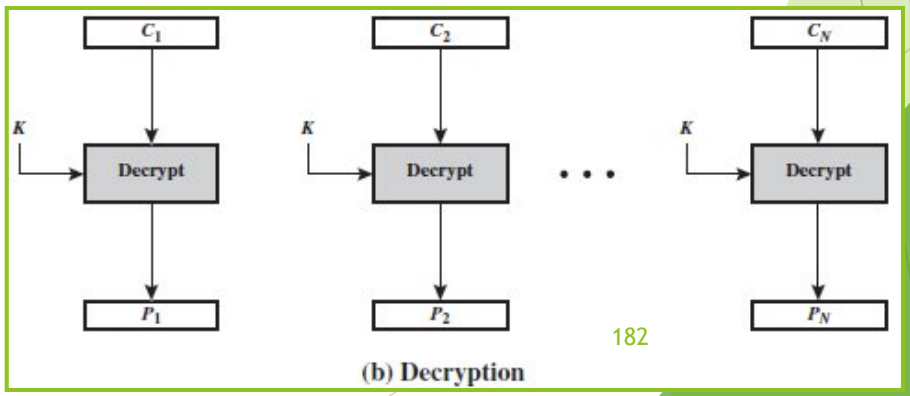
A block cipher takes a fixed-length block of text of length *b* bits and a key as input and produces a *b*-bit block of ciphertext. If the amount of plaintext to be encrypted is greater than ***b*** bits, then the block cipher can still be used by breaking the plaintext up into *b*-bit blocks. When multiple blocks of plaintext are encrypted using the same key, a number of security issues arise. To apply a block cipher in a variety of applications, five *modes of operation* have been defined by NIST.

a **mode of operation** is a technique for enhancing the effect of a cryptographic algorithm or adapting the algorithm for an application, such as applying a block cipher to a sequence of data blocks or a data stream.

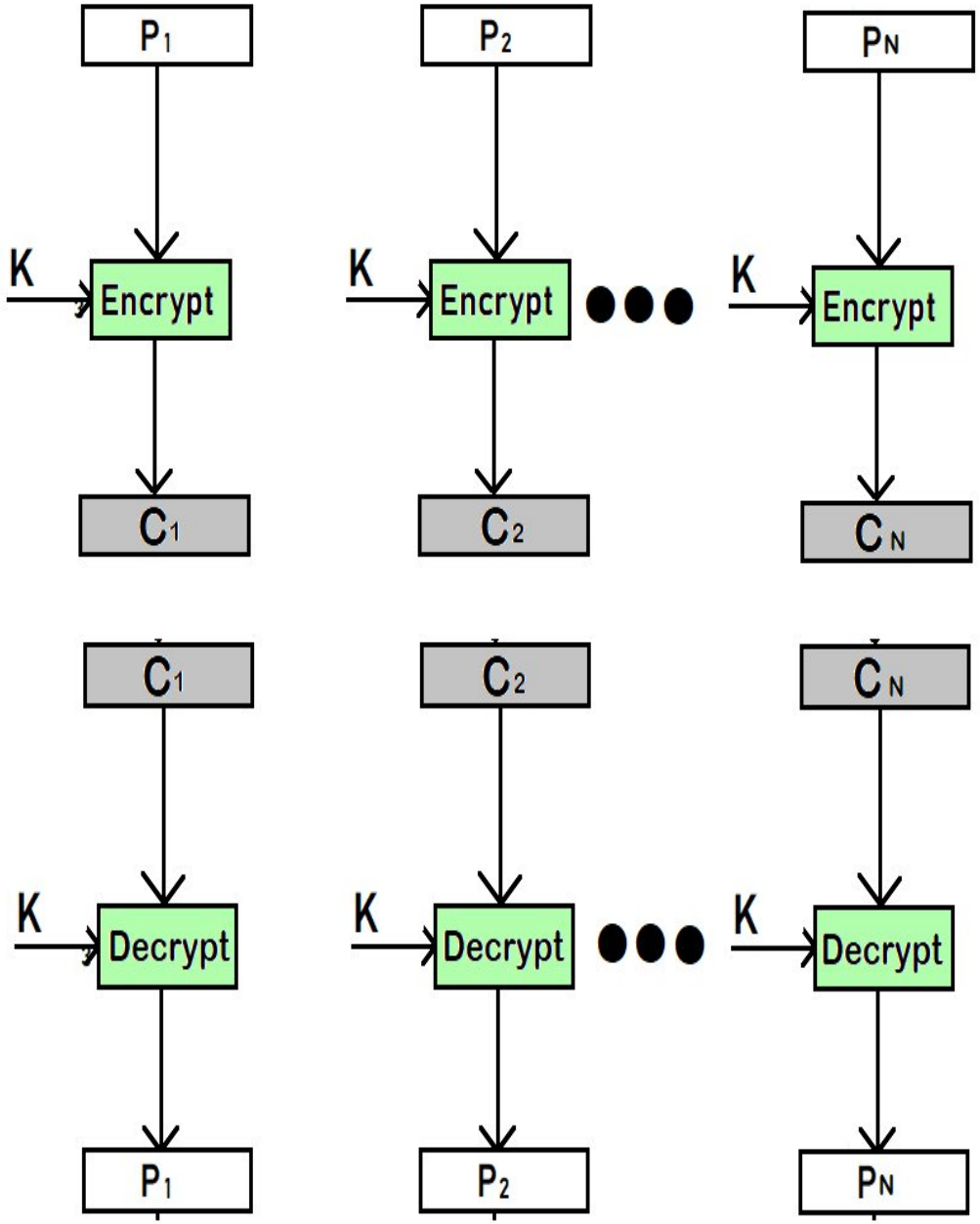
(1) Electronic code book (ECB): The simplest mode, in which plaintext is handled one block at a time and each block of plaintext is encrypted using the same key. The term *codebook* is used because, for a given key, there is a unique ciphertext for every *b*-bit block of plaintext. The ECB method is ideal for a short amount of data, such as an encryption key.



(a) Encryption



(b) Decryption


P1
P2
PN
K
Encrypt
K
Encrypt
K
Encrypt
C1
C2
CN
C1
C2
CN
K
Decrypt
K
Decrypt
K
Decrypt
P1
P2
PN

Electronic Code Book (ECB) Example

Consider the following example:

- **Plaintext:**
  Binary: 10100010001110101001 Divided into blocks of based on the size of the key
- **Key (K):**
  Length: 4 bits Example key: 1011 (in hexadecimal: B)
- **Encryption Process (using XOR):** Encrypt each block of plaintext (Pi) with the key (K). (Key is 4 bits , hence divide the plaintext into 4 bit blocks)
- **Plaintext block**: 1010 0010 0011 1010 1001 (If the divided plaintext blocks do not match the required block size defined by the key, the remaining bits in the final block are padded with repeated '0' bits to satisfy the block length)
- **Key (k):** 1011
- **Result XOR:**
  $1010 \oplus 1011 = 0001$

  $0010 \oplus 1011 = 1001$

  $0011 \oplus 1011 = 1000$

  $1010 \oplus 1011 = 0001$

  $1001 \oplus 1011 = 0010$
- **The final encrypted result is:**
  0001 1001 1000 0001 0010 (in binary)

  19812 (in hexadecimal)

What if the plaintext and Key is Hexadecimal and not in fixed size(not binary)

- Plaintext: 7A3F9C4E2B81
- Key : D4C1A

7A3F9 C4E2B 81000 (padding with “0” bits)

⊕

D4C1A D4C1A D4C1A

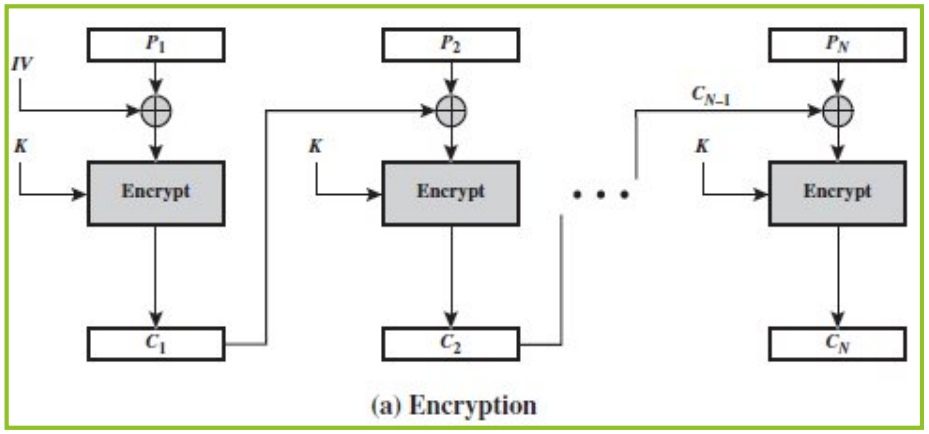
Cipher Text: AEFE3 10231 55C1A

Block Cipher Modes of Operation

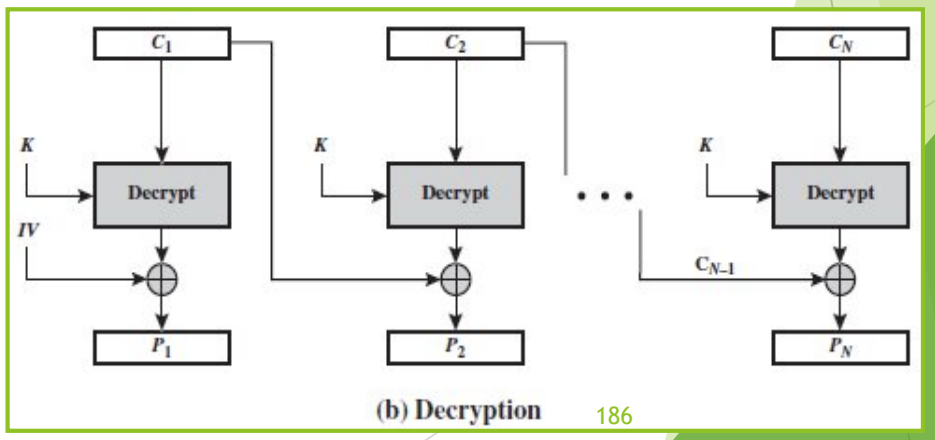
(2) Cipher Block Chaining Mode (CBC): In this scheme, the input to the encryption algorithm is the XOR of the current plaintext block and the preceding ciphertext block; the same key is used for each block. Therefore, if the same plaintext block is repeated, different ciphertext blocks are produced. For decryption, each cipher block is passed through the decryption algorithm. The result is XORed with the preceding ciphertext block to produce the plaintext block. We can define CBC mode as

CBC	$C_1 = \mathrm{E}(K, [P_1 \oplus \mathrm{IV}])$ $C_j = \mathrm{E}(K, [P_j \oplus C_{j-1}])\ j = 2, \ldots, N$	$P_1 = \mathrm{D}(K, C_1) \oplus \mathrm{IV}$ $P_j = \mathrm{D}(K, C_j) \oplus C_{j-1}\ j = 2, \ldots, N$

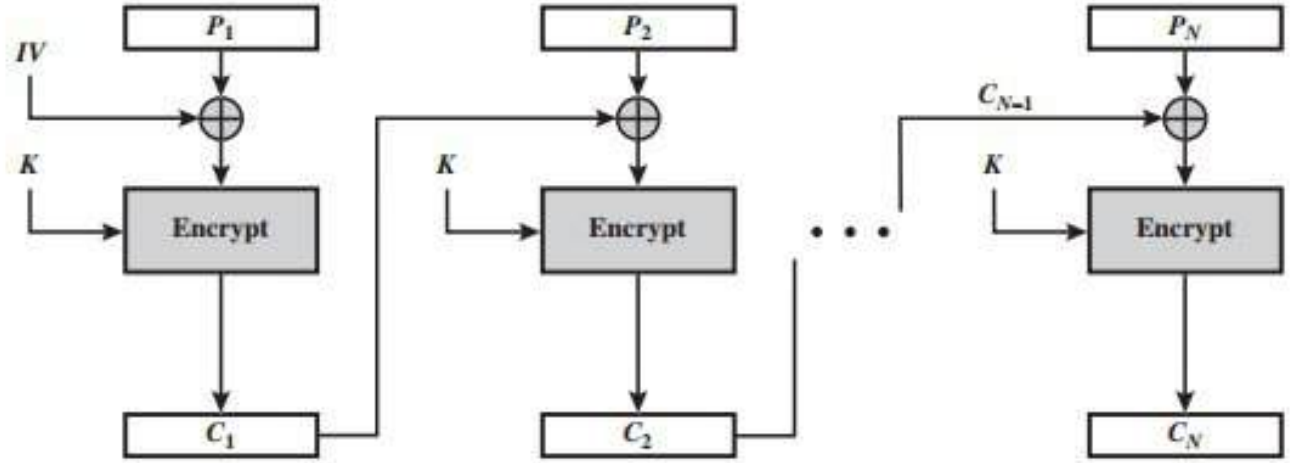
The **IV** is an initialization block, which is produced using random number generator and it should be the same size as the cipher block. This must be known to both the sender and receiver but it should be unpredictable by a third party.



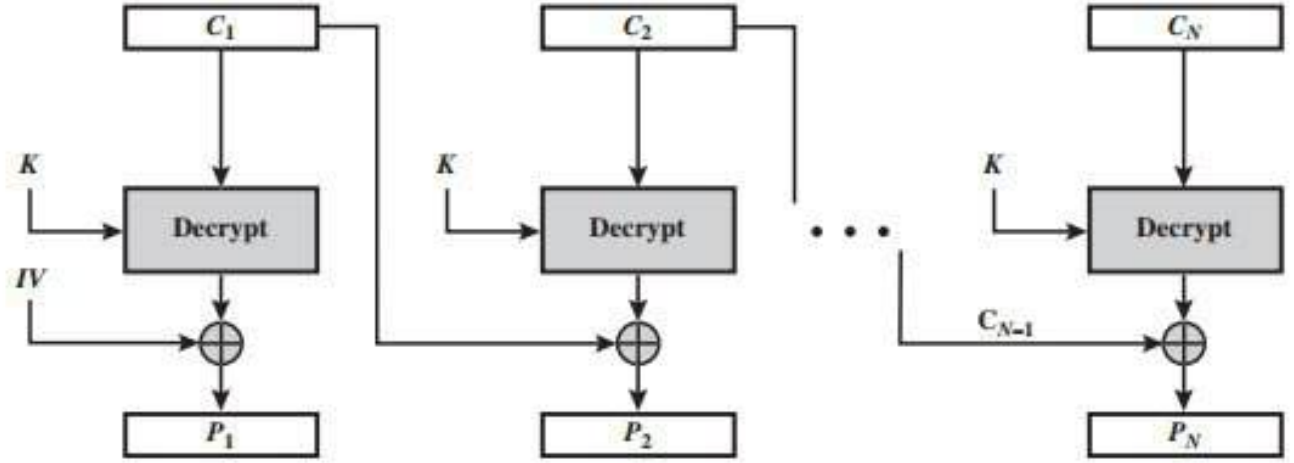
(a) Encryption



(b) Decryption



(a) Encryption



(b) Decryption

Example: Cipher Block Chaining (CBC)

Given

- **Block size = 4 bits**
- **Block cipher = permutation cipher**
- **Key (permutation) -** 3 4 2 1 (not binary)
- **IV = 1010**
- **Plaintext = 0100 1011 1100**

Step 2: For CBC encryption:

IV = 1010
C1 = E(P1 ⊕ IV)
C2 = E(P2 ⊕ C1)
C3 = E(P3 ⊕ C2)
Plaintext blocks:

- P1 = 0100
- P2 = 1011
- P3 = 1100

BLOCK 1

XOR with IV

P1 = 0100
IV = 1010

X1 = 1110

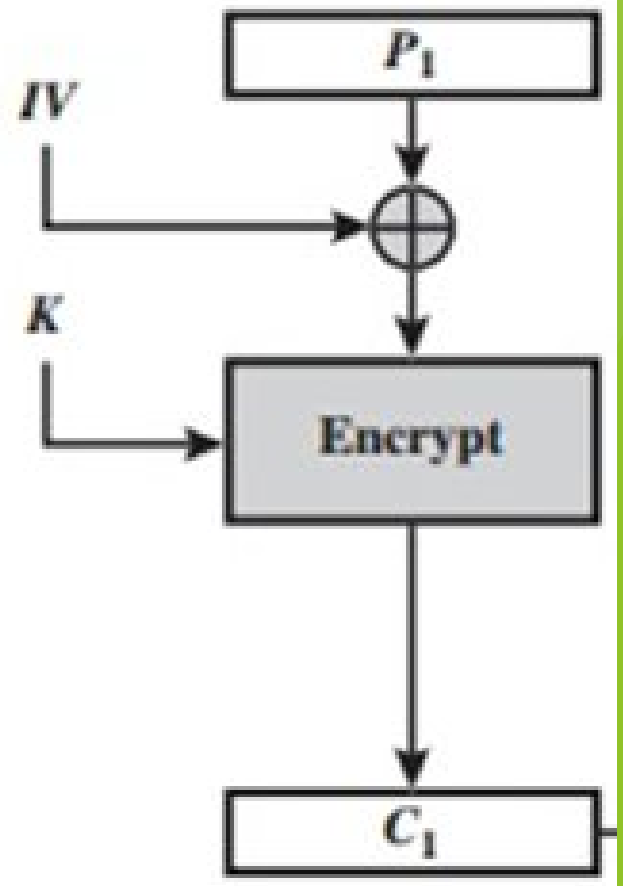
Encrypt X1

E(1110)
Bits: 1 1 1 0
Apply permutation (3,4,2,1) → 1011

Permutation

Input bits: a b c d
Output bits: c d b a
Examples for permutation
E(1010) = 1001
E(1101) = 0111
E(1100) = 0011

So
C1 = 1011



BLOCK 2

XOR with previous ciphertext

P2 = 1011
C1 = 1011

X2 = 0000

Encrypt X2

E(0000) → 0000
So
C2 = 0000

BLOCK 3

XOR with previous ciphertext

P3 = 1100
C2 = 0000

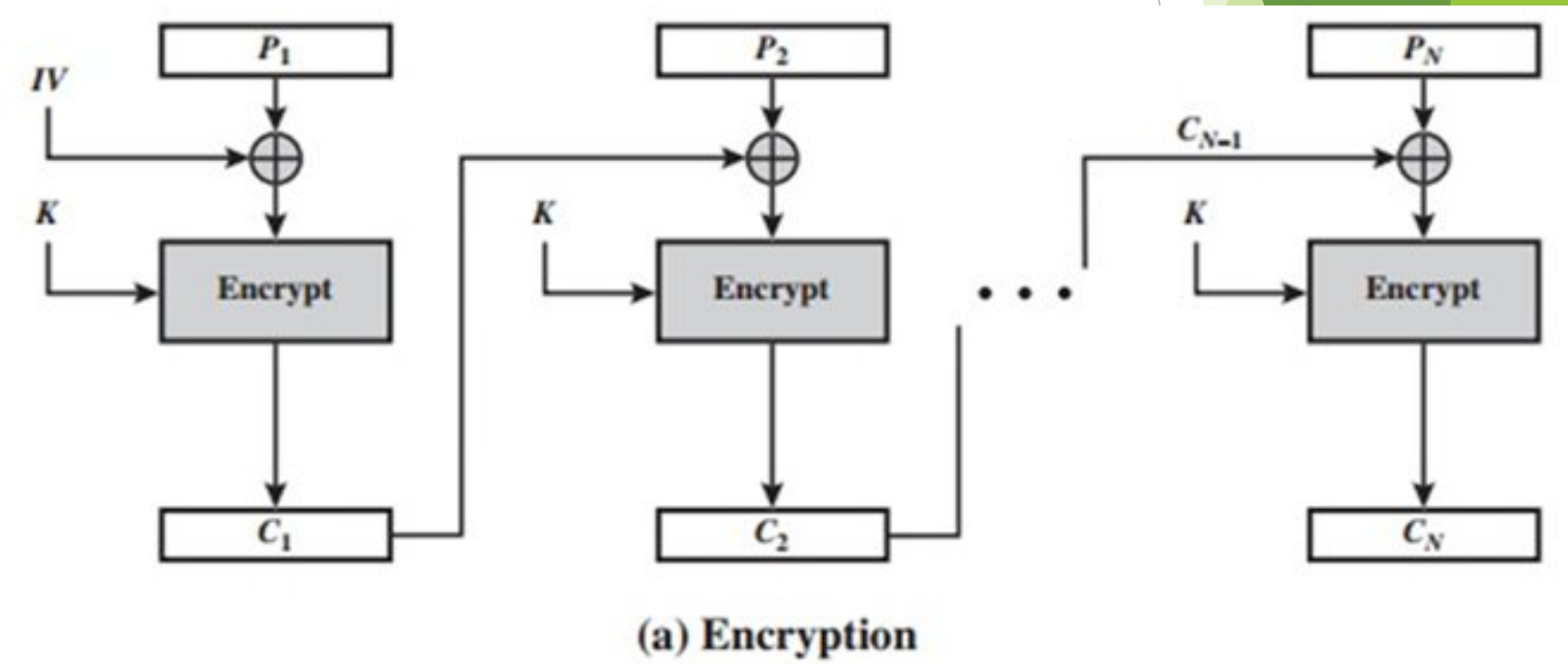
X3 = 1100

Encrypt X3

E(1100) → 0011
So
C3 = 0011

- C1 = 1011
- C2 =0000
- C3 = 0011

Ciphertext = 1011 0000 0011

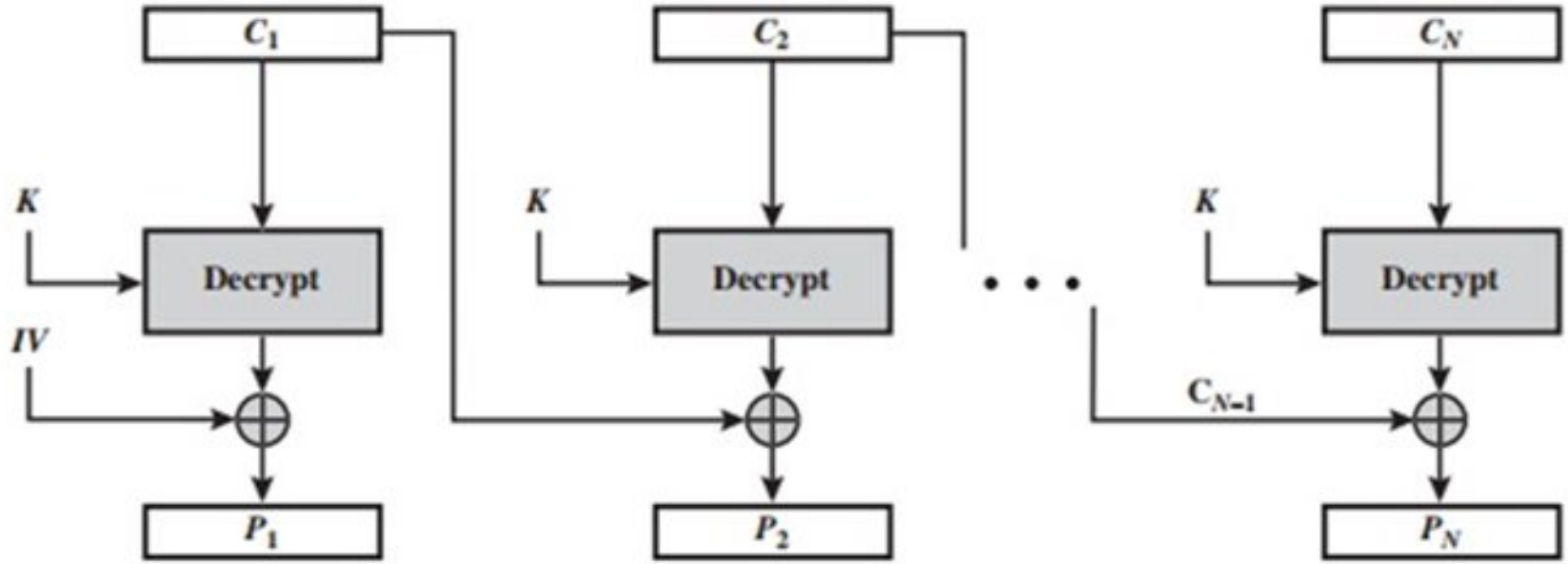


(a) Encryption

- C1 = 1011
- C2 =0000
- C3 = 0011

Ciphertext = 1011 0000 0011

- **Key (permutation) -** 3 4 2 1

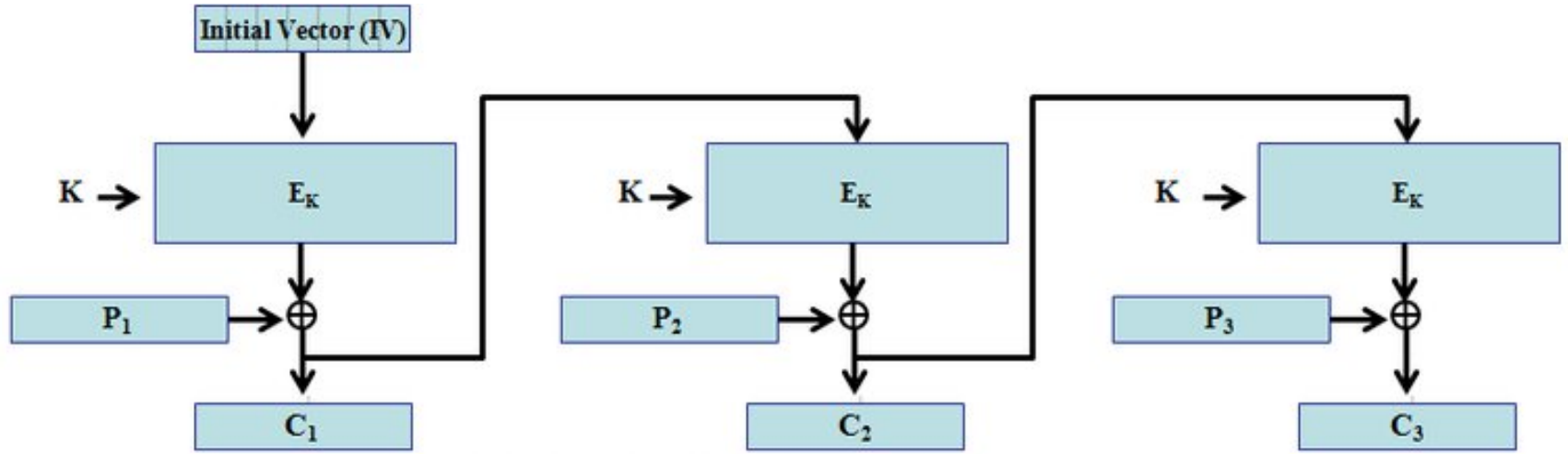


(b) Decryption

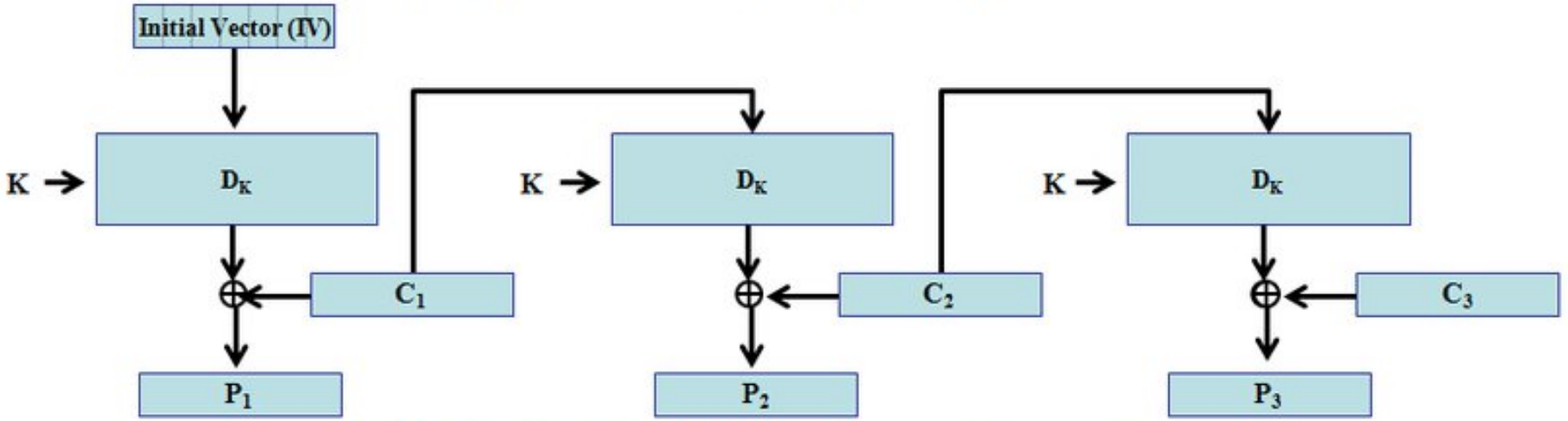
Block Cipher Modes of Operation

(3) Cipher Feedback Mode (CFB): In this scheme, the input is processed ***s*** bits at a time. The preceding ciphertext is used as input to the encryption algorithm to produce pseudorandom output, which is XORed with the plaintext to produce the next unit of ciphertext. We can define CFB mode as follows:

CFB	$I_1 = IV$			$I_1 = IV$	
	$I_j = \text{LSB}_{b-s}(I_{j-1}) \| C_{j-1}$	$j = 2, \ldots, N$		$I_j = \text{LSB}_{b-s}(I_{j-1}) \| C_{j-1}$	$j = 2, \ldots, N$
	$O_j = \text{E}(K, I_j)$	$j = 1, \ldots, N$		$O_j = \text{E}(K, I_j)$	$j = 1, \ldots, N$
	$C_j = P_j \oplus \text{MSB}_s(O_j)$	$j = 1, \ldots, N$		$P_j = C_j \oplus \text{MSB}_s(O_j)$	$j = 1, \ldots, N$



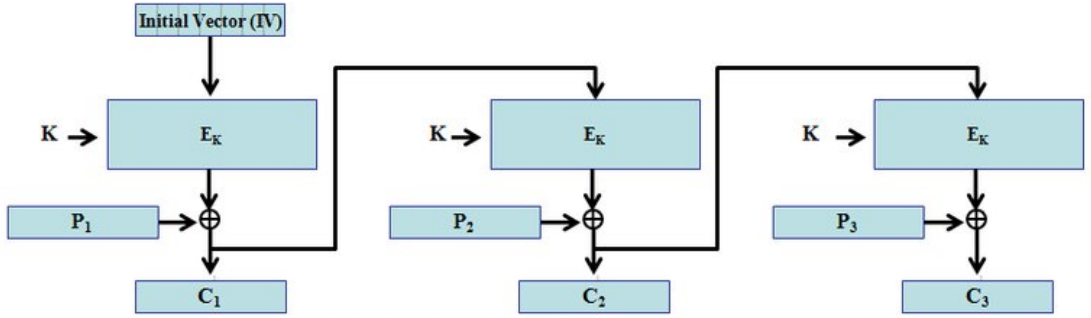
a) Cipher feedback (CFB) encryption mode



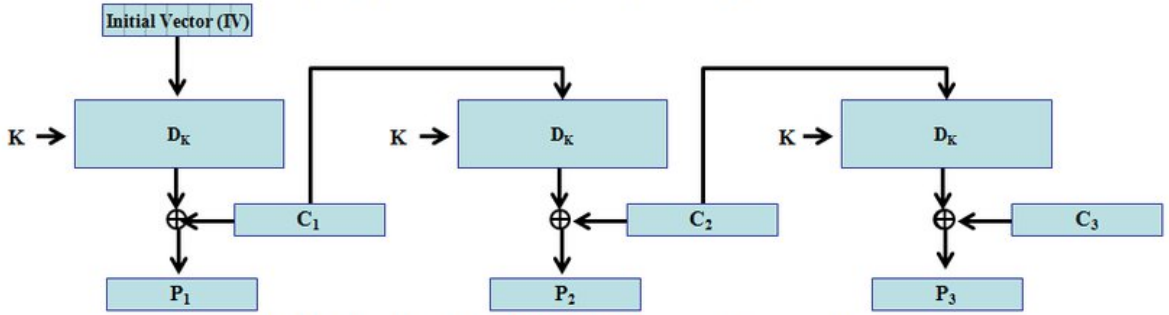
b) Cipher feedback (CFB) decryption mode

Given

- **Block size = 4 bits**
- **Block cipher = permutation cipher**
- **Key (permutation) -** 3 4 2 1
- **IV = 1010**
- **Plaintext = 0100 1011 1100**



a) Cipher feedback (CFB) encryption mode



b) Cipher feedback (CFB) decryption mode

Rule (since s=b=4):

- **$O_i = E(I_i)$**
- **$C_i = P_i \oplus O_i$**
- **Next input $I_{i+1} = C_i$** (because register length = 4 bits)

Block 1

- I1 = IV = **1010**
- O1 = E(1010) = **1001**
- C1 = P1 ⊕ O1 = 0100 ⊕ 1001 = **1101**

Block 2

- I2 = C1 = **1101**
- O2 = E(1101) = **0111**
- C2 = P2 ⊕ O2 = 1011 ⊕ 0111 = **1100**

Block 3

- I3 = C2 = **1100**
- O3 = E(1100) = **0011**
- C3 = P3 ⊕ O3 = 1100 ⊕ 0011 = **1111**

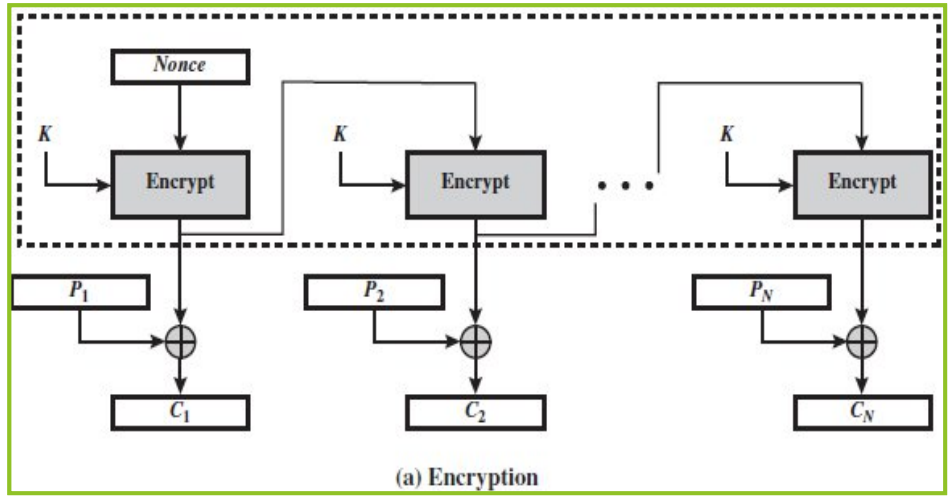
CFB ciphertext = C1 C2 C3 = 1101 1100 1111 = 110111001111

Block Cipher Modes of Operation

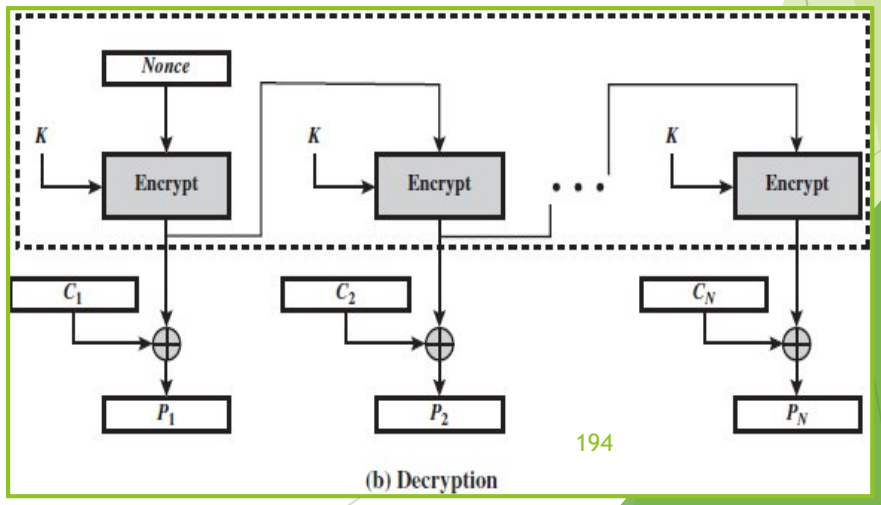
(4) Output Feedback Mode (OFB): This scheme operates on full blocks of plaintext and ciphertext where the output of the encryption function is fed back to become the input for encrypting the next block of plaintext. We can define OFB mode as follows:

OFB	$I_1 = Nonce$ $I_j = O_{j-1} \quad j = 2, \ldots, N$ $O_j = \mathrm{E}(K, I_j) \quad j = 1, \ldots, N$ $C_j = P_j \oplus O_j \quad j = 1, \ldots, N-1$ $C_N^* = P_N^* \oplus \mathrm{MSB}_u(O_N)$	$I_1 = Nonce$ $I_j = O_{j-1} \quad j = 2, \ldots, N$ $O_j = \mathrm{E}(K, I_j) \quad j = 1, \ldots, N$ $P_j = C_j \oplus O_j \quad j = 1, \ldots, N-1$ $P_N^* = C_N^* \oplus \mathrm{MSB}_u(O_N)$

Let the size of a block be ***b***. If the last block of plaintext contains ***u*** bits, with ***u*** < ***b***, the most significant ***u*** bits of the last output block O_N are used for the XOR operation. In the case of OFB, the IV must be a nonce; that is, the IV must be unique to each execution of the

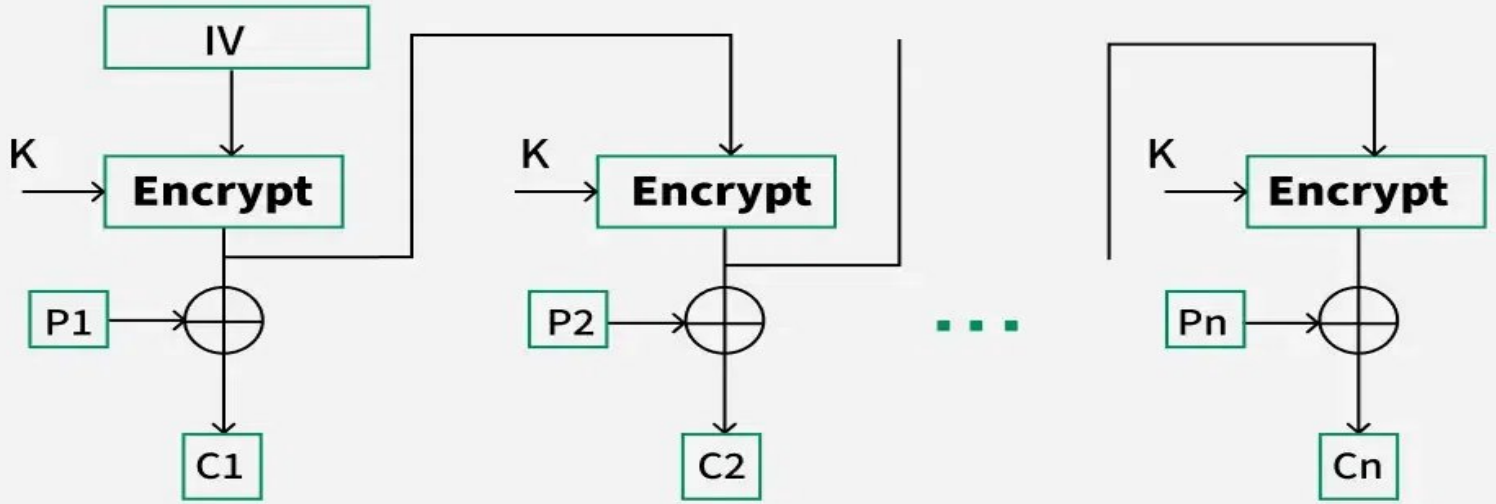


(a) Encryption

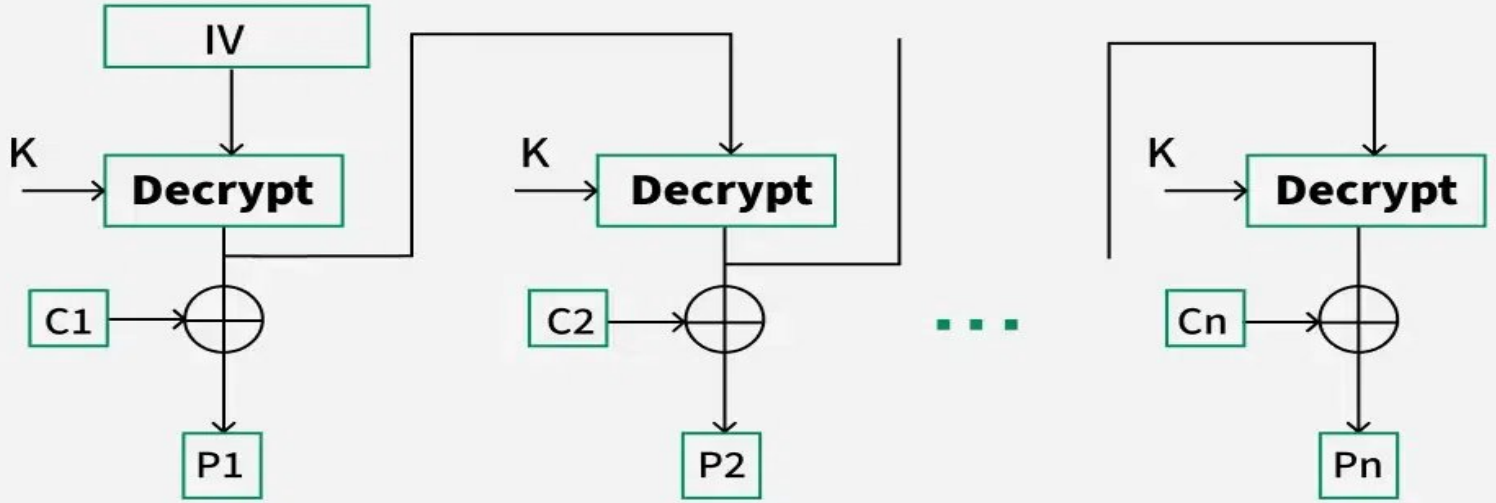


(b) Decryption

Encyrption

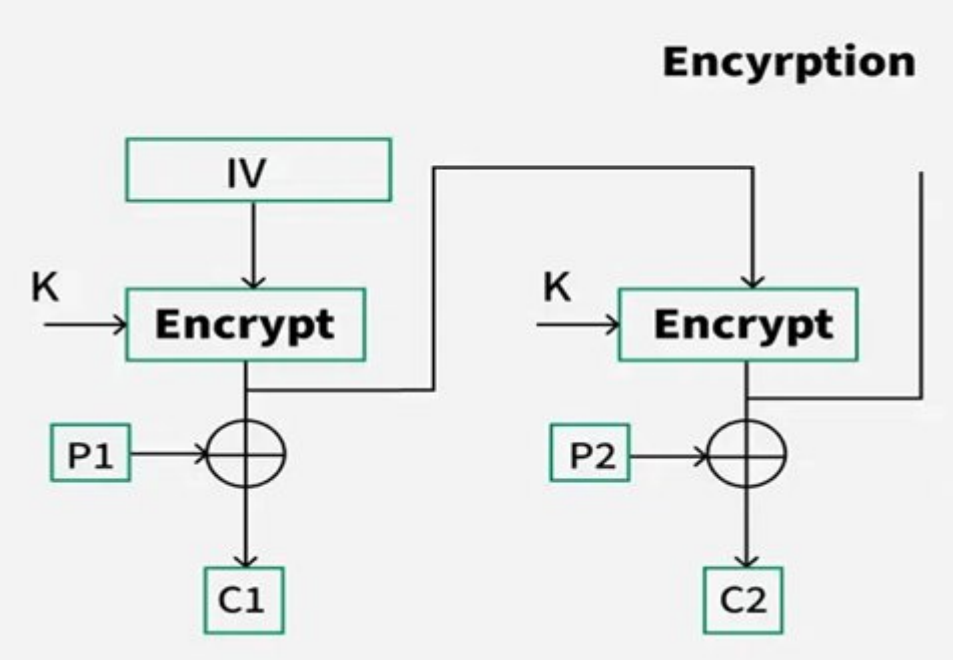


Decyrption



Given

- **Block size = 4 bits**
- **Block cipher = permutation cipher**
- **Key (permutation) -** 3 4 2 1
- **IV = 1010**
- **Plaintext = 0100 1011 1100**



Block 1

- **O1 = E(IV) = E(1010)**
- E(1010) = **1001**

Ciphertext

- **C1 = P1 ⊕ O1 = 0100 ⊕ 1001 = 1101**
  C1 = 1101

Block 2

Next keystream block

- **O2 = E(O1) = E(1001)**
- E(1001):
  - input 1 0 0 1 → output x3 x4 x2 x1 = 0 1 0 1 = **0101 = O2**

Ciphertext

- **C2 = P2 ⊕ O2 = 1011 ⊕ 0101 = 1110**
  C2 = 1110

Block 3

Next keystream block

- **O3 = E(O2) = E(0101)**
- E(0101):
  - input 0 1 0 1 → output = x3 x4 x2 x1 = 0 1 1 0 = **0110=O3**

Ciphertext

- **C3 = P3 ⊕ O3 = 1100 ⊕ 0110 = 1010**
  C3 = 1010

C1 = 1101, C2 = 1110, C3 = 1010

OFB ciphertext = 110111101010

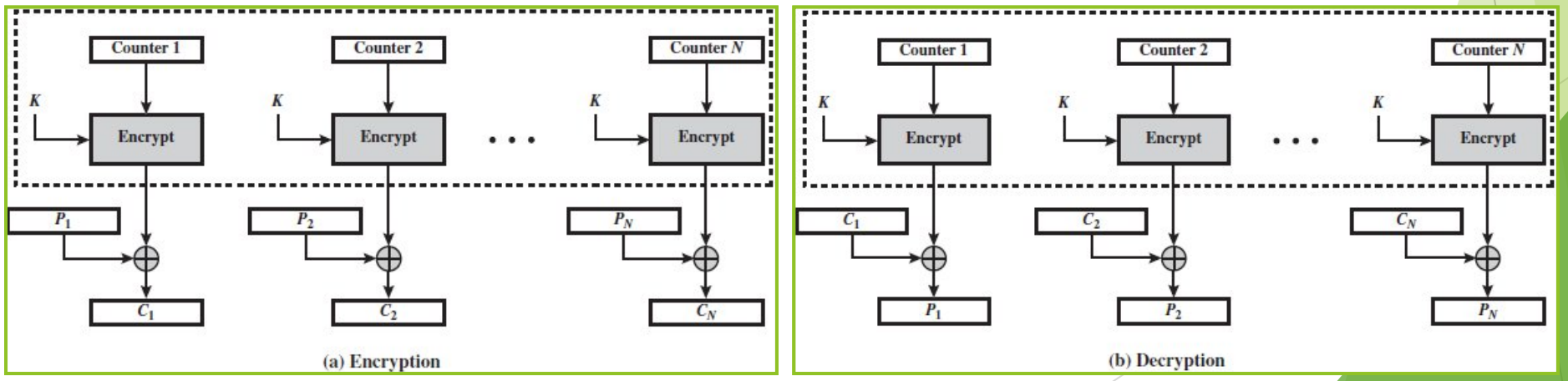
Block Cipher Modes of Operation

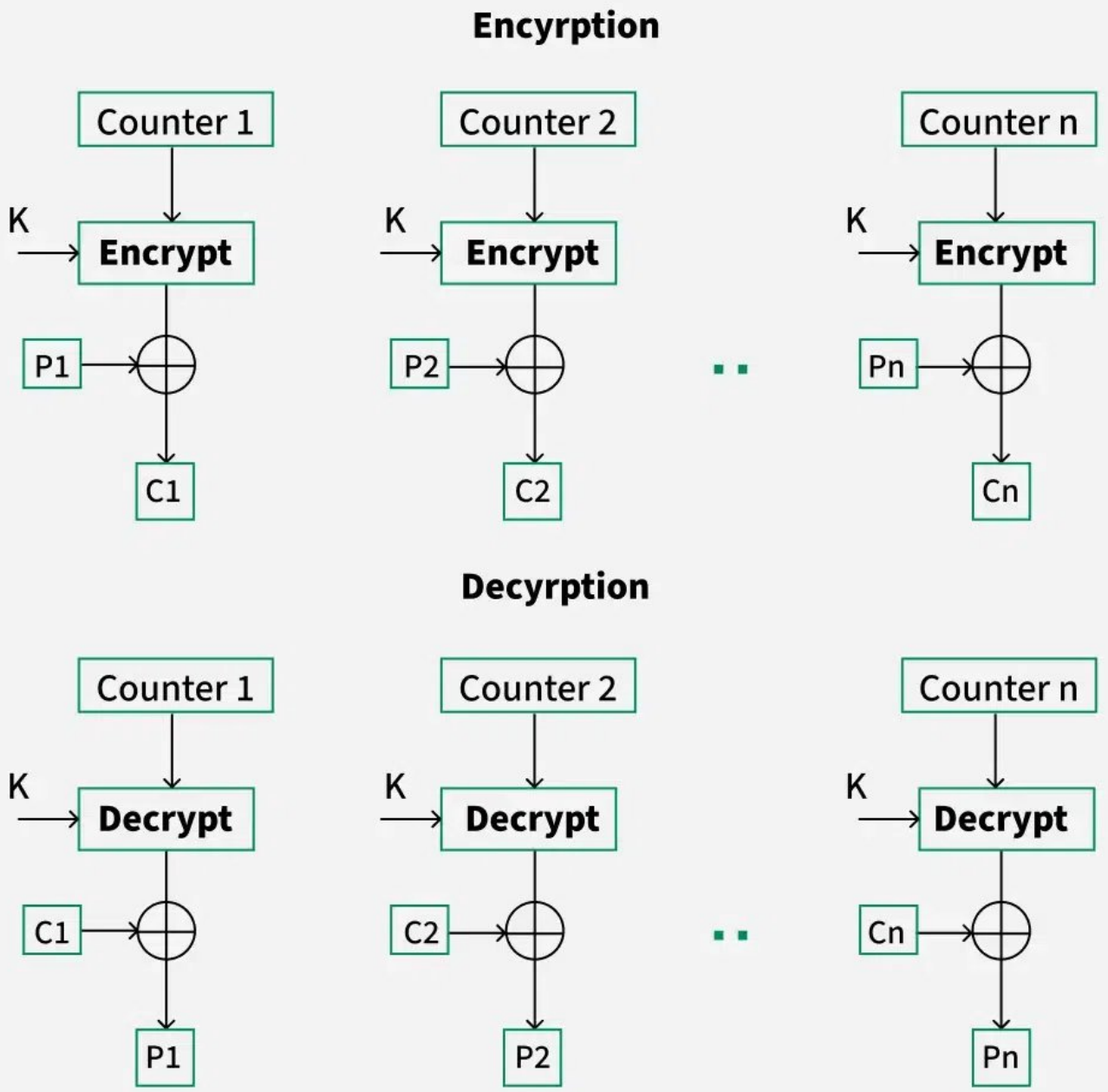
* One advantage of the OFB method is that bit errors in transmission do not propagate. The disadvantage of OFB is that it is more vulnerable to a message stream modification attack than in CFB.

(5) Counter Mode (CTR): In this mode, each block of plaintext is XORed with an encrypted counter. Typically, the counter is initialized to some value and then incremented by 1 for each subsequent block being encrypted using the same key. Given a sequence of counters T_1, T_2, …, T_N, we can define CTR mode as follows:

CTR	$C_j = P_j \oplus \mathrm{E}(K, T_j) \quad j = 1, \ldots, N-1$ $C_N^* = P_N^* \oplus \mathrm{MSB}_u[\mathrm{E}(K, T_N)]$	$P_j = C_j \oplus \mathrm{E}(K, T_j) \quad j = 1, \ldots, N-1$ $P_N^* = C_N^* \oplus \mathrm{MSB}_u[\mathrm{E}(K, T_N)]$

The advantages of the CTR are (1) hardware and software efficiency, (2) preprocessing, (3) random access, (4) provable security and (5) simplicity.




Encyrption
Counter 1
Counter 2
Counter n
K
Encrypt
P1
P2
Pn
C1
C2
Cn
Decyrption
Counter 1
Counter 2
Counter n
K
Decrypt
C1
C2
Cn
P1
P2
Pn

Given
Block size = **4 bits**
•Plaintext = `0100 1011 1100`
- **P1 = 0100**
- **P2 = 1011**
- **P3 = 1100**

•We'll use the same **IV = 1010** as the **initial counter value** (CTR usually calls this nonce/counter start).
•Block cipher: permutation (3,4,2,1)
Encryption function:
If input is **x1 x2 x3 x4**, then
E(x) = x3 x4 x2 x1

CTR Mode Rule
For each block:
1.Generate keystream block: **Ki = E(CTR_i)**
2.Encrypt: **Ci = Pi ⊕ Ki**
3.Increment counter: **CTR_{i+1} = CTR_i + 1 (mod 16)** (because 4-bit counter)

Step 1: Counters
Start:
•**CTR1 = 1010** (given IV)
Increment by 1 each time:
•**CTR2 = 1011**
•**CTR3 = 1100**

Block 1
Keystream
•CTR1 = 1010
•**K1 = E(1010) = 1001**

XOR with plaintext
```
P1 = 0100
K1 = 1001
---------
C1 = 1101
```

So **C1 = 1101**

Block 2

CTR2 = 1011

Compute **E(1011)** = **1101**
So **K2 = 1101**

XOR with plaintext

```
P2 = 1011
K2 = 1101
---------
C2 = 0110
```

So **C2 = 0110**

Block 3
Keystream

•CTR3 = 1100
Compute **E(1100)**:= **0011**
So **K3 = 0011**

XOR with plaintext

```
P3 = 1100
K3 = 0011
---------
C3 = 1111
```

So **C3 = 1111**

CTR ciphertext = 1101 0110 1111 = 110101101111

Block Cipher Modes of Operation

Mode	Description	Typical Application
Electronic Codebook (ECB)	Each block of plaintext bits is encoded independently using the same key.	• Secure transmission of single values (e.g., an encryption key)
Cipher Block Chaining (CBC)	The input to the encryption algorithm is the XOR of the next block of plaintext and the preceding block of ciphertext.	• General-purpose block-oriented transmission • Authentication
Cipher Feedback (CFB)	Input is processed *s* bits at a time. Preceding ciphertext is used as input to the encryption algorithm to produce pseudorandom output, which is XORed with plaintext to produce next unit of ciphertext.	• General-purpose stream-oriented transmission • Authentication
Output Feedback (OFB)	Similar to CFB, except that the input to the encryption algorithm is the preceding encryption output, and full blocks are used.	• Stream-oriented transmission over noisy channel (e.g., satellite communication)
Counter (CTR)	Each block of plaintext is XORed with an encrypted counter. The counter is incremented for each subsequent block.	• General-purpose block-oriented transmission • Useful for high-speed requirements

Tips to save time

- Write the Hexadecimal to binary conversion from “0”(0000) to “F”(1111) before you start the exam which is useful for DES, S-DES, AES mix columns and Block modes.
- Prepare well on the Extended Euclidean algorithm to find GCD asap.
- Try your own shortcuts to find GCD for large numbers.
- Try your own shortcuts to perform modulus for large numbers.